KAMP 제조AI데이터셋 활용 권한

No. GB20261102-PU1793622753-406461

⚠ 제조AI데이터셋 활용 주의사항

연구/공식적으로 사용하실 때에는 꼭 아래와 같이 'KAMP 출처(reference)'를 남겨주시고,
인용 시 활용한 내용과 문서 등은 아래 이메일로 보내주시기를 바랍니다.
(E-mail : kamp@kaist.ac.kr)

국문 출처 표기 양식

중소벤처기업부, Korea AI Manufacturing Platform(KAMP), 회전기계 고장유형 AI 데이터셋, KAIST(기계공학과 이필승교수), 2021.12.27., www.kamp-ai.kr

영문 출처 표기 양식

Ministry of SMEs and Startups., and KAIST(Korea Advanced Institute of Science and Technology). (2021, December 27). Rotary machine failure type AI dataset. Korea AI Manufacturing Platform(KAMP). https://www.kamp-ai.kr/

제조AI데이터셋 개요

제조AI데이터셋명 : 회전기계 고장유형 AI 데이터셋
업종 : 기타 | **목적 :** 예지보전 | **유형 :** csv
알고리즘 : 심층신경망(DNN), 합성곱신경망(CNN), 순환신경망(RNN)
사용조건 : 콘텐츠 변경허용
제공기관 : KAIST (기계공학과 이필승교수)
등록일 : 2021-12-27

이용자 정보

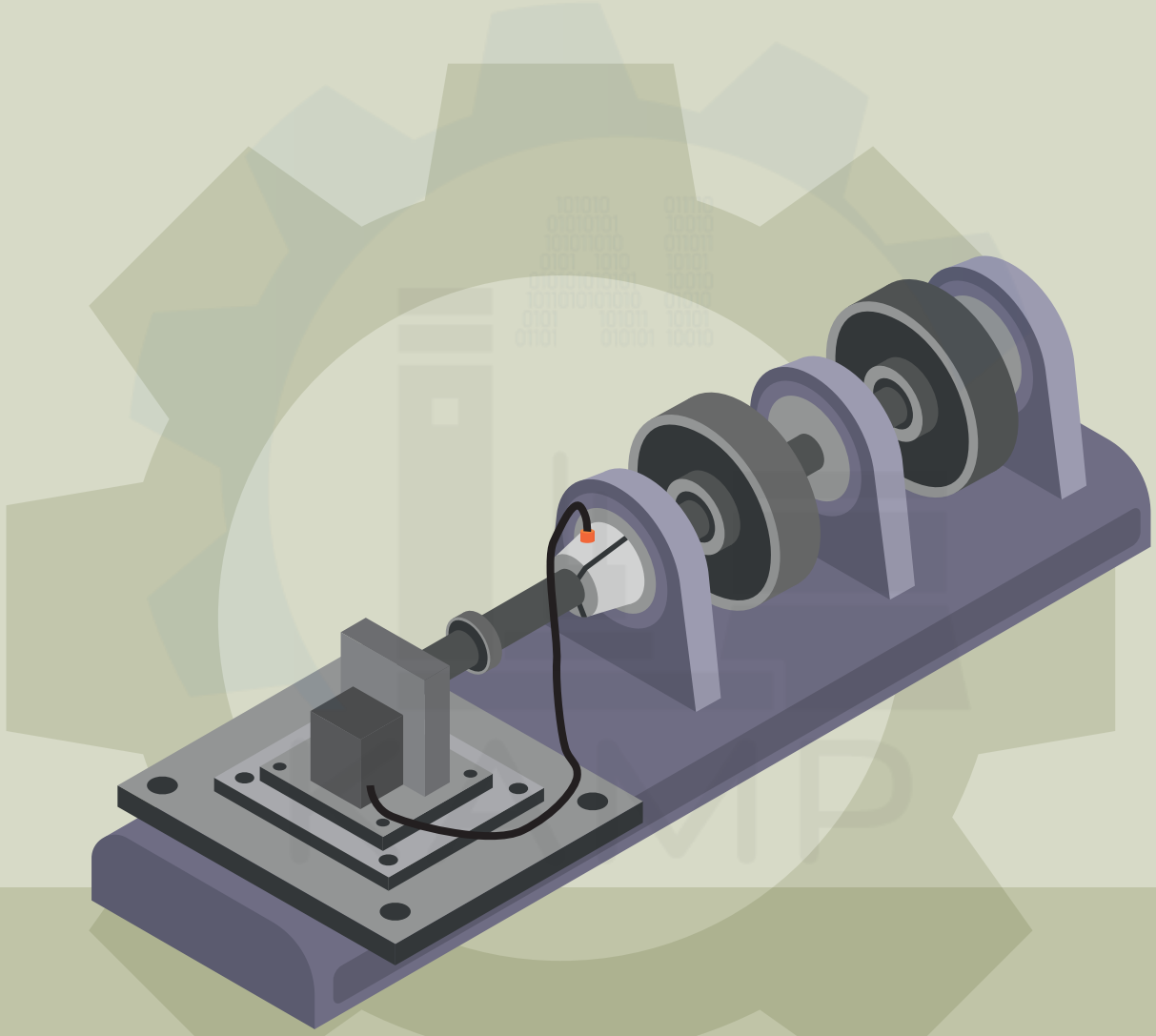
회원 ID : MSS_gukastan
다운로드일 : 2026-11-02 21:34:07
활용목적 : 제조AI 교육 및 강의

인공지능 제조 플랫폼(KAMP, Korea AI Manufacturing Platform)에서 다운로드한 제조AI데이터셋의 저작권을 위하여 문서 고유 번호와 워터마크를 부여하여, 해당 문서의 표준을 준수하였음을 증명합니다.

운영기관 : KAIST 제조AI빅데이터센터
주소 : 대전광역시 유성구 문지로 193 KAIST 문지캠퍼스 행정동
전화번호 : 042-350-1331 | 이메일 : kamp@kaist.ac.kr

KAMP
인공지능 제조 플랫폼

「회전기계 고장유형 AI 데이터셋」 분석실습 가이드북

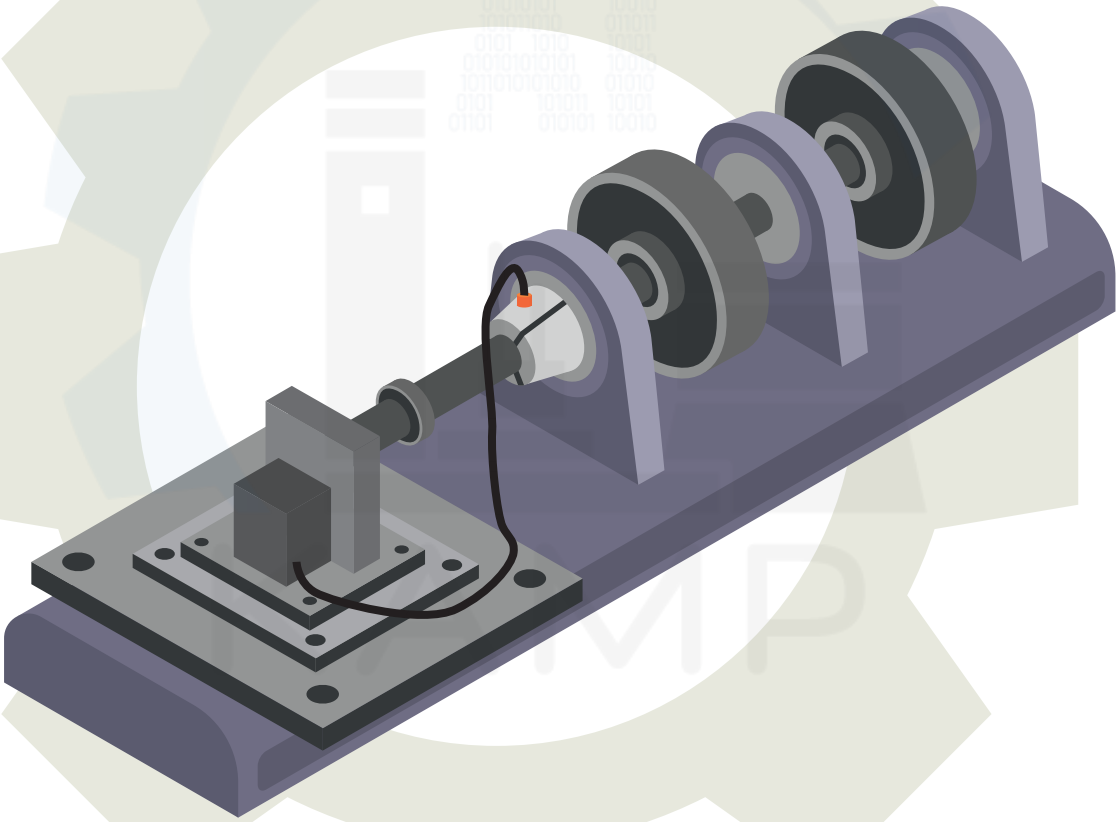


중소벤처기업부

KOSMO
스마트제조혁신추진단

KAIST

「회전기계 고장유형 AI 데이터셋」 분석실습 가이드북



중소벤처기업부 KOSMO 스마트제조혁신추진단 KAIST

Contents

1 분석요약 04

2 분석 실습 1. 분석 개요 05

1.1 분석 배경 05

1) 공정(설비) 개요

2) 이슈사항(Pain point)

1.2 분석 목표 07

1) 분석 목표

2) 데이터 정의 및 소개

3) 데이터 분석 기대효과

4) 시사점(implication) 요약기술

2. 분석실습 08

2.1 제조데이터 소개 08

1) 데이터 수집 방법

2) 데이터 유형/구조

3) 데이터 (품질) 전처리

2.2 분석 모델 소개 17

1) 데이터 흐름 및 인공지능 모델 적용 흐름도

2) AI 분석모델

2.3 분석 체험 24

1) 필요 SW, 패키지 설치 방법 및 절차 가이드

2) 분석 단계별 프로세스 – Flow Chart

[단계 ①] 라이브러리 / 데이터 불러오기

[단계 ②] 데이터 확인

[단계 ③] 데이터 특성 파악

[단계 ④] 데이터 전처리

[단계 ⑤] 데이터 분배(학습 / 검증 / 테스트)

[단계 ⑥] AI 모델 구축

[단계 ⑦] AI 모델 학습

[단계 ⑧] AI 모델 평가

3. 유사 타 현장의 「회전기계 고장유형 AI 데이터셋」 분석 적용 60

부 록 분석환경 구축을 위한 설치 가이드 63

데이터 품질 전처리 [실습 코드] 82

「회전기계 고장유형 AI 데이터셋」 분석실습 가이드북

- **필 요 SW :** Python, Anaconda - Jupyter notebook 등
- **필요 패키지 :** Pytorch, pandas, seabeorn, scikit-learn, matplotlib
- **분 석 환 경 :** [CPU]i7-8700, [RAM]16GB
- **필요 데이터 :** raw_data 폴더(g1_sensor#.csv, g2_sensor#.csv ; # = 1-4)

1 분석요약

No	구 분		내 용
1	**분석 목적** (현장 이슈, 목적)		- 산업 현장에서 사용되는 기계 구조물은 대부분 정상 상태로 가동되며, 수집되는 데이터 역시 정상 유형이 90% 이상을 차지한다. 고장유형의 경우 자연 발생적으로 취득되는 데이터가 매우 희박하고, 실제 구조물에 인위적인 고장을 가하여 데이터를 취득하는 방식은 비효율적이므로, 이는 데이터 간 불균형 문제를 초래할 수 있다. - Rotor testbed를 이용하면, 여러 유형의 고장에 대해 특징을 분석하고, 정상 및 고장의 유형을 분류할 수 있는 충분한 양의 고장 데이터를 수집할 수 있다. - 많은 산업 현장(발전소 등)에서 주로 사용되는 회전 설비의 진동데이터를 수집하고, 고장을 분석 및 진단하는 학습 모델을 구축하여, 이후 기술 확장을 통해 발전소나, 대규모 공장의 자동화 라인 등에 활용을 목표로 한다.
2	**데이터셋 형태 및 수집방법**		**1) 분석에 사용된 변수 :** Sensor 1-4(Z 방향 진동데이터, m/s^2) **2) 데이터 수집 방법 :** Signal link에서 제작한 rotor testbed **3) 데이터셋 파일 확장자 :** *.csv
3	**데이터 개수** **데이터셋 총량**		- **데이터 개수 :** 약 1,500,000개(2 sensor groups) - **데이터셋 총량 :** 73.0 MB
4	**분석적용 알고리즘**	**알고리즘**	심층신경망(DNN), 합성곱신경망(CNN), 순환신경망(RNN)
		알고리즘 간략소개	- 심층신경망(DNN)은 인공신경망에서 모델 내 은닉층을 많이 늘려서 학습의 결과를 향상시키는 방법이다. - 합성곱신경망(CNN)은 인공신경망의 한 종류이며, 주로 Matrix 데이터나 이미지 데이터에 대하여 특징을 추출하는 데에 쓰인다. - 순환신경망(RNN)은 인공신경망 종류 중 하나로 과거의 학습을 가중치를 통해 현재 학습에 반영하는 특징을 가지기 때문에, 진동데이터와 같은 순차적인 데이터(sequential data) 학습에 사용된다.
5	**분석결과 및 시사점**		- Rotor testbed를 이용하여 정상 및 고장 데이터셋을 구축하고, 전처리된 데이터를 학습하여 고장을 검출, 진단하는 인공지능 모델을 구축하였다. - 개발된 인공지능 모델은 기술 확장을 통해, 진동데이터를 모니터링하는 산업 현장에서 회전 설비의 정상/비정상 진단에 활용될 것으로 기대한다.

2 분석 실습

1. 분석 개요

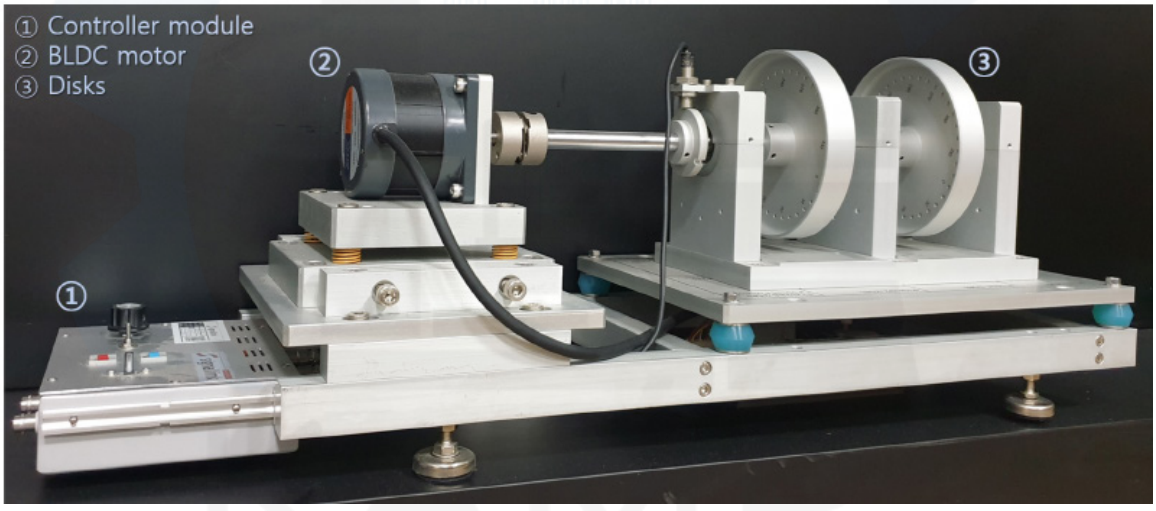
1.1 분석 배경

1) 공정(설비) 개요

• 발전소나 공장과 같은 대부분의 산업 현장에서 회전기계는 터빈, 펌프, 전동기, 압축기, 송풍기 등 다양한 형태로 존재하며, 주요 설비로 사용되고 있다. 일반적으로 회전기계는 동력원을 통해 회전축 주위를 회전운동을 하는 기계를 말한다.

• 본 가이드북은 이러한 회전기계들의 공통적인 주요 기능을 담은 rotor testbed의 고장유형에 대한 분석을 다룬다.

- 분석에 사용한 rotor testbed는 시그널링크㈜의 등록제품인 ERA(Educational Rotor Application) Test Station이며, 소음-진동 test에 적합한 기능을 제공한다. [그림 1]은 데이터 구축에 이용한 실제 rotor testbed를 보여준다.



[그림 1] Rotor testbed

• Rotor testbed 기본구성

- Controller Module

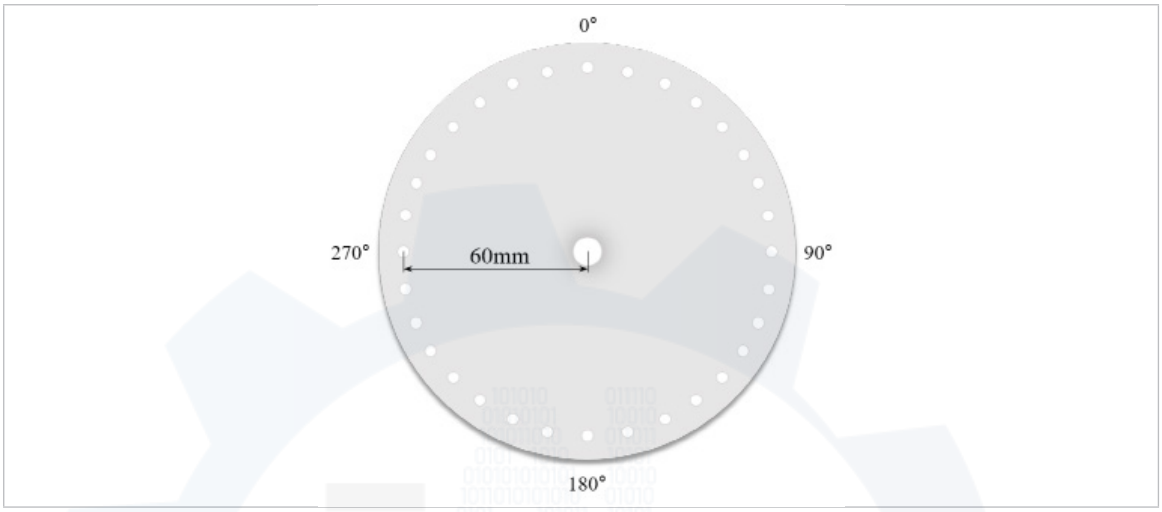
◦ Rotor testbed의 controller는 수동과 자동모드가 있으며, 수동모드의 경우 Motor 회전속도를 0~3000 RPM까지 조절할 수 있다.

◦ 본 가이드북에서는 수동 모드를 이용하여 Motor 회전속도가 약 1,500 RPM인 경우의 진동데이터를 구축하였다.

- BLDC Motor(Brushed less DC Motor)

◦ 본 가이드북에서 사용된 rotor testbed의 BLDC Motor는 브러쉬가 제거된 형태로 브러쉬를 통한 기계적인 접촉이 없는 DC Motor이다. BLDC Motor는 소음이 비교적 적으며, 출력을 세밀하게 조절할 수 있다는 장점이 있다.

- Disks



[그림 2] Aluminum disk의 단면

◦ 본 가이드북의 rotor testbed는 [그림 2]와 같이 총 2개의 aluminum disk가 있으며, 각각의 disk에는 회전 불균형을 구현할 수 있는 볼트 체결 구멍(hole)이 있다.

◦ 축의 중심으로부터 볼트 체결 구멍 사이의 거리는 약 60 mm이며, 볼트 체결 구멍은 모두 36개이다. 본 가이드북에서는 270° 위치에 볼트를 체결하여 질량의 불균형을 구현하였다.

• Rotor testbed spec

- 본 가이드북에서 사용한 rotor testbed의 기본 사양은 다음과 같다.

size	673 mm(W) × 280 mm(D) x 281 mm(H)
weight	25kg
material	Aluminum
bearing	6202ZZ × 2EA
motor	DC 12V x 0.25HP(0.2W), 0~3000 RPM
main power	220 VAC

[표 1] rotor testbed의 기본 사양

2) 이슈사항(Pain point)

• **공정(설비)상의 문제 현황**

- 많은 산업 현장에서 주요 설비의 역할을 하는 회전기계가 예상치 못한 고장으로 인해 가동이 중단될 경우, 유지보수 비용뿐만 아니라 부품 교체 기간 동안 시스템 가동 중지로 인한 비계획적 손실이 추가로 발생하는 문제점이 있음. 따라서 회전기계의 상태를 모니터링하는 것은 고장으로 인한 사고 예방 뿐아니라 제조 산업에서 생산 효율 향상에도 중요한 문제이다.

• **문제해결 장애요인**

- 고장 데이터의 부족 : 산업 현장에서 사용되는 기계 구조물은 대부분 정상 상태로 가동되며, 수집되는 데이터 역시 정상 유형이 90% 이상을 차지한다. 고장유형의 경우 자연 발생적으로 취득되는 데이터가 매우 희박하고, 실제 구조물에 인위적인 고장을 가하여 데이터를 취득하는 방식은 비효율적이므로, 이는 데이터 간 불균형 문제를 초래할 수 있다. Rotor testbed를 이용하면, 여러 고장유형에 대해 특징을 분석하고, 정상 및 고장유형을 분류할 수 있는 충분한 양의 고장 데이터를 구축할 수 있다.

• **극복 방안**

- 회전 설비의 기본이 되는 구조 : 대부분의 산업 현장(발전소 등)에서 주요 설비로 사용되는 회전기계와 주요 모니터링 변수로 사용되는 진동데이터를 수집하여 유사한 산업 현장에서의 유사한 데이터 분석에 도움을 주고자 하였다.

1.2 분석 목표

1) 분석 목표

• 분석목표 : 회전기계의 진동데이터, 즉 설비에 설치된 여러 개의 센서가 시간 순서대로 측정한 값으로 이루어진 시계열 데이터에 대해, 설비의 정상과 비정상, 더불어 비정상의 유형을 판단하는 다중 분류 문제를 해결하고자 한다.

• 목표 달성을 위한 전략 : 도메인 지식에 기반한 특징 추출과정 없이, 비전문가도 딥러닝 모델을 활용하여 진동데이터와 고장유형 간의 관계성 등 데이터에 내포된 특징을 직접 추출하고 상태 유형을 분류할 수 있는 분석 프레임워크를 구축하고자 한다.

2) 데이터 정의 및 소개

• 회전기계들의 공통적인 주요 기능을 담은 rotor testbed를 대상 설비로 하였으며, 가속도 센서를 이용하여 대상 설비로부터 진동데이터를 수집하였다.

• 해당 데이터는 rotor testbed의 상태에 따라 4개의 센서로부터 취득한 시계열 데이터이다.

3) 데이터 분석 기대효과

• 회전기계는 여러 산업 현장에서 주요 설비의 역할을 하며, 진동데이터는 많은 제조 현장에서 주로 사용하는 특성 데이터이다. rotor testbed는 그러한 회전기계의 대표적인 기능들을 구현하고 있으므로, testbed를 통해 고장을 구현하고 데이터를 분석하면 실제 현장에서 구하기 어려운 고장 데이터에 대해서도 분석하기에 충분한 양을 확보할 수 있으며 많은 특성을 확인할 수 있다.

4) 시사점(implication) 요약기술

• 회전기계의 공통적인 기능을 구현한 testbed를 이용하여 실제 현장의 진동(소음) 문제의 주요 원인인 불균형과 지지 불량 고장에 대한 데이터를 구축하고 이를 분석하였다. 진동 신호에 대한 전문지식이 없어도 데이터로부터 기계 상태에 대한 특징을 학습할 수 있는 딥러닝기반 예측/분류를 수행할 수 있을 뿐만 아니라, 실제 현장에서 적용 가능한 센서 데이터의 전처리과정을 튜토리얼을 통해 실습할 수 있다는 점에서 시사점이 크다고 판단한다. 더불어 개발된 인공지능 모델의 기술 확장을 통해, 진동 신호를 모니터링하는 산업 현장에서의 회전기계 정상/비정상 진단에 활용될 것으로 기대한다.

2. 분석실습

2.1 제조데이터 소개

1) 데이터 수집 방법

• **제조(설비) 분야** : 회전기계가 사용되는 모든 산업 분야의 회전기계 운영 시스템
• **제조 공정(설비)명** : 여러 산업 현장에서 주요 설비로 운영되는 회전기계의 공통적인 기능을 구현한 모델이므로, 일종의 회전기계시스템 및 그 운영데이터라고 할 수 있다.
• **수집장비** : 시그널링크㈜에서 제조한 스마트 진동 센서 ([그림 3] 참고)



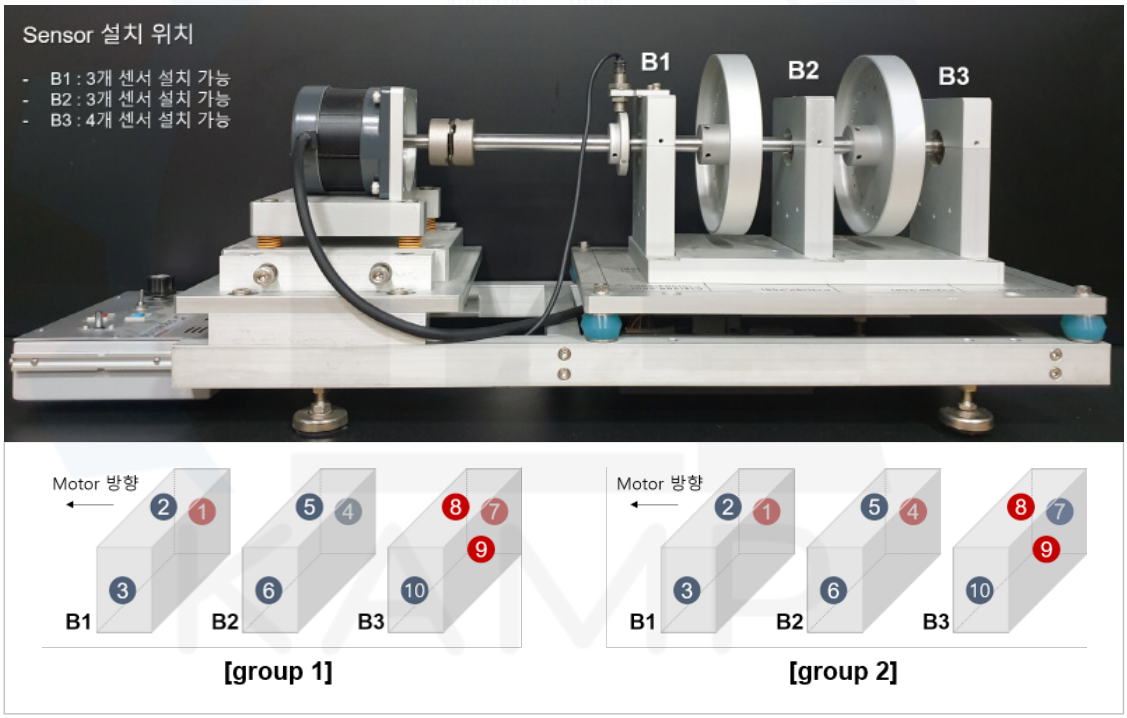
[그림 3] 데이터수집에 이용한 진동 센서

- 진동 센서 사양 ([표 2] 참고)

size	44 mm(W) x 44 mm(D) x 25 mm(H)
weight	About 40gf
material	PC
Fix	M4 Bolt or Magnetic
Frequency span	672 Hz
Frequency resolution	1.3125 Hz

[표 2] 진동 센서의 기본 사양

- 데이터의 수집 위치는 다음의 그림에서 확인할 수 있다. 본 가이드북에서는 2개의 센서 그룹으로부터 데이터를 수집하였으며, 이를 통해 고장유형에 따른 최적의 센서 부착 위치에 대한 고찰 가능성을 제시한다.



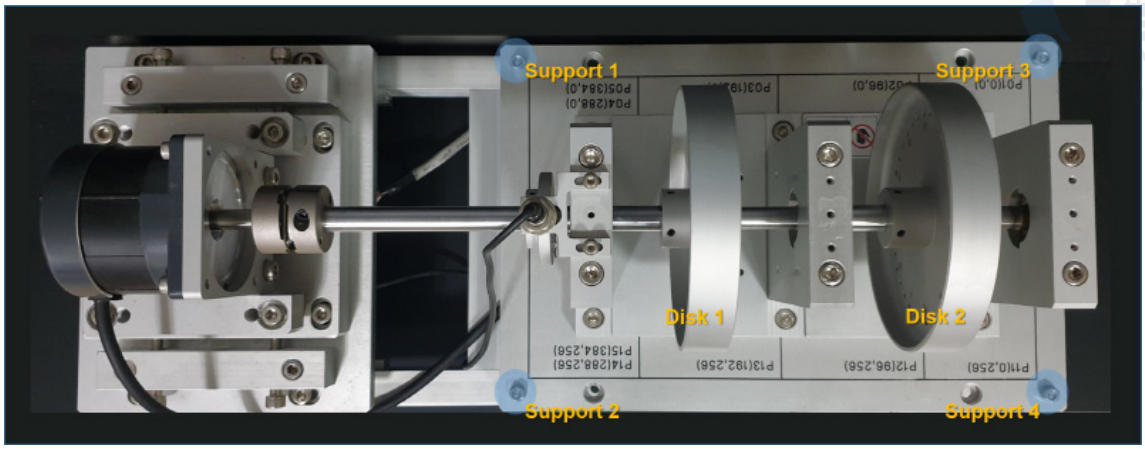
[그림 4] 센서 설치 위치

• **수집 주기** : [그림 4]에 표시된 4개의 센서 부착 위치에서, 약 140초의 가속도(진동) 신호를 수집한다.

2) 데이터 유형/구조

• **데이터 유형**

- 본 가이드북에서 구현한 rotor testbed의 상태 유형은 다음과 같다.



[그림 5] Rotor testbed를 위에서 찍은 사진

- 정상 상태 (Normal)
- 질량 불균형(Unbalance - Type 1)
  - 회전기기의 진동 원인으로 여러 가지가 있으나 그 중 질량 불균형이 차지하는 비중이 가장 크다. 이는 회전기기에서 rotor의 질량 중심이 회전 중심(축 중심)과 일치하지 않을 때 나타나는 현상이다. 따라서 비대칭적인 질량분포가 있는 rotor를 회전시켰을 때 발생하는 비대칭적 원심력은 회전기계에서 진동(소음)을 발생시킨다.
  - 본 가이드북에서는 불균형을 구현하기 위해 [그림 5]에서 확인할 수 있는 disk 2의 270° 위치의 볼트 체결 구멍에 볼트와 너트를 이용하여 질량 불균형을 구현하였다.
- 지지 불량(Mechanical Looseness - Type 2)
  - 회전기계를 지면 혹은 평판 등에 견고하게 지지하지 않은 경우, 기계의 수평이 틀어지거나 장비의 기울어짐 등이 확인될 수 있다. 특히 기계 지지가 구조적으로 이완되고 및 취약한 경우, 다른 고장유형과 유사한 진동 스펙트럼을 나타내므로 불평형, 축 정렬 불량 등으로 잘못 진단할 수 있다.
  - 본 가이드북에서는 지지 불량을 구현하기 위해 [그림 5]에서 확인할 수 있는 support 4 위치의 고정 나사를 헐겁게 조절하였다.
- 질량 불균형과 지지 불량(Type 3)

• **가공 전 데이터(Raw data)**

	time	normal	type1	type2	type3
0	0.000000	-0.853307	-3.464579	0.555219	3.919664
1	0.000736	-0.740463	-2.448986	-0.234687	4.145351
2	0.001472	-0.138630	-1.922383	-0.009000	2.941685
3	0.002208	0.049443	-0.906790	-0.272301	2.603155
4	0.002944	-0.289088	-0.568259	-0.986978	1.361874
...	...	...	...	...	...

[그림 6] 센서 1의 가공 전 데이터

- 데이터셋 구조 : 테이블 형식 (tabular)
- 데이터 개수
  - (group1) column 수 5개, row 수 754,477개, 데이터수 3,772,385개
  - (group2) column 수 5개, row 수 754,477개, 데이터수 3,772,385개
- AI 데이터셋 주요 변수 정의

속성 (column)	설명	비고
Time	데이터의 수집 시간	float
Normal	정상상태	float
Type 1	질량불균형 고장상태	float
Type 2	지지불량 고장상태	float
Type 3	질량불균형과 지지불량 고장상태	float

[표 3] 데이터셋 주요 변수 (group 1 and 2)

- 주요 변수 기술 통계 (sensor group 1, sensor group 2)

(Group 1) Sensor 1							
속성 (column)	데이터형	개수	평균	표준편차	최소값	중앙값	최대값
Time	float	190,218	69.9999	40.4148	0.0000	69.9999	139.9997
Normal	float	190,218	-0.000037	0.7084	-2.2511	0.08099	2.0374
Type 1	float	190,218	-0.000087	1.7002	-6.6994	-0.0247	6.6161
Type 2	float	190,218	0.000036	0.7248	-2.3669	-0.0309	2.3117
Type 3	float	190,218	-0.000067	1.8708	-6.9416	0.3927	7.5400
(Group 1) Sensor 2							
Time	float	184,211	69.9999	40.4148	0.0000	69.9999	139.9997
Normal	float	184,211	-0.000011	0.9992	-3.7819	0.0488	3.4507
Type 1	float	184,211	0.000063	1.4950	-3.8841	-0.3641	4.8297
Type 2	float	184,211	-0.000008	0.8512	-2.8553	-0.0229	2.6312
Type 3	float	184,211	-0.000091	1.5792	-3.4062	-0.1358	3.9380
(Group 1) Sensor 3							
Time	float	196,079	69.9999	40.4148	0.0000	69.9999	139.9997
Normal	float	196,079	0.000010	0.3251	-1.9217	0.0120	1.4051
Type 1	float	196,079	0.000008	0.3433	-1.9662	0.0116	1.4487
Type 2	float	196,079	-0.0000009	0.3728	-1.6517	0.0114	1.6267
Type 3	float	196,079	0.000006	0.3537	-1.6052	0.0108	1.5392
(Group 1) Sensor 4							
Time	float	183,969	69.9999	40.4148	0.0000	69.9999	139.9997
Normal	float	183,969	-0.000017	0.6579	-2.8581	0.0051	2.2646
Type 1	float	183,969	-0.00014	0.6225	-2.7429	0.0145	2.5326
Type 2	float	183,969	0.000012	0.9880	-2.9029	-0.0584	3.1585
Type 3	float	183,969	-0.000023	1.1780	-4.2606	0.0945	3.7252

[표 4] 데이터셋 주요 변수 (Group 1)

(Group 2) Sensor 1							
속성 (column)	데이터형	개수	평균	표준편차	최소값	중앙값	최대값
Time	float	190,218	69.9999	40.4148	0.0000	69.9999	139.9997
Normal	float	190,218	−0.000003	0.6440	−3.5273	0.0084	3.1332
Type 1	float	190,218	0.000043	1.0511	−4.0552	0.0469	3.9992
Type 2	float	190,218	0.000015	0.6342	−2.9675	0.0095	3.2087
Type 3	float	190,218	−0.000038	0.8183	−3.9982	−0.0709	3.7503
(Group 2) Sensor 2							
Time	float	184,211	69.9999	40.4148	0.0000	69.9999	139.9997
Normal	float	184,211	0.000026	1.1043	−3.7443	0.0454	3.9202
Type 1	float	184,211	0.000078	1.6063	−4.0376	−0.5109	4.8171
Type 2	float	184,211	−0.000023	0.8771	−2.4500	−0.0198	2.6202
Type 3	float	184,211	−0.000056	1.5967	−3.9975	−0.3235	3.8690
(Group 2) Sensor 3							
Time	float	196,079	69.9999	40.4148	0.0000	69.9999	139.9997
Normal	float	196,079	0.000005	0.3330	−1.6071	0.0123	1.3586
Type 1	float	196,079	0.000002	0.3314	−1.6494	0.0116	1.5396
Type 2	float	196,079	0.000010	0.3703	−1.6513	0.0111	1.5843
Type 3	float	196,079	−0.000004	0.3871	−1.9650	0.0112	1.5824
(Group 2) Sensor 4							
Time	float	183,969	69.9999	40.4148	0.0000	69.9999	139.9997
Normal	float	183,969	0.000014	0.6484	−2.6640	−0.0204	2.5337
Type 1	float	183,969	−0.000003	0.6281	−2.7459	0.0426	2.3810
Type 2	float	183,969	−0.000015	0.8116	−2.6303	−0.0342	2.8421
Type 3	float	183,969	−0.000021	0.9692	−3.2857	0.1146	3.3069

[표 5] 데이터셋 주요 변수 (Group 2)

- 센서 1-4의 sampling rate가 같지 않으므로 140초간 수집한 데이터의 행(row)의 수가 [그림 7]에서 확인할 수 있듯이 차이가 있다.

```
sensor 1의 데이터 크기 (190218, 5)
sensor 2의 데이터 크기 (184211, 5)
sensor 3의 데이터 크기 (196079, 5)
sensor 4의 데이터 크기 (183969, 5)
```

[그림 7] 센서별 데이터 크기

◦ 1차 가공 데이터(Processed data) : 사용한 4개 센서의 sampling rate가 다르므로, 신호 수집 간격의 동일화를 위해 선형보간을 한 데이터이다.

- 데이터셋 구조 : 테이블 형식 (tabular)

◦ 선형보간이 완료된 데이터는 각 센서의 신호 수집 간격이 동일하다. 따라서 본래의 수집 순서에 따라 회전기계 상태 유형에 대한 4개 센서의 신호로 구성되며 이는(140000, 4)가 된다.

- 데이터 개수

◦ (group1) column수 4개, row수 140,000개, 데이터수 560,000개
◦ (group2) column수 4개, row수 140,000개, 데이터수 560,000개
◦ 라벨링 과정을 통해 회전기계 상태 유형에 대한 열(column)이 추가된다.

독립변수	센서 1~4 데이터(총 4개)
종속변수	회전기계 상태 유형(총 1개)

[표 6] rotor testbed의 기본 사양

- AI 데이터셋 주요 변수 정의

속성 (column)	설명	비고
Sensor 1	선형보간된 센서 1의 데이터	float
Sensor 2	선형보간된 센서 2의 데이터	float
Sensor 3	선형보간된 센서 3의 데이터	float
Sensor 4	선형보간된 센서 4의 데이터	float

[표 7] 데이터셋 주요 변수 (Group 1 and 2)

- 주요 변수 기술 통계 (sensor group 1, sensor group 2)

(Group 1) Normal							
속성 (column)	데이터형	개수	평균	표준편차	최소값	중앙값	최대값
Sensor 1	float	140,000	0.000154	0.6621	-1.9778	0.1087	1.7618
Sensor 2	float	140,000	-0.000176	0.9199	-3.2700	0.0616	3.2189
Sensor 3	float	140,000	-0.000055	0.2890	-1.8555	0.0172	1.2655
Sensor 4	float	140,000	-0.000033	0.5966	-2.3790	-0.0224	2.1721
(Group 1) Type 1							
Sensor 1	float	140,000	-0.000147	1.6790	-6.1884	-0.0229	6.3275
Sensor 2	float	140,000	0.000173	1.4407	-3.4906	-0.4239	4.8074
Sensor 3	float	140,000	-0.000221	0.2997	-1.5605	0.0112	1.3498
Sensor 4	float	140,000	0.000039	0.5507	-2.4827	0.0173	2.3606
(Group 1) Type 2							
Sensor 1	float	140,000	0.000112	0.6923	-1.9157	-0.0431	2.0983
Sensor 2	float	140,000	0.000114	0.8298	-2.6806	-0.0169	2.4432
Sensor 3	float	140,000	-0.000018	0.3337	-1.4635	0.0102	1.5192
Sensor 4	float	140,000	-0.000012	0.9545	-2.7714	-0.0495	2.8743
(Group 1) Type 3							
Sensor 1	float	140,000	-0.000063	1.8559	-6.7454	0.3952	7.0531
Sensor 2	float	140,000	0.000300	1.5667	-3.3361	-0.1343	3.7897
Sensor 3	float	140,000	-0.000032	0.3100	-1.4820	0.0053	1.3245
Sensor 4	float	140,000	-0.000029	1.1480	-4.2361	0.0997	3.6601

[표 8] 데이터셋 주요 변수 (Group 1)

(Group 2) Normal							
속성 (column)	데이터형	개수	평균	표준편차	최소값	중앙값	최대값
Sensor 1	float	140,000	0.000155	0.4618	-2.8127	-0.0053	2.7537
Sensor 2	float	140,000	0.000299	1.0259	-3.5017	0.0551	3.7080
Sensor 3	float	140,000	-0.000153	0.2934	-1.3936	0.0237	1.2200
Sensor 4	float	140,000	-0.000116	0.5914	-2.5966	-0.0257	2.4079
(Group 2) Type 1							
Sensor 1	float	140,000	-0.000062	0.9610	-3.2362	0.0733	3.8598
Sensor 2	float	140,000	0.000192	1.5522	-3.6336	-0.5946	4.3458
Sensor 3	float	140,000	-0.000128	0.2845	-1.5645	0.0148	1.4445
Sensor 4	float	140,000	-0.000081	0.5512	-2.4414	0.0181	1.9383
(Group 2) Type 2							
Sensor 1	float	140,000	0.000094	0.4638	-2.4596	0.0131	3.0411
Sensor 2	float	140,000	-0.000084	0.8589	-2.2864	-0.0189	2.5190
Sensor 3	float	140,000	-0.000022	0.3263	-1.4977	0.0073	1.3298
Sensor 4	float	140,000	-0.000064	0.7772	-2.2704	-0.0410	2.6150
(Group 2) Type 3							
Sensor 1	float	140,000	-0.000112	0.7760	-3.5787	-0.0882	3.5582
Sensor 2	float	140,000	-0.000053	1.5847	-3.9027	-0.3237	3.7473
Sensor 3	float	140,000	-0.000078	0.3337	-1.6438	0.0101	1.3912
Sensor 4	float	140,000	0.000041	0.9417	-3.0465	0.1172	3.2396

[표 9] 데이터셋 주요 변수 (Group 2)

3) 데이터 (품질) 전처리

• **데이터 품질 전처리 목적**

- 실제 공정에서 발생하는 데이터는 의미 없는 값이거나, 누락 및 오타가 발생하여 품질이 좋지 못하다. 품질이 낮은 데이터를 분석에 이용하면 좋은 결과를 얻기 힘들다. 그러므로 데이터 (품질) 전처리는 데이터의 분석에 있어서 필수적인 단계이다.
- 데이터 품질 전처리를 위해 아래 6가지의 데이터 품질지수를 파악하고, 데이터의 품질지수를 향상시킨다.

• **데이터 품질지수 ([표 10] 참고)**

완전성 (Completeness)	필수항목에 누락이 없어야 한다.
유일성 (Uniqueness)	데이터 항목은 유일해야 하며 중복되어서는 안 된다.
유효성 (Validity)	데이터 항목은 정해진 유효범위 및 도메인을 충족해야 한다.
일관성 (Consistency)	데이터가 지켜야 할 구조, 값, 표현되는 형태가 일관되게 정의되고, 서로 일치해야 한다.
정확성 (Accuracy)	실제 존재하는 객체의 표현 값이 정확하게 반영이 되어야 한다.
무결성 (Integrity)	데이터베이스 자료의 오류 없이 변화에 영향을 받지 않고 데이터의 정확성과 일관성, 유효성이 보호되어야 한다.
무결성 (Integrity)	데이터베이스 자료의 오류 없이 변화에 영향을 받지 않고 데이터의 정확성과 일관성, 유효성이 보호되어야 한다.

[표 10] 6가지 데이터 품질지수

• **데이터 품질지수 세부 설명**

- 완전성 품질지수

$$\text{완전성 품질지수} = \left(1 - \frac{\text{결측 데이터의 개수}}{\text{전체 데이터의 개수}}\right) \times 100$$

① 결측 데이터의 값이 30% 이상인 데이터들은 데이터의 완전성을 떨어뜨리기 때문에 각 열(column)에 대한 결측 데이터값의 비율을 확인하여 삭제한다.
② 총 결측 데이터의 개수를 확인하기 위해 'isnull()' 함수를 사용하여 각 열(column)에서의 결측 데이터 개수를 구한 뒤 'sum()' 함수를 이용하여 합산한다.
③ 앞서 구한 결측 데이터의 개수로부터 위 식에 따라 완전성 품질지수를 구한다.

• 회전기계 고장유형 AI 데이터셋은 결측 데이터를 갖는 열(column)이 존재하지 않으므로 완전성을 판단하지 않는다.

- 유일성 품질지수

$$\text{유일성 품질지수} = \left(\frac{\text{유일한 데이터의 개수}}{\text{전체 데이터의 개수}}\right) \times 100$$

① 데이터에서 유일한 값을 갖는 열(column)을 찾는다. (EX) 시계열 데이터 : 시간
② 유일한 값을 갖는 열(column)에 대해 'groupby()' 함수를 이용하여, 각 열(column)에서의 유일한 데이터 값을 확인하고, 'size()' 함수를 이용하여 그 개수를 구한다.
③ 앞서 구한 유일한 데이터의 개수로부터 위 식에 따라 유일성 품질지수를 구한다.
• 회전기계 고장유형 AI 데이터셋은 유일한 값을 갖는 열(column)이 존재하지 않으므로 유일성을 판단하지 않는다.

- 유효성 품질지수

$$\text{유효성 품질지수} = \left(\frac{\text{유효성을 만족한 데이터의 개수}}{\text{전체 데이터의 개수}}\right) \times 100$$

① 'max()', 'min()' 함수를 이용하여 데이터가 유효범위를 벗어나지 않는지를 확인한다.
② Pandas의 '*.info()' 함수로 데이터의 타입이 정해진 형식에 맞는지 확인한다.
③ 그 외에 수집된 날짜 안에 들어가 있는지 등을 검증하여 유효성을 만족한 데이터의 개수를 구해 위 식에 따라 유효성 품질지수를 구한다.
• 회전기계 고장유형 AI 데이터셋은 유효한 값을 갖는 데이터로 구성되므로 유효성을 판단하지 않는다.

- 일관성 품질지수

$$\text{일관성 품질지수} = \left(\frac{\text{일관성을 만족한 데이터의 개수}}{\text{전체 데이터의 개수}}\right) \times 100$$

① 데이터 테이블 간 종속관계를 확인한다.
② Pandas의 '*.info()' 함수로 데이터의 타입이 정해진 형식에 맞는지 확인한다.
③ 그 외에 수집된 날짜 안에 들어가 있는지 등을 검증하여 일관성을 만족한 데이터의 개수를 구해 위 식에 따라 일관성 품질지수를 구한다.

• 회전기계 고장유형 AI 데이터셋은 일관성 있는 값을 갖는 데이터로 구성되므로 일관성을 판단하지 않는다.

- 정확성 품질지수

$$\text{정확성 품질지수} = 1 - \left(\frac{\text{정확성에 위배되는 데이터의 개수}}{\text{전체 데이터의 개수}} \right) \times 100$$

① 데이터의 '평균값'이 수집된 데이터에 한하여, 다음과 같이 정확성 품질지수를 확인할 수 있다.
② 누락된 데이터가 있는지 확인하기 위해, '*.mean()' 함수를 이용하여 데이터들의 평균을 구한다.
③ '*.mean()' 함수로 구한 데이터의 평균에서 데이터의 '평균값'과의 차이를 구한다.
④ 이 차이가 0이 아니면 데이터를 위배하는 데이터가 존재하게 된다.

• 회전기계 고장유형 AI 데이터셋은 누락된 데이터가 존재하지 않으므로 정확성을 판단하지 않는다.

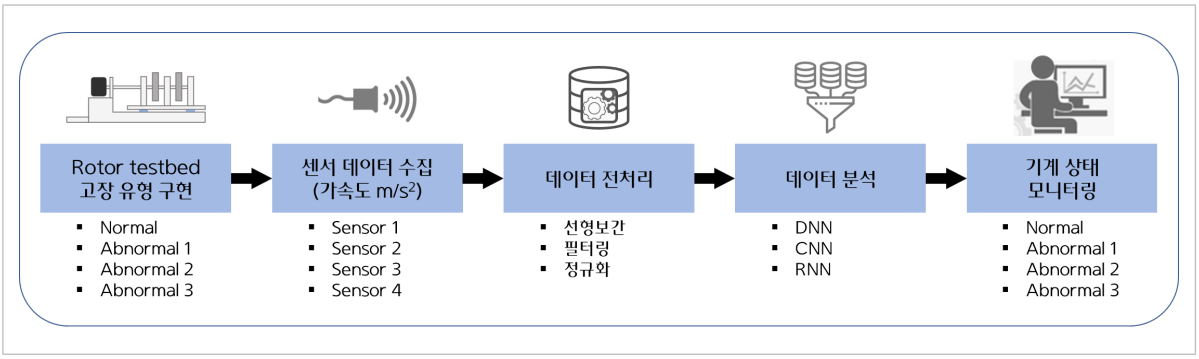
- 무결성 품질지수

$$\text{무결성 품질지수} = \left\{ 1 - \left(\frac{\text{유일성, 유효성, 일관성 중 100\%가 아닌 지수}}{3} \right) \right\} \times 100$$

• 무결성 품질지수는 데이터가 '유일성', '유효성', '일관성' 지수를 만족하는지 확인하는 지표이다. 일반적으로 세 가지 지수 중 100% 품질을 담당하는 지수의 비율을 무결성 품질지수로 정의한다.
• 회전기계 고장유형 AI 데이터셋은 유일성, 유효성, 일관성을 판단하지 않으므로 무결성 지수 역시 판단하지 않는다.

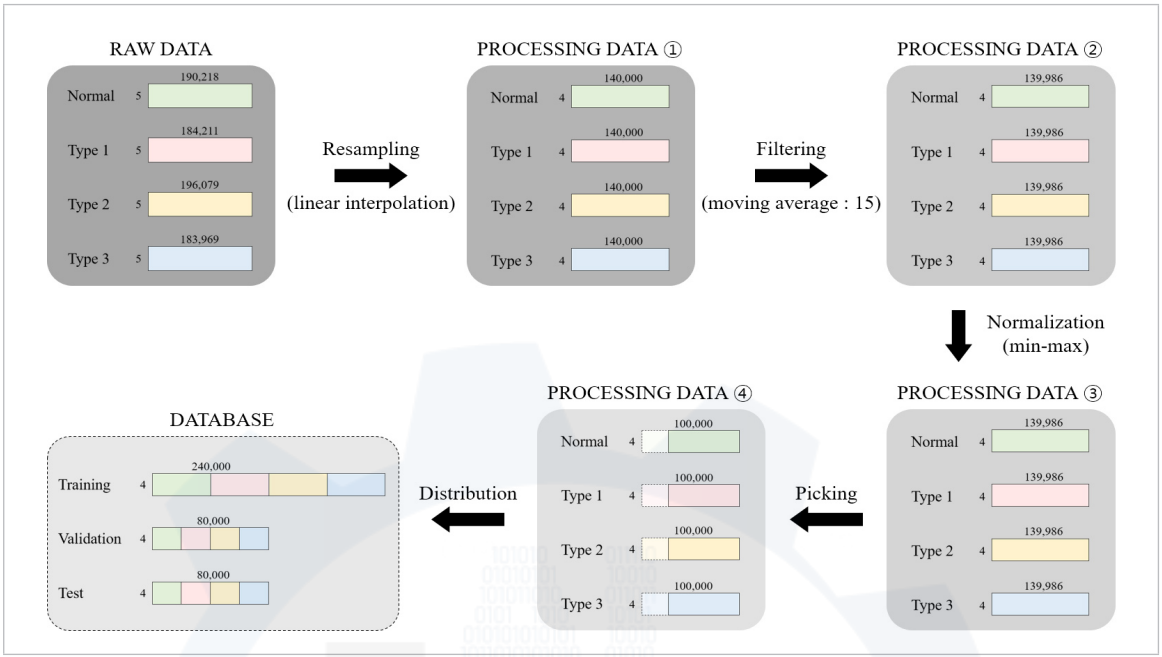
2.2 분석 모델 소개

1) 데이터 흐름 및 인공지능 모델 적용 흐름도 ([그림 8] 참고)



[그림 8] 전처리과정에 따른 데이터 크기 변화

• 본 가이드북에서 데이터 전처리에 사용한 방법은 다음의 [그림 9]과 같다.



[그림 9] 전처리과정에 따른 데이터 크기 변화

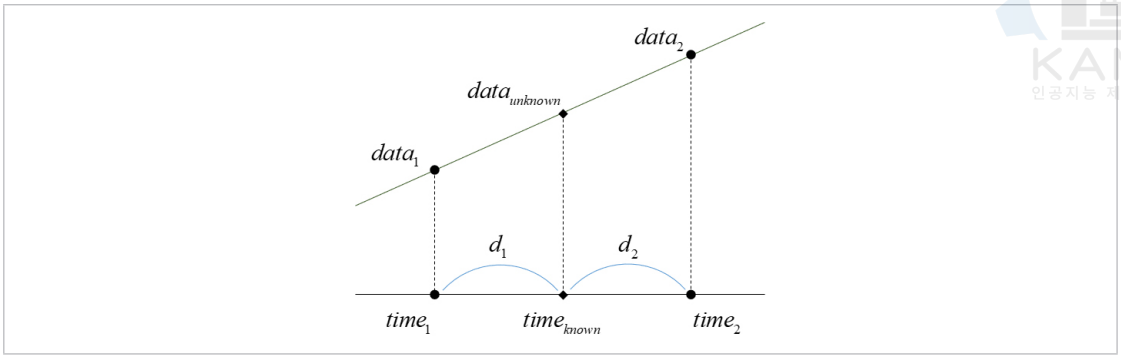
• **선형보간(linear interpolation)**

- 전처리 사유 : 회전기계 고장유형 AI 데이터셋의 계측 환경처럼 다수의 센서를 사용하는 경우, 각 센서로부터 수집된 데이터의 저장 간격이 일정하지 않을 수 있다. 따라서 데이터 간 정확한 비교를 위해 계측 시간의 동일화(resampling)이 요구된다. 실제로 본 가이드북에서 사용한 4개의 센서의 sampling period는 [표 10]과 같으며 같은 종류의 가속도 센서이지만 sampling rate이 다르다.

sensor 1	sensor 2	sensor 3	sensor 4
0.000736	0.000760	0.000714	0.000761

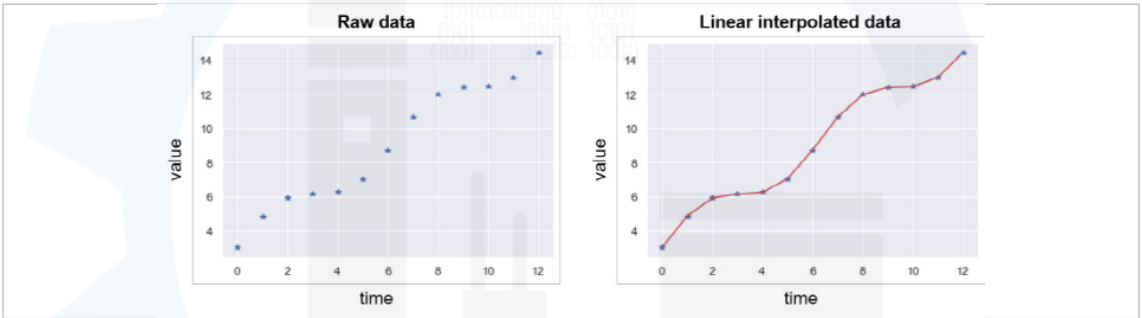
[표 10] 센서의 sampling period

- 아래는 선형보간과 같은 계측 시간의 동일화가 필요한 사례이다.
  - 다수의 센서를 사용하는 환경에서 각 센서의 sampling rate가 다른 경우
  - 여러 종류의 센서를 사용하는 환경의 경우(온도, 진동, 전력량 등)
  - 데이터 저장 효율을 위해 데이터의 변동이 있을 시에만 데이터를 저장하는 event-driven data collection 방식을 사용하는 경우(대부분의 산업 현장)



[그림 10] 선형보간법

- 보간(interpolation)이란 알려진 데이터 사이의 값을 알려진 두 데이터로부터 추정하는 것을 말한다. 이때, 선형보간(linear interpolation)은 [그림 10]과 같이 두 데이터 사이의 값을 두 데이터와의 직선거리에 따라 선형적으로 결정하는 방법이다. [그림 11]은 선형보간의 결과를 보여주는 예시이다.



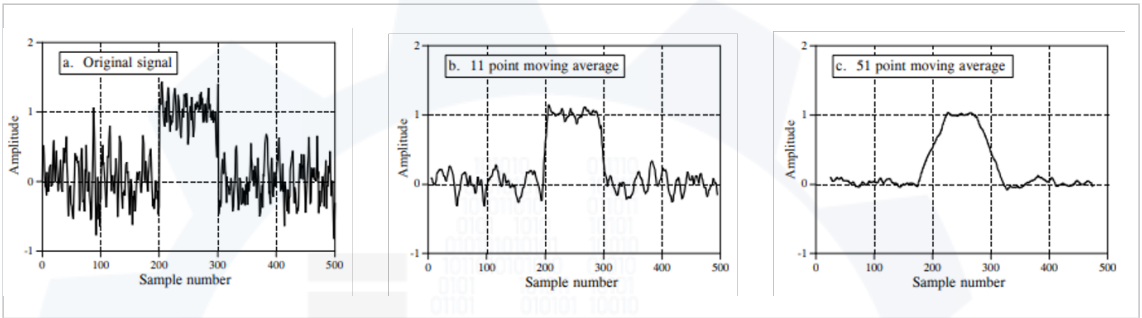
[그림 11] 선형보간법의 결과(예시)

- 회전기계의 속도와 sampling rate의 관계 : 모터 최대 속도가 3000 RPM인 회전기계의 최소 sampling rate은 Nyquist 이론에 의해 회전기계의 최대주파수(25 ㎐)의 2배 이상으로 설정해야 한다. 또한, 계측 데이터의 신호를 분석하기 위해서는 최대주파수의 5-10배의 계측 sampling rate(125-250 ㎐)이 요구된다. 본 가이드북에서는 4개 센서의 계측 데이터를 최대한 보존하는 범위에서 신호 분석이 가능하도록 resampling을 하기 위해 0.001 sec의 sampling period로 down-sampling을 진행하였다.

• **이동평균 필터링(Moving average filtering)**

- 전처리 사유 : 본 가이드북의 계측 환경과 같이 센서로부터 취득한 데이터는 기본적으로 기계학습에 좋지 않은 영향을 주는 노이즈(Noise)가 포함되어 있다. 따라서 AI 분석 모델의 훈련 성능의 향상을 위해 데이터 필터링이 요구된다.

- 일반적으로 센서 데이터에 평균을 취하면, 데이터의 노이즈를 제거할 수 있다. 그러나 측정하려는 신호값이 시간에 따라 변하는 경우 이 방식은 적절하지 않다. (데이터의 동적인 변화를 모두 스무딩(smoothing)하는 결과) 따라서 본 가이드북에서는 최근 M개의 측정값으로부터 평균을 구하고 이후 M개의 값의 평균을 구하는 방식으로 데이터에서 노이즈를 제거하면서, 동적인 변화를 반영하는 이동평균 필터링을 이용하였다.
- 모든 데이터의 평균을 계산하는 것이 아니라, 정해진 크기(M개)의 최근 측정값들로만 평균을 구하기 때문에, M값은 필터링 및 차후 AI 분석 모델 학습 결과에 영향을 미친다. [그림 12]는 M값에 따른 필터링 결과를 나타낸다.



[그림 12] M값에 따른 이동평균 필터링 결과

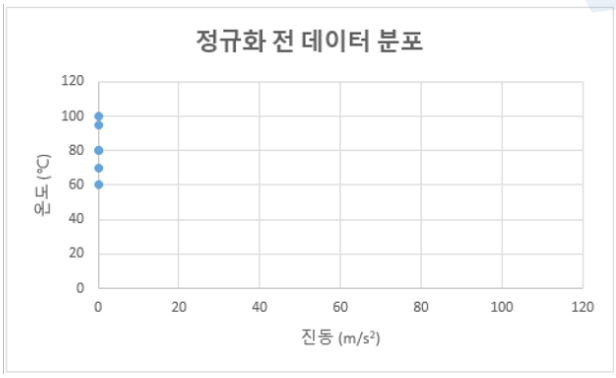
* Smith, S. W. “The Scientist & Engineer's Guide to Digital Signal Processing”, Second Edition, California Technical Publishing,(1999)."

• **최대-최소 정규화(Min-max normalization)**

- 전처리 사유 : 본 가이드북의 센서 값들은 다양한 범위의 값을 갖는다. 이렇듯 데이터가 가진 특성(feature)의 스케일이 차이가 나는 경우, AI 분석 모델 훈련에 좋지 않은 영향을 미칠 수 있다.
- 예를 들어, [그림 13]은 데이터 feature scale의 차이가 심한 경우의 예를 보여준다. 진동데이터와 온도 데이터의 경우 다음과 같이 데이터 scale의 차이가 크다. 이 경우 정규화 전 데이터의 분포를 확인하면, 진동 값은 온도 값에 비해 큰 변화가 없으므로, 결국 온도 값에 따라 데이터의 훈련이 좌지우지되는 것이다.

정규화 전 데이터셋

진동 (㎧)	온도 (℃)
0.0003	60
0.0009	80
0.0005	70
0.0008	100
0.0007	80
0.0008	95

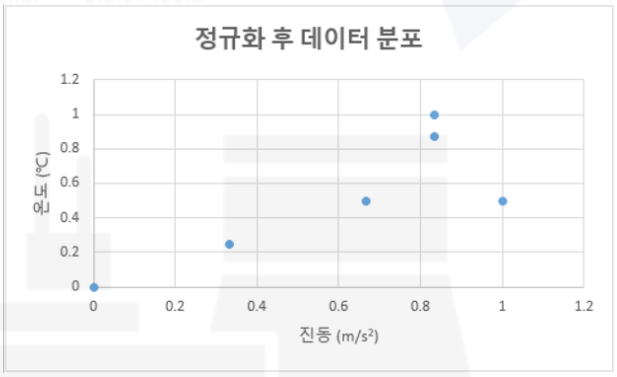


[그림 13] 정규화 전 데이터 (예시)

- [그림 14]는 [그림 13]의 데이터를 Min-max 방식으로 정규화한 결과를 보여준다. 이 경우 진동데이터와 온도 데이터 모두 값이 0과 1사이로 정규화되었기 때문에, 데이터의 분석에서 온도와 진동 모두를 동일한 가중치로 학습에 이용할 수 있게 된다.

정규화 후 데이터셋

진동 (㎧)	온도 (℃)
0	0
1	0.5
0.333333333	0.25
0.833333333	1
0.666666667	0.5
0.833333333	0.875



[그림 14] 정규화 전 데이터(예시)

- 일반적으로 기계학습을 위해 인풋 데이터를 위와 같이 정규화하는데, 본 가이드북에서는 최대-최소 정규화를 이용하였다. 최대-최소 정규화는 다음의 식을 통해 데이터를 0과 1 사이의 값으로 스케일링(scaling)한다.

$$X_{norm} = \frac{X - X_{\min}}{X_{\max} - X_{\min}}$$

① Normal data의 최대값(max)과 최소값(min)을 구한다.

② 정규화할 데이터 값(x)과 앞서 구한 최대값(Xmax), 최소값(Xmin)을 다음의 식에 대입한다.

• **라벨링(Labeling)**

- 본 가이드북의 분류 class의 개수는 정상 상태, 질량 불균형, 지지 불량, 질량 불균형과 지지 불량의 혼합으로 총 4가지이다. 따라서 제안된 데이터 분석 AI 모델의 지도학습을 위해 각 유형별 데이터 순서대로 다음의 라벨을 부여한다. python은 0부터 시작하는 프로그래밍 언어이므로 0부터 순서대로 회전기계의 상태를 나타낼 수 있도록 라벨링하였다. ([표 11] 참고)

Normal	Type 1	Type 2	Type 3
0	1	2	3

[표 11] 각 상태 유형에 따른 라벨

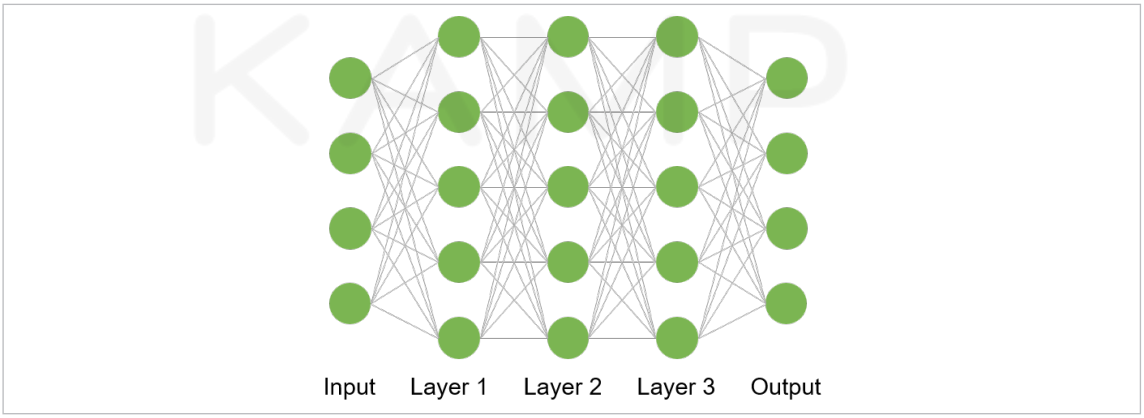
2) AI 분석모델

• **해당 AI 방법론(알고리즘) 선정 이유 기술**

- 산업 현장 내 운영 시스템의 상태 유형을 분류하는 모델의 경우 지도학습을 통해 훈련시키는 것이 가장 효율적이며 좋은 성능을 보이므로, 위의 대표적인 3가지 분류 모델을 선정하였다.

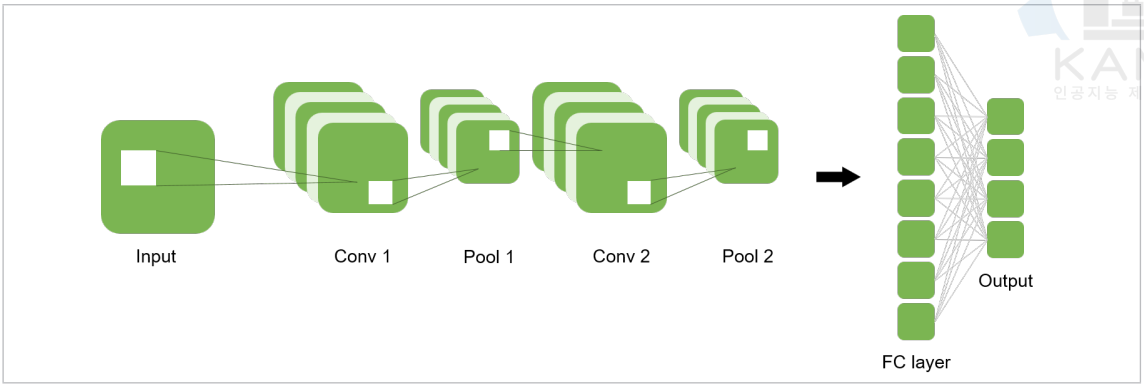
• **적용하고자하는 AI 분석 방법론(알고리즘)의 구체적 소개**

- 심층신경망(Deep Neural Network) : 인공신경망에서 모델 내 은닉층을 많이 늘려서 학습의 결과를 향상시키는 방법이다.



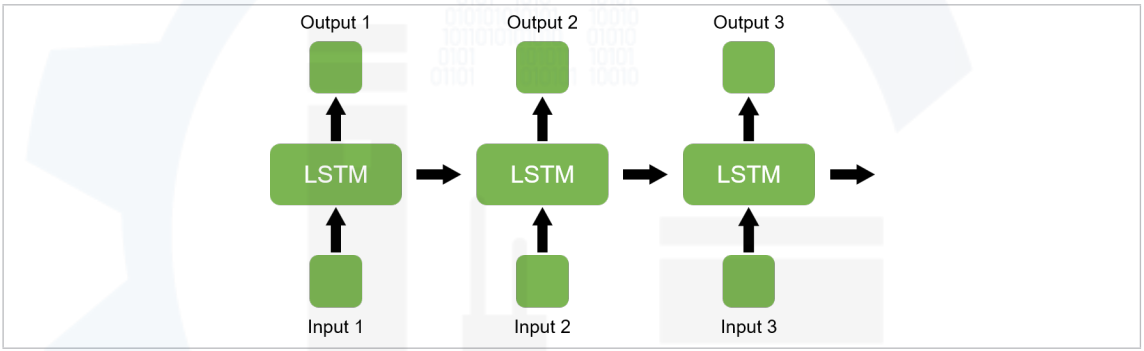
[그림 15] 기본적인 심층신경망

- 합성곱신경망(Convolutional Neural Network) : 인공신경망의 한 종류이며, 주로 Matrix 데이터나 이미지 데이터에 대하여 특징을 추출하는 데에 쓰인다.



[그림 16] 기본적인 심층신경망

- 순환신경망(Recurrent Neural Network) : 인공신경망 종류 중 하나로 과거의 학습을 가중치를 통해 현재 학습에 반영하는 특징을 가지기 때문에, 진동데이터와 같은 순차적인 데이터(sequential data) 학습에 사용된다.



[그림 17] 기본적인 심층신경망

• **AI 분석 방법론(알고리즘) 구축 절차 설명 ([그림 8] 참고)**

- rotor testbed를 이용하여 가능한 고장유형을 구현한다.
- 가속도 센서를 이용하여 데이터의 수집한다.
- 구축된 데이터를 분석하기 쉬운 형태로 전처리(보간, 필터링, 정규화) 한다.
- 딥러닝 분류 모델(DNN, CNN, RNN)을 이용하여 데이터로부터 기계의 상태를 예측하도록 훈련시킨다.
- 훈련된 AI 모델에 학습에 사용하지 않은 데이터를 입력하여 성능을 확인한다.

2.3 분석 체험

1) 필요 SW, 패키지 설치 방법 및 절차 가이드

• **필요 SW**

- Anaconda : 4.8.3
- Python : 3.7.3 (jupyter notebook 등)
- 설치된 SW의 버전을 확인을 위해 Anaconda prompt 창에 다음을 입력한다.

```
conda -V
python —version
```



[그림 18] Anaconda version 확인



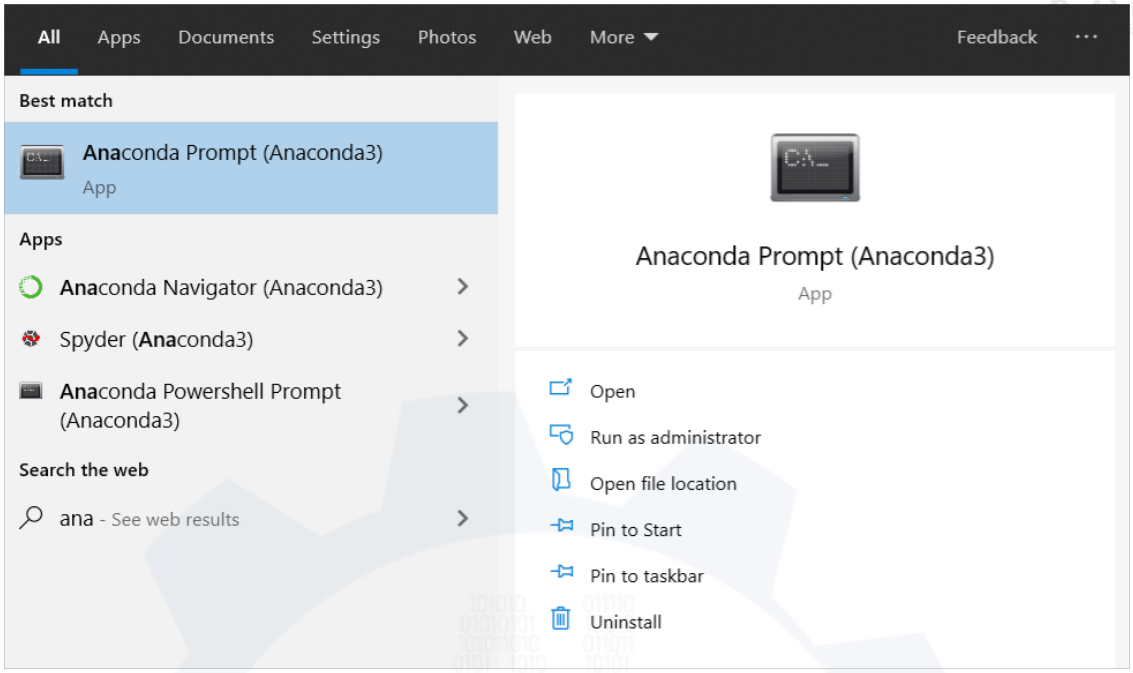
[그림 19] Python version 확인

▶ SW 설치는 '부록_분석환경 구축을 위한 설치 가이드'참고

▶ **가상환경 구축 가이드**

- 3. 부록 [분석 환경 구축을 위한 설치 가이드]에서 설치한 Anaconda에서 python의 독립적인 가상의 작업 환경을 구축한다. python의 모듈은 다양한 버전이 존재하기 때문에 충돌이 일어나기 쉬우며, 이러한 충돌을 방지하기 위해 실습마다 독립적인 가상 환경 구축을 권장한다. 가상 환경을 사용하기 위해서는 환경을 구축하고 실행(활성화)해야 하고, 실습이 끝난 이후에는 환경을 비활성화시켜야 한다. jupyter notebook에서 가상 환경을 실행하는 구체적인 순서 및 실행 방법은 아래와 같다.

- Anoconda prompt 실행



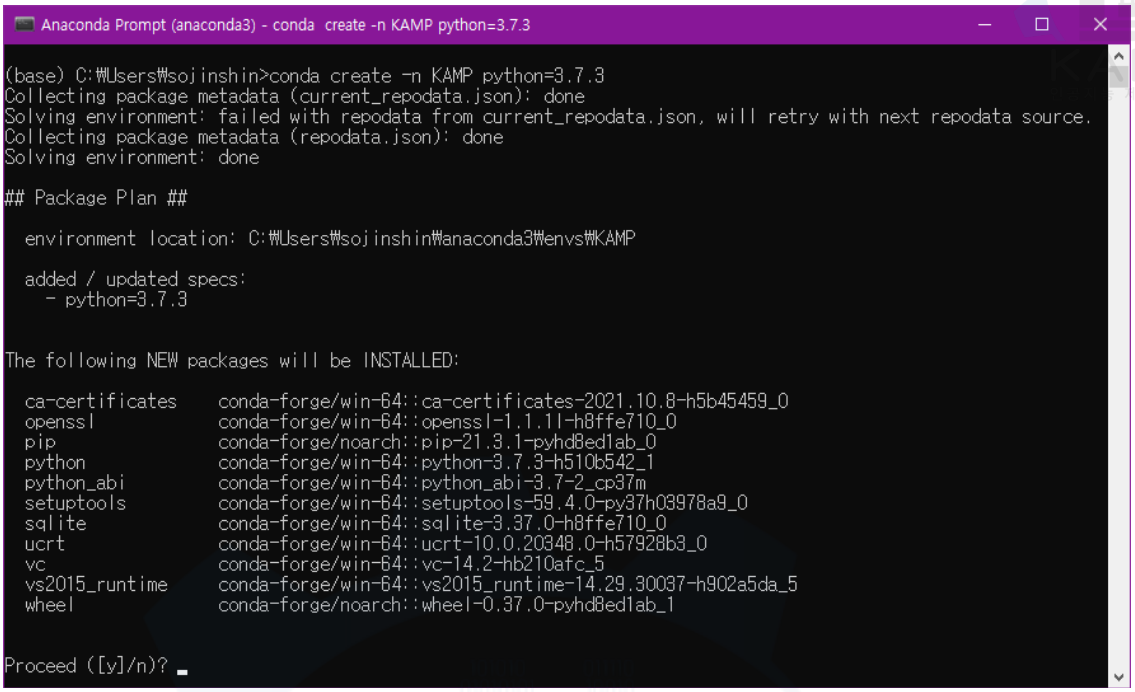
[그림 20] Anaconda prompt 실행 방법



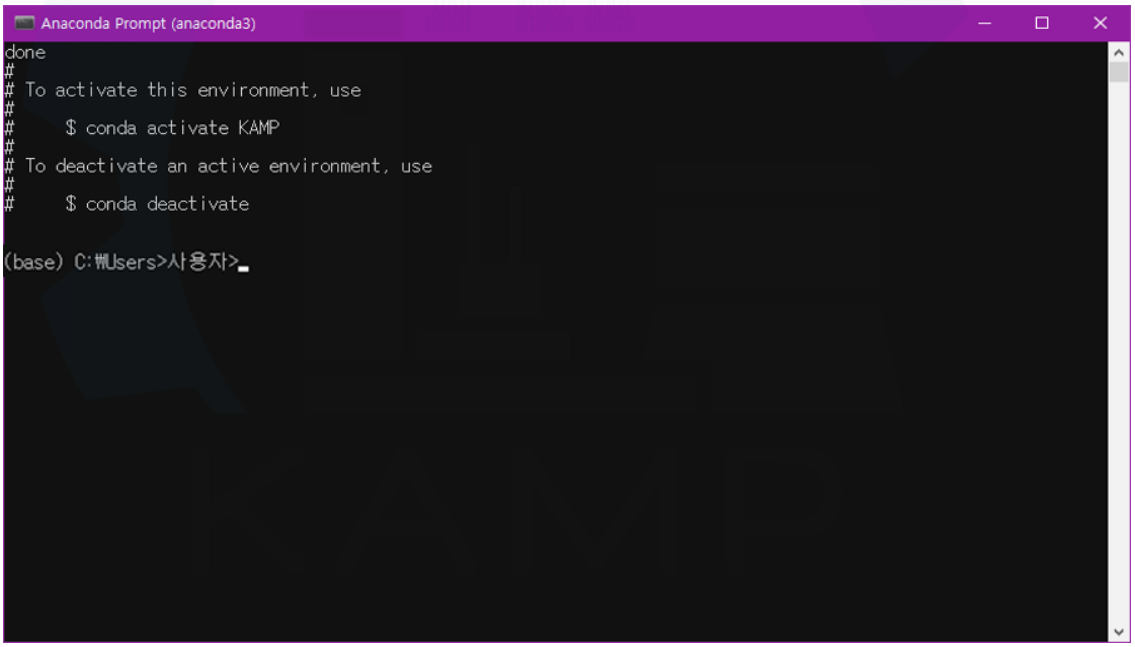
[그림 21] Anaconda prompt 실행 결과

- 가상 환경 생성 : Anaconda prompt 창에 다음을 입력하여 가상 환경을 생성한다.

```
# conda create -n 가상환경이름 python=버전
conda create -n KAMP python=3.7.3
```



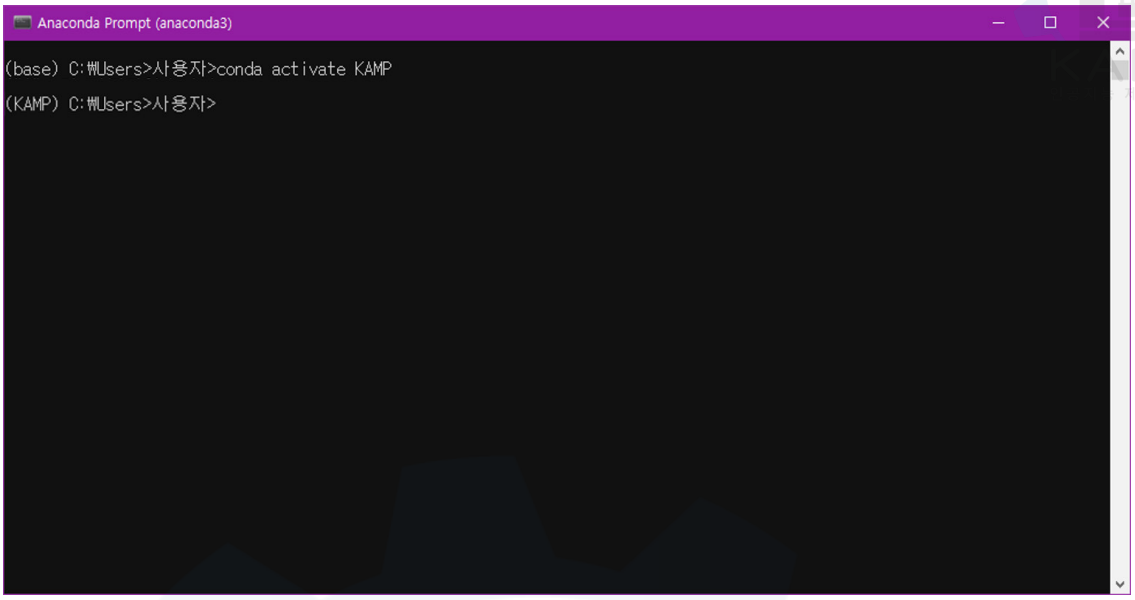
[그림 22] 가상 환경 생성 과정 (y를 입력하여 진행)



[그림 23] 가상 환경 생성 결과

- 가상 환경 실행 : 가상 환경이 활성화되면 아래 그림처럼 명령 프롬프트 앞에 (가상 환경 이름)이 표시된다.

```
# conda activate 가상환경이름
conda activate KAMP
```



[그림 24] 가상 환경 실행

- 분석이 끝난 후에는 가상 환경을 종료한다.

```
conda deactivate
```

- jupyter notebook에서 가상 환경 실행

◦ jupyter notebook에서 가상 환경을 실행하기 위해 가상 환경 활성화 상태에서 아래의 코드를 순서대로 작성한다.

```
pip install ipykernel
# python -m ipykernel install --user --name 가상환경이름 --display-name "가상환경이름"
python -m ipykernel install --user --name KAMP —display-name "KAMP"
# jupyter notebook 설치
pip install jupyter notebook
# jupyter notebook 실행
jupyter notebook
```

◦ jupyter notebook 실행창에서 빨간 박스처럼 생성한 가상 환경이 있는 것을 확인할 수 있다. 해당 가상 환경을 선택 후 실습을 진행한다.



[그림 25] jupyter notebook에서 생성한 가상환경 확인

• **필요 라이브러리(패키지, 모듈 포함)**

- 본 가이드북의 분석에 필요한 모든 라이브러리는 requirements.txt에 버전을 포함하여 기재되어 있으며, 다음의 [코드 1]로 설치할 수 있다.

• **라이브러리 설치**

- jupyter notebook에서 라이브러리 설치하기

```
!pip install -r requirements.txt
```

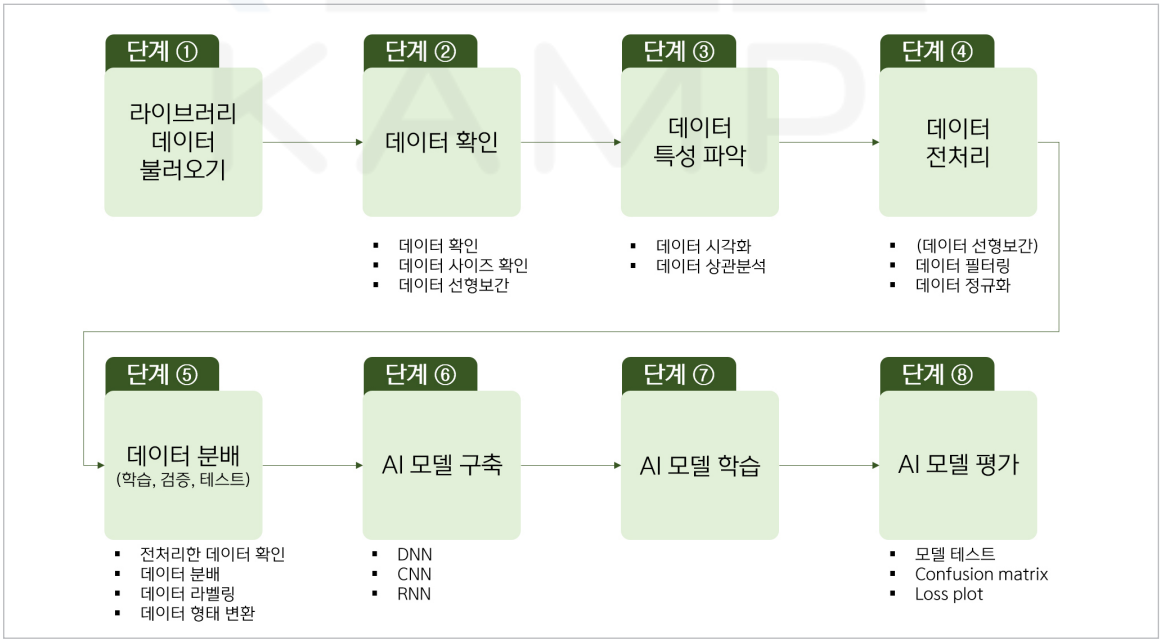
- 본 가이드북에서 사용한 라이브러리([표 13] 참고)

numpy	선형대수 라이브러리(다차원 배열의 데이터 처리에 유용한 기능 제공)
pandas	데이터 분석 라이브러리(테이블 형태의 데이터 처리에 유용한 기능 제공)
scipy	수치계산 라이브러리(과학 기술 계산을 위한 여러 기능 제공)
sklearn	머신러닝 라이브러리(머신러닝 분석을 위한 여러 기능 제공)
torch	머신러닝 라이브러리(기본 data structure : tensor)
matplotlib	데이터 시각화 라이브러리(numpy를 활용하여 plot)
seaborn	데이터 시각화 라이브러리(matplotlib에 기반하여 통계 그래프를 plot)

[표 13] 가이드북에서 사용한 라이브러리 목록

2) 분석 단계별 프로세스 - Flow Chart

• **본 가이드북의 머신러닝 파이프라인(machine learning pipeline)**



[그림 26] 머신러닝 파이프라인

- 분석을 위한 환경 구축을 완료하였다면, 위 파이프라인 [그림 26]을 토대로 이제 실제적인 데이터 분석을 위한 모델을 구현한다.

• 분석 단계별 세부 설명

[단계 ①] 라이브러리 / 데이터 불러오기

①-1. 라이브러리 불러오기(import)

- 라이브러리 불러오기는 다음의 코드로 실행되며, 약자는 사용자가 임의로 지정할 수 있으나, 코드의 재사용을 위해 흔히 통용되는 약자를 사용한다.

  • import 라이브러리이름
  • import 라이브러리이름 as 약어

```
import numpy as np
import pandas as pd
import random
import seaborn as sns
import torch
import torch.nn as nn
import matplotlib.pyplot as plt
```

[코드 2] 라이브러리 불러오기

①-2. 데이터 불러오기(pandas)

- pd.read_csv("데이터파일_경로명")을 통해(*.csv) 형태의 데이터를 pandas의 DataFrame으로 읽어 들인다.

```
# group 1의 데이터를 사용할 경우, 아래 코드를 실행합니다.
sensor1 = pd.read_csv('raw_data/g1_sensor1.csv', names = ['time', 'normal', 'type1', 'type2', 'type3'])
sensor2 = pd.read_csv('raw_data/g1_sensor2.csv', names = ['time', 'normal', 'type1', 'type2', 'type3'])
sensor3 = pd.read_csv('raw_data/g1_sensor3.csv', names = ['time', 'normal', 'type1', 'type2', 'type3'])
sensor4 = pd.read_csv('raw_data/g1_sensor4.csv', names = ['time', 'normal', 'type1', 'type2', 'type3'])
```

[코드 3] 데이터 불러오기

[단계 ②] 데이터 확인

②-1. 데이터 확인(pandas)

- 데이터를 제대로 불러왔는지 확인하기 위해, 아래의 코드를 사용한다.

```
sensor1
```

	time	normal	type1	type2	type3
0	0.000000	-0.853307	-3.464579	0.555219	3.919664
1	0.000736	-0.740463	-2.448986	-0.234687	4.145351
2	0.001472	-0.138630	-1.922383	-0.009000	2.941685
3	0.002208	0.049443	-0.906790	-0.272301	2.603155
4	0.002944	-0.289088	-0.568259	-0.986978	1.361874
5	0.003680	-0.401932	0.447334	-0.798905	1.060958
6	0.004416	0.312744	1.237239	-1.062207	0.233438
7	0.005152	-0.138630	1.876687	-1.475967	-0.481239
8	0.005888	-0.101016	3.117967	-1.363123	-0.293166
9	0.006624	0.162286	3.569341	-1.701654	-1.572061
10	0.007360	0.839348	3.682185	-1.475967	-2.474810

[코드 4] 데이터 확인

- 총 5개의 column을 갖는 sensor 1의 유형별 데이터를 확인할 수 있다.

②-2. 데이터 사이즈 확인(data.shape)

- 데이터의 사이즈를 확인하기 위해, 아래의 코드를 사용한다.

```
print('sensor 1의 데이터 크기', sensor1.shape)
print('sensor 2의 데이터 크기', sensor2.shape)
print('sensor 3의 데이터 크기', sensor3.shape)
print('sensor 4의 데이터 크기', sensor4.shape)
```

```
sensor 1의 데이터 크기 (190218, 5)
sensor 2의 데이터 크기 (184211, 5)
sensor 3의 데이터 크기 (196079, 5)
sensor 4의 데이터 크기 (183969, 5)
```

[코드 5] 데이터 사이즈 확인

- 4개의 센서로부터 140초간 수집한 데이터의 사이즈를 확인할 수 있다. 여기에서 데이터의 열(row) 값이 다른 것을 확인 할 수 있는데, 이는 각 센서의 sampling rate(샘플링 속도)가 다르기 때문이다.

- 센서별 sampling period([표 13] 참고)

sensor 1	sensor 2	sensor 3	sensor 4
0.000736	0.000760	0.000714	0.000761

[표 13] 센서의 sampling period

②-3. 데이터 선형보간(interpolate.interp1d())

- 센서로부터 계측된 데이터의 시간 간격(sampling period)이 일정하지 않은 경우, 데이터 간 정확한 비교가 어렵다. 따라서 계측 시간의 동일화를 위해 선형보간 함수(interpolate.interp1d())를 이용하였다. (자세한 설명은 [분석 모델 소개-전처리]에서 확인할 수 있음)

```
from scipy import interpolate
x_new = np.arange(0, 140, 0.001)
y_new1 = []; y_new2 = []; y_new3 = []; y_new4 = []
for item in ['normal', 'type1', 'type2', 'type3']:
    f_linear1 = interpolate.interp1d(sensor1['time'], sensor1[item], kind='linear');
y_new1.append(f_linear1(x_new))
    f_linear2 = interpolate.interp1d(sensor2['time'], sensor2[item], kind='linear');
y_new2.append(f_linear2(x_new))
    f_linear3 = interpolate.interp1d(sensor3['time'], sensor3[item], kind='linear');
y_new3.append(f_linear3(x_new))
    f_linear4 = interpolate.interp1d(sensor4['time'], sensor4[item], kind='linear');
y_new4.append(f_linear4(x_new))

sensor1 = pd.DataFrame(np.array(y_new1).T, columns = ['normal', 'type1', 'type2', 'type3'])
sensor2 = pd.DataFrame(np.array(y_new2).T, columns = ['normal', 'type1', 'type2', 'type3'])
sensor3 = pd.DataFrame(np.array(y_new3).T, columns = ['normal', 'type1', 'type2', 'type3']))
sensor4 = pd.DataFrame(np.array(y_new4).T, columns = ['normal', 'type1', 'type2', 'type3'])
```

```
sensor1
```

	normal	type1	type2	type3
0	-0.853307	-3.464579	0.555219	3.919664
1	-0.524588	-2.260096	-0.153734	3.713601
2	-0.003708	-1.193805	-0.197890	2.698827
3	-0.297674	-0.490986	-0.972668	1.338978
4	-0.091203	0.790771	-0.913384	0.701167
5	-0.045412	1.744627	-1.390517	-0.333642
6	-0.060948	3.186654	-1.414639	-0.487780
7	0.508177	3.626990	-1.586357	-2.033248
8	-0.043776	4.336351	-1.377842	-2.867309
...				
139994	-0.685354	-0.792309	0.531309	-0.723412
139995	-1.057411	-1.389236	0.664186	-1.498190
139996	-0.513635	-1.368793	1.050553	-0.982626
139997	-1.085213	-2.281763	0.868204	-2.144997
139998	-0.677177	-1.887220	0.792975	-2.146224
139999	-1.387764	-2.427315	1.187928	-3.057968

140000 rows × 4 columns

[코드 6] 데이터 선형보간

- resampling 결과 센서 데이터의 사이즈는 (140000, 4)이다.

②-4. 1차 가공 데이터셋

- 선형보간 작업을 위해 데이터를 센서별로 묶어두었지만, 실제 사용할 데이터는 기계 상태에 따라 4개의 지점에서 4개의 센서로부터 취득한 데이터이므로 다음과 같이 실제 취득된 형태로 데이터를 다시 조정한다.

```
normal_ = pd.concat([sensor1['normal'], sensor2['normal'], sensor3['normal'],
sensor4['normal']], axis=1)
type1_  = pd.concat([sensor1['type1'], sensor2['type1'], sensor3['type1'],
sensor4['type1']], axis=1)
type2_  = pd.concat([sensor1['type2'], sensor2['type2'], sensor3['type2'],
sensor4['type2']], axis=1)
type3_  = pd.concat([sensor1['type3'], sensor2['type3'], sensor3['type3'],
sensor4['type3']], axis=1)

normal_.columns = ['s1', 's2', 's3', 's4']; type1_.columns = ['s1', 's2', 's3', 's4']
type2_.columns  = ['s1', 's2', 's3', 's4']; type3_.columns = ['s1', 's2', 's3', 's4']
```

```
normal_
```

	s1	s2	s3	s4
0	-0.853307	0.048823	-0.437626	-1.116226
1	-0.524588	-0.029477	-0.149817	-0.659694
2	-0.003708	-0.004751	0.280889	-0.857345
3	-0.297674	0.009673	-0.303281	-0.685318
4	-0.091203	0.096215	-0.320263	-0.303108
5	-0.045412	0.009673	-0.079500	-0.484763
6	-0.060948	0.009673	-0.210826	0.150928
7	0.508177	0.024096	0.154973	0.154443
8	-0.043776	-0.027417	-0.354981	0.291474
9	0.865924	-0.023295	0.102392	0.264322
10	0.598124	0.009673	-0.523163	-0.646806
11	0.610390	0.009673	0.181263	0.235294

[코드 7] 데이터 조정

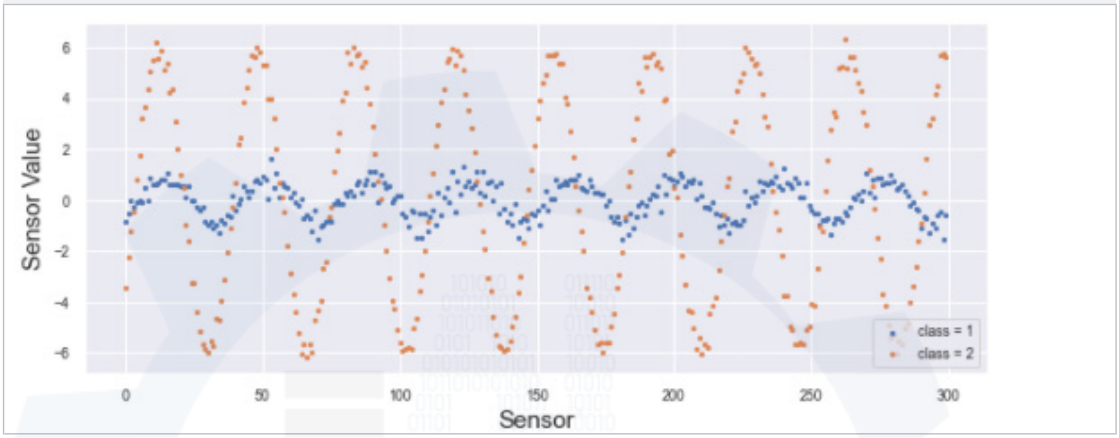
- 1차 가공 데이터는 다음과 같다(각 유형별 4개의 센서 데이터).
  - normal data :(140000, 4)
  - type1 data :(140000, 4)
  - type2 data :(140000, 4)
  - type3 data :(140000, 4)

[단계 ③] 데이터 특성 파악

③-1. 데이터 시각화(plt.scatter)

```
# 실행되지 않는 경우, 아래 코드는 주석처리하고 넘어가도 괜찮습니다. (데이터 확인용)
plt.figure(figsize = (10, 4))
plt.scatter(range(0,300), normal_['s1'][:300], label="class = "+str(1), marker='o', s=5)
plt.scatter(range(0,300), type1_['s1'][:300], label="class = "+str(2), marker='o', s=5)

plt.legend(loc="lower right")
plt.xlabel("Sensor", fontsize=15)
plt.ylabel("Sensor Value", fontsize=15)
plt.show()
plt.close()
```



[코드 8] 데이터 시각화

- 선형보간된 데이터를 시각화하기 위해 plt.scatter 함수를 사용하였다. 위 그래프는 정상상태(class 1)와 비정상상태(class 2)에서의 센서 1의 시간에 따른 계측 값을 보여준다.

③-2. 데이터 상관분석(df.corr / np.corrcoef)

- df.corr()을 입력하면 상태 데이터의 센서별 상관관계를 확인할 수 있다.

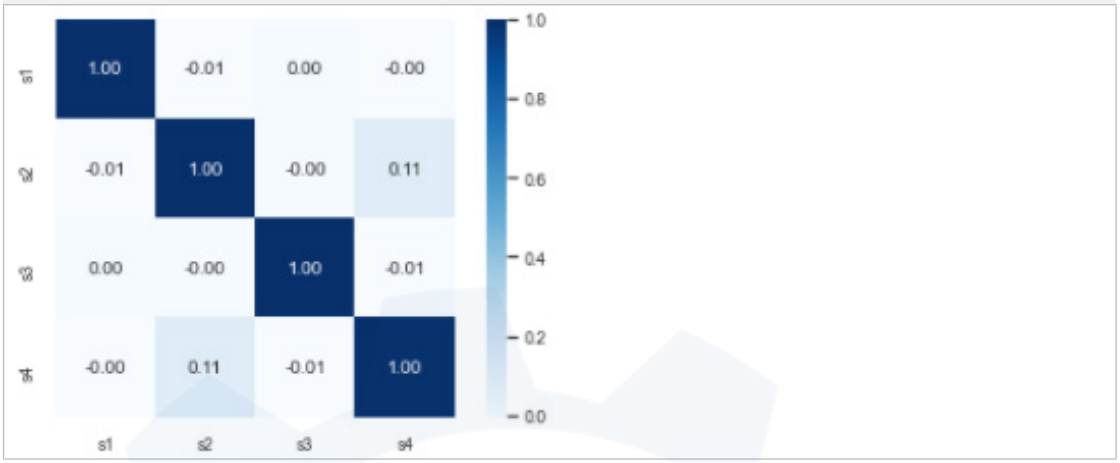
```
normal_.corr()
```

	s1	s2	s3	s4
s1	1.000000	-0.007093	0.004297	-0.003377
s2	-0.007093	1.000000	-0.002992	0.111121
s3	0.004297	-0.002992	1.000000	-0.006434
s4	-0.003377	0.111121	-0.006434	1.000000

[코드 9] 데이터 상관분석

- Correlation Matrix(상관행렬)를 통해 센서별 상관관계를 시각화 할 수 있다.

```
# 실행되지 않는 경우, 아래 코드는 주석처리하고 넘어가도 괜찮습니다. (데이터 확인용)
names = ['s1','s2','s3','s4']
cm = np.corrcoef(normal_[names].values.T)
sns.set(font_scale=0.8)
sns.heatmap(cm, annot=True, square=True, fmt='.2f', annot_kws={'size': 10},
yticklabels=names, xticklabels=names, cmap=plt.cm.Blues)
```



[코드 10] 데이터 상관관계 시각화

- 이러한 상관관계 분석을 통해 비교하려는 두 센서 사이의 선형 상관관계의 여부를 확인할 수 있다. 본 가이드북에서는 일반적으로 이용되는 피어슨 상관계수로(-1, 1)의 값을 갖는다. 값이 1에 가까울수록 센서 간의 선형관계가 강하다고 볼 수 있으며, 양수일 경우 정(+)의 상관관계를 음수일 경우 부(-)의 상관관계를 나타낸다. 절대값이 0.6보다 클 수록 강한 상관관계를 가지며, 0.3보다 작을수록 약한 상관관계를 갖는다. 만일 두 센서 사이에 비선형 상관관계가 있다면 이를 통해서는 확인하기 어렵지만, 선형적인 상관관계는 정량적으로 확인 할 수 있다.

[단계 ④] 데이터 전처리

[단계 ①~③]을 통해 1차 가공된 데이터를 AI 분석 모델 훈련에 사용하기 위해 사용한 전처리는 총 3가지(선형보간, 필터링, 정규화)로 구분되며, ②-3에서 선형보간된 데이터를 가지고 시작한다.

④-1. (데이터 선형보간)

②-3에서 선형보간한 1차 가공 데이터를 가지고 다음 단계로 넘어간다.

④-2. 데이터 필터링(np.convolve())

- 센서로부터 취득한 데이터는 기본적으로 AI 분석 모델 훈련에 좋지 않은 영향을 주는 노이즈(Noise)가 포함되어 있다. 따라서 본 가이드북에서는 데이터 필터링을 위

해 이동평균필터를 구현한 함수(np.convolve())를 사용하였다. (자세한 설명은 [분석 모델 소개-전처리]에서 확인할 수 있음)

```
M =15

normal_s1 = np.convolve(normal_['s1'], np.ones(M), 'valid') / M; normal_s1 =
normal_s1.reshape(len(normal_s1),1)
normal_s2 = np.convolve(normal_['s2'], np.ones(M), 'valid') / M; normal_s2 =
normal_s2.reshape(len(normal_s2),1)
normal_s3 = np.convolve(normal_['s3'], np.ones(M), 'valid') / M; normal_s3 =
normal_s3.reshape(len(normal_s3),1)
normal_s4 = np.convolve(normal_['s4'], np.ones(M), 'valid') / M; normal_s4 =
normal_s4.reshape(len(normal_s4),1)

type1_s1 = np.convolve(type1_['s1'], np.ones(M), 'valid') / M; type1_s1 =
type1_s1.reshape(len(type1_s1),1)
type1_s2 = np.convolve(type1_['s2'], np.ones(M), 'valid') / M; type1_s2 =
type1_s2.reshape(len(type1_s2),1)
type1_s3 = np.convolve(type1_['s3'], np.ones(M), 'valid') / M; type1_s3 =
type1_s3.reshape(len(type1_s3),1)
type1_s4 = np.convolve(type1_['s4'], np.ones(M), 'valid') / M; type1_s4 =
type1_s4.reshape(len(type1_s4),1)

type2_s1 = np.convolve(type2_['s1'], np.ones(M), 'valid') / M; type2_s1 =
type2_s1.reshape(len(type2_s1),1)
type2_s2 = np.convolve(type2_['s2'], np.ones(M), 'valid') / M; type2_s2 =
type2_s2.reshape(len(type2_s2),1)
type2_s3 = np.convolve(type2_['s3'], np.ones(M), 'valid') / M; type2_s3 =
type2_s3.reshape(len(type2_s3),1)
type2_s4 = np.convolve(type2_['s4'], np.ones(M), 'valid') / M; type2_s4 =
type2_s4.reshape(len(type2_s4),1)

type3_s1 = np.convolve(type3_['s1'], np.ones(M), 'valid') / M; type3_s1 =
type3_s1.reshape(len(type3_s1),1)
type3_s2 = np.convolve(type3_['s2'], np.ones(M), 'valid') / M; type3_s2 =
type3_s2.reshape(len(type3_s2),1)
type3_s3 = np.convolve(type3_['s3'], np.ones(M), 'valid') / M; type3_s3 =
type3_s3.reshape(len(type3_s3),1)
type3_s4 = np.convolve(type3_['s4'], np.ones(M), 'valid') / M; type3_s4 =
type3_s4.reshape(len(type3_s4),1)

normal_temp = np.concatenate((normal_s1,normal_s2,normal_s3,normal_s4), axis =1)
type1_temp = np.concatenate((type1_s1,type1_s2,type1_s3,type1_s4), axis =1)
type2_temp = np.concatenate((type2_s1,type2_s2,type2_s3,type2_s4), axis =1)
type3_temp = np.concatenate((type3_s1,type3_s2,type3_s3,type3_s4), axis =1)
```

[코드 11] 데이터 필터링(이동평균필터)

- np.convolve() : 1차원 신호를 입력으로 받아, 최근 M개의 측정값들의 평균을 구한다. 일반적으로 신호 데이터의 변동이 빠른 경우에는 M값을 작게 주고, 신호 데이터의 변동이 느린 경우에는 M값을 크게 준다.
- np.concatenate() : 1차원 필터링이므로 센서별로 나눠서 필터링한 데이터를 다시 하나로 합쳐주는 과정이다.

④-3. 데이터 정규화(MinMaxScaler())

- 센서 값들은 다양한 범위의 값을 갖는다. 이렇듯 데이터의 feature scale의 차이가 심한 경우, AI 분석 모델 훈련 성능에 안 좋은 영향을 미칠 수 있다. 본 가이드북에서는 데이터 정규화를 위해 min-max normalization을 구현한 함수(sklearn.preprocessing-MinMaxScaler())를 사용하였다. (자세한 설명은 [분석 모델 소개-전처리]에서 확인할 수 있음)

```
from sklearn.preprocessing import MinMaxScaler
scaler = MinMaxScaler()
scaler.fit(normal_)
normal = scaler.transform(normal_temp)
type1  = scaler.transform(type1_temp)
type2  = scaler.transform(type2_temp)
type3  = scaler.transform(type3_temp)
```

[코드 12] 데이터 정규화

- MinMaxScaler.fit : normal(정상 상태) data의 min, max를 구한다.
- MinMaxScaler.transform : type 1~3(고장유형) 데이터를 normal(정상 상태) data의 min, max 값으로 정규화한다.

[단계 ⑤] 데이터 분배(학습 / 검증 / 테스트)

⑤-1. 전처리한 데이터 확인(2차 가공 데이터셋)

- 선형보간, 필터링, 정규화를 완료한 데이터셋을 확인한다. 데이터가 (0, 1)로 정규화된 것을 확인할 수 있으며, (140000, 4) 사이즈의 데이터가 줄어든 것을 확인할 수 있는데 이는 이동평균필터를 사용한 영향이다. 이동평균 필터의 M값을 15을 사용한 경우 정확한 평균 값처리를 위해 앞선 15개의 데이터로 평균을 구하므로 최종적으로는 최초의 9개의 데이터는 평균 계산을 위해서만 사용된다. 따라서 데이터의 raws는 140,000-(15-1) = 139,986 이 된다.

```
print(normal)
print('-------------------------------------------------')
print('normal data size = ', normal.shape)
```

```
[[0.57598822 0.50553898 0.56439491 0.44551164]
 [0.60960317 0.50513674 0.58169332 0.44848001]
 [0.62990663 0.5047345  0.59547186 0.46260474]
 ...
 [0.51874981 0.50106998 0.60999194 0.50715712]
 [0.49713039 0.51588928 0.6074125  0.53093987]
 [0.45708747 0.52025039 0.60349765 0.53976513]]
-------------------------------------------------
normal data size =  (139986, 4)
```

[코드 13] 데이터 확인

- 본 가이드북에서는 숫자 확인의 효율성을 위해 데이터의 사이즈를(100000, 4)로 조정하였다. 특히 정상 상태(steady state)의 데이터를 AI 분석 모델 학습에 이용하기 위해, rotor testbed 가동 후 몇 초간(약 30초)의 데이터를 탈락시키고 이후 데이터를 사용한다.

```
normal = normal[30000:130000][:]
type1  = type1[30000:130000][:]
type2  = type2[30000:130000][:]
type3  = type3[30000:130000][:]
print(normal)
print('--------------------------------------------------')
print('normal data size = ', normal.shape)
```

```
[[0.42588581 0.56360116 0.59076635 0.45259554]
 [0.39980512 0.54859132 0.57699856 0.4486034 ]
 [0.38031149 0.54190146 0.57998374 0.44726771]
 ...
 [0.48028648 0.53048363 0.62114742 0.51024851]
 [0.44818273 0.53444251 0.62103995 0.49126006]
 [0.42832757 0.5501298  0.60917718 0.47827626]]
--------------------------------------------------
normal data size =  (100000, 4)
```

[코드 14] 데이터 조정 및 확인

⑤-2. 데이터 분배

- 데이터 분배(학습 / 검증 / 테스트)

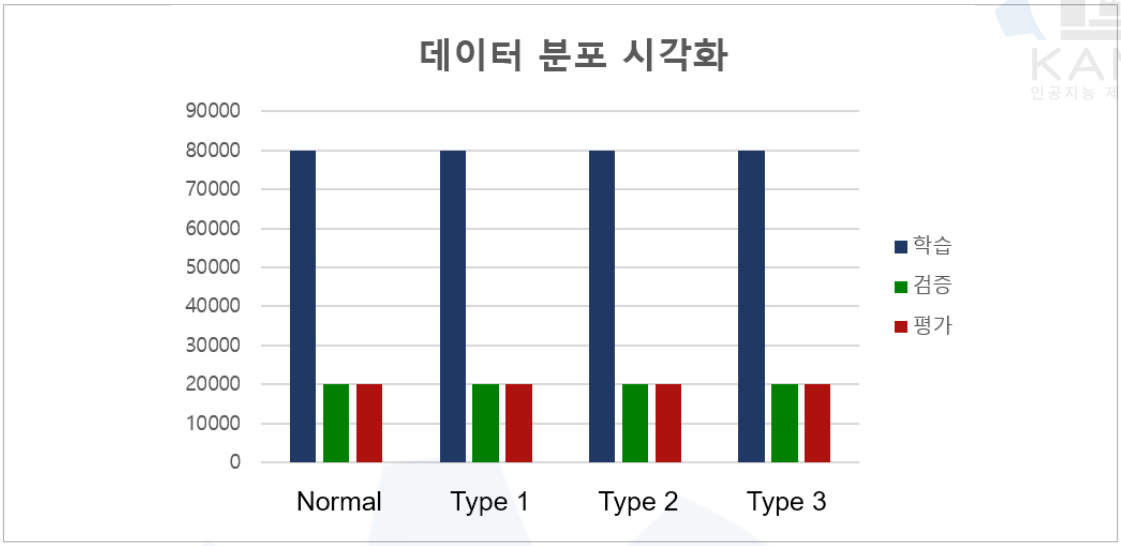
```
# 데이터 분배
normal_train = normal[:][:60000]; normal_valid = normal[:][60000:80000]; normal_test =
normal[:][80000:]
type1_train  = type1[:][:60000];  type1_valid  = type1[:][60000:80000];  type1_test  =
type1[:][80000:]
type2_train  = type2[:][:60000];  type2_valid  = type2[:][60000:80000];  type2_test  =
type2[:][80000:]
type3_train  = type3[:][:60000];  type3_valid  = type3[:][60000:80000];  type3_test  =
type3[:][80000:]

train = np.concatenate((normal_train,type1_train,type2_train,type3_train))
valid = np.concatenate((normal_valid,type1_valid,type2_valid,type3_valid))
test = np.concatenate((normal_test,type1_test,type2_test,type3_test))
print("train data의 형태:", train.shape)
print("valid data의 형태:", valid.shape)
print(" test data의 형태:", test.shape)
```

```
train data의 형태: (240000, 4)
valid data의 형태: (80000, 4)
 test data의 형태: (80000, 4)
```

[코드 15] 데이터 분배

- Normal, type 1~3 abnormal data를 1:1:1:1의 비율로 각각 training data, validation data, test data로 분배한다. 그 결과(학습 : 검증 : 테스트 = 6 : 2 : 2)이다. [그림 27]은 데이터의 분배 결과를 시각화한 차트이다.



[그림 27] 데이터 분포 시각화

⑤-3. 데이터 라벨링(np.concatenate())

- 본 가이드북에서는 지도학습을 통해 3가지 신경망을 훈련시킨다. 따라서 각 유형별 데이터에 순서대로 [표 14]와 같은 라벨을 부여한다. python은 0부터 시작하는 프로그래밍 언어이므로 0부터 순서대로 회전기계의 상태를 나타낼 수 있도록 라벨링 하였다.

Normal	Type 1	Type 2	Type 3
0	1	2	3

[표 14] 각 상태 유형에 따른 라벨

```
train_label = np.concatenate((np.full((60000,1),0), np.full((60000,1),1),
np.full((60000,1),2), np.full((60000,1),3)))
valid_label = np.concatenate((np.full((20000,1),0), np.full((20000,1),1),
np.full((20000,1),2), np.full((20000,1),3)))
test_label = np.concatenate((np.full((20000,1),0), np.full((20000,1),1),
np.full((20000,1),2), np.full((20000,1),3)))
```

[코드 16] 데이터 라벨링

- 데이터 뒤섞기(np.random.shuffle()) : AI 분석 모델이 데이터를 학습하는 과정에서 데이터의 순서에 의존하지 않고 임의의 데이터가 입력되는 것으로 간주하기 위해 라벨이 부여된 데이터를 뒤섞는다(shuffling).

```
idx = np.arange(train.shape[0]); np.random.shuffle(idx);
train = train[:][idx]; train_label = train_label[:][idx]

idx_v = np.arange(valid.shape[0]); np.random.shuffle(idx_v);
valid = valid[:][idx_v]; valid_label = valid_label[:][idx_v]

idx_t = np.arange(test.shape[0]); np.random.shuffle(idx_t);
test = test[:][idx_t]; test_label = test_label[:][idx_t]
```

[코드 17] 데이터 뒤섞기

⑤-4. 데이터 형태 변환(torch.from_numpy().float())

- 본 가이드북에서 사용하는 AI 분석 모델은 pytorch의 머신러닝 모듈이다. pytorch의 경우 tensor 형태의 데이터를 입력으로 받기 때문에, 데이터의 형태를(torch.from_numpy().float()) 함수를 사용하여 array에서 tensor 형태로 변환한다.

```
x_train = torch.from_numpy(train).float()
y_train = torch.from_numpy(train_label).float().T[0]
x_valid = torch.from_numpy(valid).float()
y_valid = torch.from_numpy(valid_label).float().T[0]
x_test = torch.from_numpy(test).float()
y_test = torch.from_numpy(test_label).float().T[0]
print("변경 전")
train
```

```
array([[0.2270437 , 0.67812094, 0.56094538, 0.59805898],
       [0.60814034, 0.42733266, 0.61705344, 0.66131373],
       [0.90323228, 0.73748879, 0.56732872, 0.62994229],
       ...,
       [0.92709828, 0.3161954 , 0.59485629, 0.32612464],
       [0.40829755, 0.49626763, 0.56937096, 0.47835657],
       [0.35468444, 0.43555095, 0.58058547, 0.42853661]])
```

```
print("변경 후")
x_train
```

```
tensor([[0.2270, 0.6781, 0.5609, 0.5981],
        [0.6081, 0.4273, 0.6171, 0.6613],
        [0.9032, 0.7375, 0.5673, 0.6299],
        ...,
        [0.9271, 0.3162, 0.5949, 0.3261],
        [0.4083, 0.4963, 0.5694, 0.4784],
        [0.3547, 0.4356, 0.5806, 0.4285]])
```

[코드 18] 데이터 형태 변환

TensorDataset와 DataLoader(optional)

- Pytorch에는 Dataset과 DataLoader라는 모듈이 있어서 미니 배치 학습이나 데이터 셔플 등의 기능을 제공해준다.

```
from torch.utils.data import TensorDataset
from torch.utils.data import DataLoader

train = TensorDataset(x_train, y_train)
train_dataloader = DataLoader(train, batch_size =5000, shuffle=True)

valid = TensorDataset(x_valid, y_valid)
valid_dataloader = DataLoader(valid, batch_size =len(x_valid), shuffle=False)

test = TensorDataset(x_test, y_test)
test_dataloader = DataLoader(test, batch_size =len(x_valid), shuffle=False)
```

[코드 19] 데이터 묶기

- TensorDataset : 데이터 x_data와 레이블 y_data를 묶어 놓는 컨테이너이다.
- DataLoader : TensorDataset의 결과가 전달되는데, 이를 통해 batch size나 데이터를 섞을지 여부 등을 결정해 준다.
- training data의 경우 학습을 위해 batch size를 5000으로, 학습을 반복할 때마다 데이터를 다시 섞어주기 위해 shuffle을 사용하였다.
- validation and test data의 경우 학습의 검증과 평가에 사용되므로 따로 batch size 지정하지 않았으며, shuffle 역시 단계 [⑤-3]에서 이미 섞어주었으므로 사용하지 않았다.

본 가이드북에서는 총 3개의 AI 모델을 이용하여 회전기계의 고장유형 AI데이터셋을 분석한다. [단계 ①~⑤]는 AI 분석 모델의 학습과 평가에 이용할 수 있도록 데이터를 전처리하는 과정이었다면, [단계 ⑥]에서는 앞선 단계에서 구축한 데이터가 공통적으로 적용되는 3개의 AI 모델을 설계한다. 이후 [단계 ⑦]에서 [단계 ⑥] AI 모델을 훈련(학습) 시킨다. [단계 ⑧]에서는 [단계 ⑦]에서 훈련된 AI 모델을 평가하고, 결과를 분석한다. 아래는 코드에서 3개의 AI 모델을 구분하기 위한 색을 표시하였다.

공통
심층신경망(DNN)
합성곱신경망(CNN)
순환신경망(RNN)

[단계 ⑥] AI 모델 구축

⑥-1. 심층신경망(DNN)

- 심층신경망(DNN)은 인공신경망에서 모델 내 은닉층을 많이 늘려서 학습의 결과를 향상시키는 방법이다. 라벨이 있는 데이터의 지도학습 분류(classification), 회귀(regression) 문제에서 주로 사용된다.

```
# layer를 더 늘리고 싶은 경우 주석을 제거하고 사용합니다.

class KAMP_DNN(nn.Module):
    def __init__(self):
        super(KAMP_DNN, self).__init__()
        self.layer1   = nn.Linear(in_features =4, out_features =100)
        self.layer2   = nn.Linear(in_features =100, out_features =100)
        self.layer3   = nn.Linear(in_features =100, out_features =100)
        self.layer4   = nn.Linear(in_features =100, out_features =4)

        self.dropout  = nn.Dropout(0.2)
        self.relu     = nn.ReLU()

    def forward(self, input):
        out =self.layer1(input)
        out =self.relu(out)
        out =self.dropout(out)

        out =self.layer2(out)
        out =self.relu(out)
        out =self.dropout(out)

        out =self.layer3(out)
        out =self.relu(out)
        out =self.dropout(out)

        out =self.layer4(out)
        return out

model_check = KAMP_DNN()
print(model_check)
```

```
KAMP_DNN(
  (layer1): Linear(in_features=4, out_features=100, bias=True)
  (layer2): Linear(in_features=100, out_features=100, bias=True)
  (layer3): Linear(in_features=100, out_features=100, bias=True)
  (layer4): Linear(in_features=100, out_features=4, bias=True)
  (dropout): Dropout(p=0.2, inplace=False)
  (relu): ReLU()
)
```

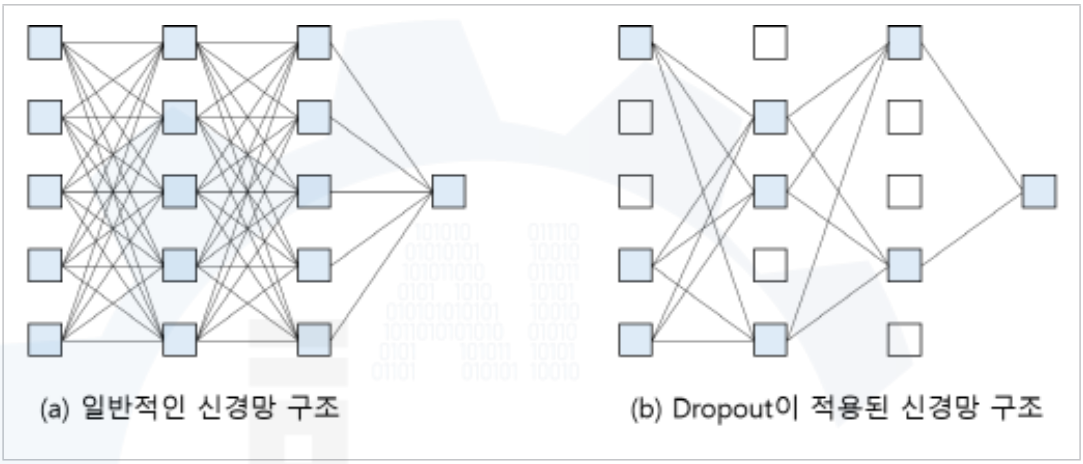
[코드 20] 심층신경망 모델 구축

1. KAMP_ ss는 nn.Module class를 상속한다.
2. __init__()에서는 신경망 layer를 초기화한다.
   - nn.Linear() : 선형계층으로 weight와 bias을 사용하여 입력에 선형 변환을 적용하는 모듈
     - in_features : 신경망으로 입력되는 size
     - out_features : 신경망에서 출력되는 size

• nn.ReLU() : ReLU 활성화함수 계층으로 비선형 활성화를 통해 모델의 입력과 출력 사이에 복잡한 관계(mapping)를 만든다.

◦ 비선형 활성화는 nn.Linear로 이루어진 선형계층에 비선형성을 도입하여 신경망이 보다 복잡한 현상을 학습할 수 있도록 돕는다.

• nn.Dropout() : 드롭아웃 계층으로 신경망 일부를 의도적으로 학습에서 제외함으로써 모델을 일반화하는 역할을 한다. (과적합 방지; [그림 28] 참고)



[그림 28](a) 일반적인 신경망(b) Dropout이 적용된 신경망의 구조

• input_size = 4 : 계측되는 sensor 값들 4개

• ouput_size = 4 : 정상 + 고장유형 개수 4개

• 본 가이드북에서 은닉층의 뉴런수를 100으로 하였으나, data에 따라 조절

• layer의 개수는 데이터에 따라 조절(여기서는 2~4개의 layer 사용)

3. forward()에서는 초기화된 layer에 입력 데이터를 전달하고 최종 출력을 얻는다.

⑥-2. 합성곱신경망(CNN)

- 합성곱신경망(CNN)은 인공신경망의 한 종류이며, 주로 Matrix 데이터나 이미지 데이터에서의 특징 추출에 쓰인다.

```
# layer를 더 늘리고 싶은 경우 주석을 제거하고 사용합니다.

class KAMP_CNN(nn.Module):
    def __init__(self):
        super(KAMP_CNN, self).__init__()
        self.conv1 = nn.Sequential(
        nn.Conv1d(in_channels=1, out_channels=100, kernel_size=2, stride=1, padding='same'),
        nn.BatchNorm1d(100),
        nn.ReLU(),
        nn.MaxPool1d(kernel_size=1, stride=1),
        nn.Dropout(p=0.2))

#         self.conv2 = nn.Sequential(
#         nn.Conv1d(in_channels=100, out_channels=100, kernel_size=2, stride=1, padding='same'),
#         nn.BatchNorm1d(100),
#         nn.ReLU(),
#         nn.MaxPool1d(kernel_size=1, stride=1),
#         nn.Dropout(p=0.2))

#         self.conv3 = nn.Sequential(
#         nn.Conv1d(in_channels=100, out_channels=100, kernel_size=2, stride=1, padding='same'),
#         nn.BatchNorm1d(100),
#         nn.ReLU(),
#         nn.MaxPool1d(kernel_size=1, stride=1),
#         nn.Dropout(p=0.2))

        self.conv4 = nn.Sequential(
        nn.Conv1d(in_channels=100, out_channels=4, kernel_size=2, stride=1, padding='same'),
        nn.BatchNorm1d(4),
        nn.ReLU(),
        nn.MaxPool1d(kernel_size=1, stride=1))

        self.final_pool = nn.AdaptiveAvgPool1d(1)
        self.linear = nn.Linear(4, 4)
    def forward(self, input):
        input = input.unsqueeze(1)
        out =self.conv1(input)
#         out = self.conv2(out)
#         out = self.conv3(out)
        out =self.conv4(out)
        out =self.final_pool(out)
        out =self.linear(out.squeeze(-1))
        return out
model_check = KAMP_CNN()
print(model_check)
```

```
KAMP_CNN(
  (conv1): Sequential(
    (0): Conv1d(1, 100, kernel_size=(2,), stride=(1,), padding=same)
    (1): BatchNorm1d(100, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
    (2): ReLU()
    (3): MaxPool1d(kernel_size=1, stride=1, padding=0, dilation=1, ceil_mode=False)
    (4): Dropout(p=0.2, inplace=False)
  )
  (conv4): Sequential(
    (0): Conv1d(100, 4, kernel_size=(2,), stride=(1,), padding=same)
    (1): BatchNorm1d(4, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
    (2): ReLU()
    (3): MaxPool1d(kernel_size=1, stride=1, padding=0, dilation=1, ceil_mode=False)
    (4): Dropout(p=0.2, inplace=False)
  )
  (final_pool): AdaptiveAvgPool1d(output_size=1)
  (linear): Linear(in_features=4, out_features=4, bias=True)
)
```

[코드 21] 합성곱신경망 모델 구축

1. KAMP_CNN을 구현하는 class는 nn.Module class를 상속한다.
2. __init__()에서는 신경망 layer를 초기화한다.
   - nn.Sequential() : 계층이 순차적으로 실행되도록 담는 sequential container로서 연산순서를 원하는 데로 다음 계층으로 전달할 수 있다.
     - in_features : 신경망으로 입력되는 size
     - out_features : 신경망에서 출력되는 size

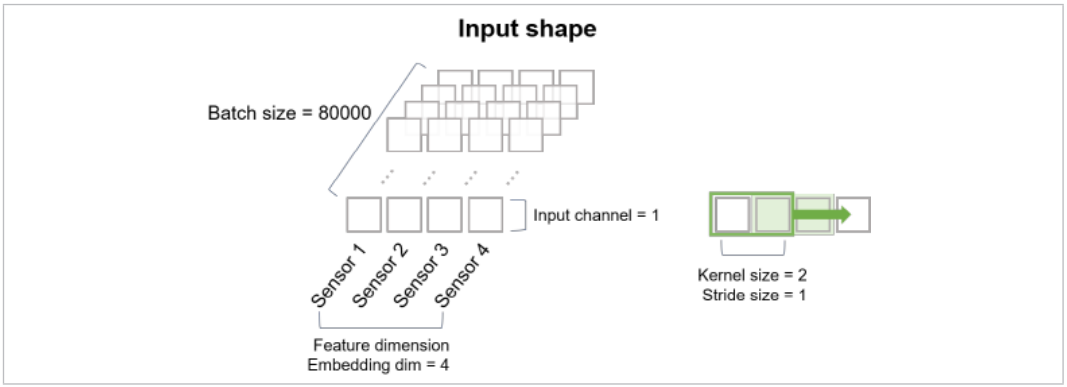
   - nn.Conv1d() : 1차원 합성곱 계층으로서 주로 자연어와 같은 시계열 데이터에 많이 사용된다.
     - 이미지처리에서 주로 사용되는 2차원 합성곱 계층은 filter가 가로와 세로로 이동하면서 연산한다. 반면, 1차원 합성곱 계층의 연산은 filter가 가로로만 이동하면서 output을 계산한다. ([그림 29] 참고)



[그림 29] 1차원 합성곱 계층의 계산 방식 시각화

     - input shape : torch.nn.conv1d()의 입력 사이즈는 [Batch size, Input channel size, sequence length]이다. 데이터를 학습하는 방식에 따라 입력 사이즈를 조절할 수 있으며, 본 가이드북에서 사용한 사이즈는 다음과 같다.

       [Batch size = 80000, Input channel size = 1, sequence length = 4]



[그림 30] 가이드북에서 사용한 합성곱신경망 시각화

◦ in_channels : 신경망으로 입력되는 데이터의 channel size(컬러 이미지의 경우 첫 계층에서 RGB = 3을 사용)

◦ out_channels : 신경망에서 출력되는 데이터의 channel size(본 가이드북에서는 100을 사용하였으나 parameter study로 조절)

◦ kernel_size : 특징을 추출하는 filter의 size

◦ stride : kernel(filter)이 적용될 때, 이동하는 step size(2 이상인 경우, 데이터 size가 줄어든다)

◦ padding : 데이터 가장자리에 0을 채워 넣는 역할(디폴트는 valid) {'vaild' : No padding, 'same' : Zero-padding, 혹은 다른 값}

◦ bias : bias term의 사용 여부(디폴트는 True)

• nn.BatchNorm1d() : Batch 차원의 정규화 계층으로 학습의 안정화에 기여한다.

• nn.ReLU() : ReLU 활성화함수 계층으로 비선형 활성화를 통해 모델의 입력과 출력 사이에 복잡한 관계(mapping)를 만든다.

◦ 비선형 활성화는 nn.Linear로 이루어진 선형계층에 비선형성을 도입하여 신경망이 보다 복잡한 현상을 학습할 수 있도록 돕는다.

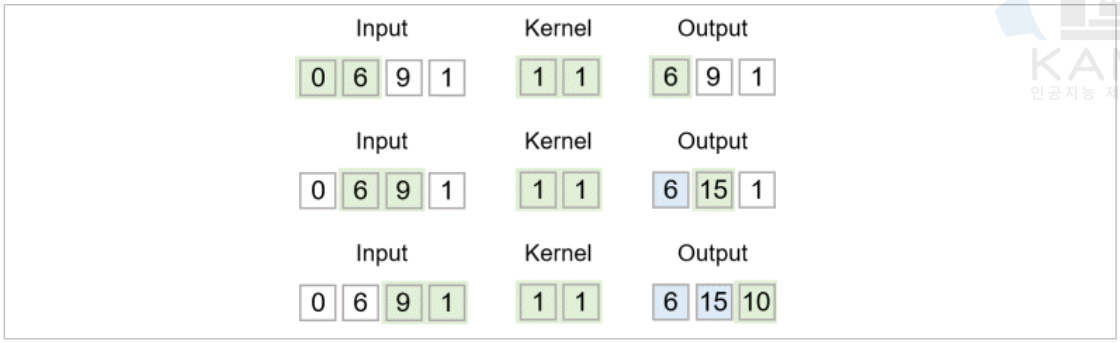
• nn.MaxPool1d() : sub-sampling 계층으로 데이터의 특징을 보존하면서 크기를 줄여주는 역할을 한다(과적합 방지).

• nn.Dropout() : 드롭아웃 계층으로 신경망의 일부를 의도적으로 학습에서 제외함으로써 모델을 일반화하는 역할을 한다(과적합 방지).

• layer의 개수는 데이터에 따라 조절(여기서는 2~4개의 layer 사용)

3. forward()에서는 초기화된 layer에 입력 데이터를 전달하고 최종 출력을 얻는다.

- [그림 31]은 본 가이드북에서 사용한 kernel과 stride size가 합성곱 신경망에서 어떻게 작용하는지 시각적으로 확인하기 위한 예시를 나타내었다. (편의를 위해 out_channels = 4)



[그림 31] 합성곱 연산 시각화

```
print('--------------------------------------------------------------------------------')
input = torch.tensor([[[0.0, 6.0, 9.0, 1.0]]])
print("'input is same below.'")
print(input)
print('--------------------------------------------------------------------------------')
model = nn.Conv1d(1, 4, 2, bias =False)
model.weight.data = torch.zeros(model.weight.data.size())
model.weight.data[:, :, :2] =1
print("'kernal is same below.'")
print(model.weight.data)
print('--------------------------------------------------------------------------------')
output = model(input)
print("'output is same below (without bias).'")
print(output)
print('--------------------------------------------------------------------------------')
model1 = nn.Conv1d(1, 4, 2)
model1.weight.data = torch.zeros(model1.weight.data.size())
model1.weight.data[:, :, :2] =1
output = model1(input)
print("'output is same below (with bias).'")
print(output)
print('--------------------------------------------------------------------------------')
```

```
--------------------------------------------------------------------------------
"input is same below."
tensor([[[0., 6., 9., 1.]]])
--------------------------------------------------------------------------------
"kernal is same below."
tensor([[[1., 1.]],

        [[1., 1.]],

        [[1., 1.]],

        [[1., 1.]]])
--------------------------------------------------------------------------------
"output is same below (without bias)."
tensor([[[ 6., 15., 10.],
         [ 6., 15., 10.],
         [ 6., 15., 10.],
         [ 6., 15., 10.]]], grad_fn=<SqueezeBackward1>)
--------------------------------------------------------------------------------
"output is same below (with bias)."
tensor([[[ 5.8956, 14.8956,  9.8956],
         [ 5.7652, 14.7652,  9.7652],
         [ 6.6935, 15.6935, 10.6935],
         [ 6.3792, 15.3792, 10.3792]]], grad_fn=<SqueezeBackward1>)
--------------------------------------------------------------------------------
```

[코드 22] 합성곱신경망 모델 연산 시각화

⑥-3. 순환신경망(RNN)

구분	특징
RNN	가장 기본적인 모델
LSTM	RNN에서 gradient vanishing 문제 해소
GRU	LSTM보다 빠른 속도

[그림 32] RNN, LSTM, GRU 특징

- 순환신경망(RNN, LSTM, GRU)은 과거의 학습을 가중치를 통해 현재 학습에 반영하는 특징이 있다. 따라서 순차적으로 변화하는 시계열(time series) 데이터의 학습에 탁월한 효과를 갖는다. 본 가이드북에서는 pytorch의 LSTM 모듈을 사용하였다. ([그림 32] 참고)

```
import torch
import torch.nn as nn
class KAMP_RNN(nn.Module):
    def __init__(self):
        super(KAMP_RNN, self).__init__()
        self.lstm    = nn.LSTM(input_size =4, hidden_size =100, num_layers =2,
batch_first=True, dropout =0.2)
        self.fc      = nn.Linear(in_features =100, out_features =4)
    def forward(self, input):
        input = input.unsqueeze(1)
        out, _ =self.lstm(input)
        out = out.view(-1,100)
        output =self.fc(out)
        return output

model_check = KAMP_RNN()
print(model_check)
```

```
KAMP_RNN(
  (lstm): LSTM(4, 100, num_layers=2, batch_first=True, dropout=0.2)
  (fc): Linear(in_features=100, out_features=4, bias=True)
)
```

[코드 23] 순환신경망 모델 구축

1. KAMP_DNN을 구현하는 class는 nn.Module class를 상속한다.
2. __init__()에서는 신경망 layer를 초기화한다.
   - nn.LSTM() : LSTM 순환신경망 계층(Long Short-Term Memory Network)으로서 주로 시계열 데이터 처리에 많이 사용된다.
     - input shape : torch.nn.LSTM의 입력 사이즈는 [Batch size, sequence length, input size]이며, 데이터를 학습시키는 방식에 따라 입력 사이즈를 조절할 수 있으며, 본 가이드북의 입력 사이즈는 다음과 같다.

       input shape = [Batch size = 80000, sequence length = 1, input size = 4]

- input size : 신경망으로 입력되는 데이터의 특징 개수(time step 아님)
  - hidden size : 은닉층에서 사용할 특징 개수(적절한 값 사용)
  - batch_first : [batch, sequence, feature]로 input data 사용(디폴트는 False, [sequence, batch, feature])
  - num_layers : LSTM layer 개수 조절 옵션(여기서는 4개의 layer 사용)
  - dropout : 드롭아웃 옵션(과적합 방지, 디폴트는 0)

- nn.Linear() : 선형계층으로 weight와 bias을 사용하여 입력에 선형 변환을 적용하는 모듈로, 여기서는 LSTM의 fully connected layer로 사용하였다.
  - in_features : 신경망으로 입력되는 size
  - out_features : 신경망에서 출력되는 size

3. forward()에서는 초기화된 layer에 입력 데이터를 전달하고 최종 출력을 얻는다.

[단계 ⑦] AI 모델 학습

⑦-0. 모델 학습 코드(공통)

[단계 ⑥]에서 구축한 3개의 AI 모델을 학습시키기 위한 공통 code는 다음과 같으며, '모델 학습, 모델 검증, 학습 과정 시각화'의 3가지로 구성된다.

```
def train_model(model, criterion, optimizer, num_epoch, train_dataloader, PATH):
    loss_values = []
    loss_values_v = []
    check =0; accuracy_past =0
    for epoch in range(1, num_epochs +1):
        #---------------------- 모델 학습 ----------------------#
        model.train()
        batch_number =0
        running_loss =0.0
        for batch_idx, samples in enumerate(train_dataloader):
            x_train, y_train = samples
            # 변수 초기화
            optimizer.zero_grad()
            y_hat = model.forward(x_train)
            loss = criterion(y_hat,y_train.long())
            loss.backward()
            optimizer.step()
            running_loss += loss.item()
            batch_number +=1
        loss_values.append(running_loss / batch_number)
        #---------------------- 모델 검증 ----------------------#
        model.eval()
        accuracy =0.0
        total =0.0
        for batch_idx, data in enumerate(valid_dataloader):
            x_valid, y_valid = data

            v_hat = model.forward(x_valid)
            v_loss = criterion(v_hat,y_valid.long())
            _, predicted = torch.max(v_hat.data, 1)
            total += y_valid.size(0)
            accuracy += (predicted == y_valid).sum().item()
        loss_values_v.append(loss.item())
        accuracy = (accuracy / total)
        #-----------------Check for early stopping----------------#
        if epoch % 1 ==0:
            print('[Epoch {}/{}] [Train_Loss: {:.6f} /Valid_Loss: {:.6f}]'.format(epoch,
num_epochs, loss.item(),v_loss.item()))
            print('[Epoch {}/{}] [Accuracy  : {:.6f}]'.format(epoch, num_epochs,
accuracy))
        if accuracy_past 〉accuracy:
            check +=1
        else:
            check =0
            accuracy_past = accuracy
        if check 〉50:
            print('This is time to do early stopping')
    torch.save(model, PATH +'model.pt')
    return loss_values, loss_values_v
```

[코드 24] AI 분석 모델 학습(전체)

- train_model()의 입력 데이터

  • model : [단계 ⑥]에서 구축한 DNN, CNN, RNN 모델

  • criterion : loss 계산을 위한 함수(nn.CrossEntropyLoss())

    ◦ label이 존재하는 supervised learning에서 기계학습 모델은 예측값(y_hat)이 실제값(label)

에 가까워지도록 학습이 이루어진다. 이때 예측값과 실제값이 얼마나 차이가 나는지를 나타내는 지표로 loss를 사용한다.

◦ pytorch에서 제공하는 loss 함수([표 15] 참고)

nn.MSELoss	– mean squared error로 loss를 계산하는 함수 – 주로 회귀 문제에 사용되며, 분류 문제에서도 사용
nn.CrossEntropyLoss	– 이진, 다중 분류(classification) 문제를 다룰 때 사용 – nn.SoftMax와 nn.CrossEntropyLoss의 기능을 동시에 제공
nn.BCELoss	– 이진 분류 문제에 주로 사용
nn.BCEWithLogitsLoss	– nn.Sigmoid와 nn.BCELoss의 기능을 동시에 제공 – 두 함수를 각각 사용하는 것보다는 nn.BCEWithLogitsLoss를 사용하는 것이 안정적
nn.NLLLoss	– 분류 문제에서 특정 가중치를 사용해 데이터의 불균형을 처리할 때 사용

[표 15] pytorch에서 제공하는 loss 함수

◦ 본 가이드북에서 사용한 nn.CrossEntropyLoss()은 회전기계 고장유형 AI데이터 셋과 같은 다중 분류 문제를 다룰 때 유용하다.

• optimizer : 최적화 함수(torch.optim.Adam(model.parameters()))

◦ 기계학습에서 네트워크 오차(loss)를 줄이고 알고리즘의 정확도를 높이기 위해 모델의 가중치를 변화시켜주는 역할을 한다.

◦ pytorch에서 제공하는 optimizer 함수([표 16] 참고)

torch.optim.SGD	– loss의 미분값을 이용하여, 기울기를 따라 내려가면서 가중치를 업데이트하는 방법
torch.optim.AdaGrad	– 학습률의 감소(learning rate decay)를 통해 학습률에 따른 학습 효율 및 정확도 문제를 해결하는 방법
torch.optim.RMSprop	– AdaGrad에서 더 나아가 기울기를 단순 누적하지 않고, 지수 가중 이동평균(Exponentially weighted moving average)을 이용하여 최신 기울기를 더 많이 반영하도록 하는 방법
torch.optim.Adam	– SGD + momentum와 RMSprop의 기능을 합친 최적화 함수로 gradient를 효율적으로 조절(보편적으로 많이 사용됨)

[표 16] pytorch에서 제공하는 optimizer 함수

◦ 본 가이드북에서 사용한 torch.optim.Adam()은 SGD, momentum, RMSprop optimizer의 기능들을 합친 최적화 함수로 학습 방향과 스텝 사이즈를 자동으로 맞춰주기 때문에 처음 모델을 구축할 때 사용하면 좋다.

• num_epoch : 전체 데이터의 반복 학습 횟수

• train_dataloader : [단계 ⑤]에서 구축한 전처리가 완료된 데이터

• PATH : 학습 결과가 저장되는 위치

- train_model()의 출력 데이터

• loss_values : 학습 과정 확인을 위한 학습 손실 값 history

• loss_values_v : 학습 과정 확인을 위한 검증 손실 값 history

- 전체 프로세스는 입력한 반복 학습 횟수(epoch)만큼 진행되며 매 epoch마다 모델을 검증한다. 또한, 과적합 문제와 학습 효율성을 위해 학습의 중단지점을 구축해야 한다. 따라서 가장 높았던 검증 정확도(accuracy_past)를 기준으로 정확도가 정해진 epoch 동안 향상되지 않으면 반복 학습을 중단하라는 메시지를 출력하도록 하였다. 메시지만 출력하도록 한 이유는 초기에 AI 분석 모델이 구축되면 최대 반복 횟수까지 학습을 진행하면서 학습 추이를 확인하고 loss plot으로부터 중단지점을 설정하기 위함이다.

〈 모델 학습 〉

```
def train_model(model, criterion, optimizer, num_epoch, train_dataloader, PATH):
    loss_values = []
    loss_values_v = []
    check =0; accuracy_past =0
    for epoch in range(1, num_epochs +1):
        #---------------------- 모델 학습 ----------------------#
        model.train()
        batch_number =0
        running_loss =0.0
        for batch_idx, samples in enumerate(train_dataloader):
            x_train, y_train = samples

            optimizer.zero_grad()
            y_hat = model.forward(x_train)
            loss = criterion(y_hat,y_train.long())
            loss.backward()
            optimizer.step()

            running_loss += loss.item()
            batch_number +=1
        loss_values.append(running_loss / batch_number)
        #-------------------------------------------------------#
```

[코드 25] 모델 학습 코드(일부)

• model.train() : training 과정에서 사용해야 하는 layer들을 알아서 사용하도록 하는 함수이다.

• train_dataloader : 학습데이터는 배치를 나누어 사용하게 되며, enumerate() 함수를 통해 batch index(batch_idx)와 해당 데이터(samples)를 동시에 사용한다. 그 결과 학습에 사용되는 데이터는 각 배치별(x_train, y_train)이다.

• optimizer.zero_grad() : 가중치 변수 초기화 함수로, pytorch는 backward 과정에서, gradients값들을 계속 더해주기 때문에 back propagation을 하기 전에 gradients를 zero로 만드는 변수 초기화를 해야 한다.

• y_hat = model.forward(x_train) : 학습 데이터를 AI 분석 모델에 입력하여 예측값(y_hat)을 구한다.

• loss = criterion(y_hat,y_train.long()) : AI 분석 모델의 예측값(y_hat)과 실제 라벨값(y_train) 사이의 차이인 loss 값을 계산한다.

• loss.backward() : loss의 역전파를 위한 함수, 모델의 파라미터와 관련된 loss의 gradients를 계산한다.

• optimizer.step() : 신경망의 가중치를 업데이트한다.

• loss_values.append(running_loss / batch_number) : training loss 값을 저장한다.

〈 모델 검증 〉

```
#---------------------- 모델 검증 ----------------------#
model.eval()
accuracy =0.0
total =0.0
for batch_idx, data in enumerate(valid_dataloader):
    x_valid, y_valid = data

    v_hat = model.forward(x_valid)
    v_loss = criterion(v_hat,y_valid.long())
    _, predicted = torch.max(v_hat.data, 1)
    total += y_valid.size(0)
    accuracy += (predicted == y_valid).sum().item()
loss_values_v.append(loss.item())
accuracy = (accuracy / total)
#-----------------------------------------------------#
```

[코드 26] 모델 검증 코드(일부)

• model.eval() : evaluation 과정에서 사용하지 않아야 하는 layer들을 알아서 off 시키도록 하는 함수

  ◦ 학습 과정에서 과적합을 방지하기 위해 dropout layer를 추가하는데, 검증과 평가의 과정에서 이 layer를 사용하지 않는 것이 좋다. 따라서 이러한 layer를 자동으로 배제할 수 있는 함수인 셈이다.

• valid_dataloader : 검증데이터는 배치를 나누지 않았기 때문에 전체 데이터를 사용하게 되며, 이는(x_valid, y_valid)이다.

• v_hat = model.forward(x_valid) : 검증 데이터를 AI 분석 모델에 입력하여 예측값(v_hat)을 구한다.

• v_loss = criterion(v_hat,y_valid.long()) : AI 분석 모델의 예측값(v_hat)과 실제 라벨값(y_valid) 사이의 차이인 v_loss 값을 계산한다.

• _, predicted = torch.max(v_hat.data, 1) : 모델의 검증 정확도를 구하기 위해 AI 분석 모델

의 예측값(v_hat)에서 가장 큰 확률을 가지는 클래스를 라벨로 한다.

- accuracy +=(predicted == y_valid).sum().item() : 실제 라벨값(y_valid)과 비교를 통해 accuracy를 계산한다.
- loss_values_v.append(loss.item()) : validation loss 값을 저장한다.

〈 학습 과정 시각화 〉

```
        #----------------Check for early stopping---------------#
        if epoch % 1 ==0:
            print('[Epoch {}/{}] [Train_Loss: {:.6f} /Valid_Loss: {:.6f}]'.format(epoch,
num_epochs, loss.item(),v_loss.item()))
            print('[Epoch {}/{}] [Accuracy  : {:.6f}]'.format(epoch, num_epochs,
accuracy))

        if accuracy_past > accuracy:
            check +=1
        else:
            check =0
            accuracy_past = accuracy

        if check >50:
            print('This is time to do early stopping')

    torch.save(model, PATH +'model.pt')
    return loss_values, loss_values_v
```

[코드 27] 학습 과정 시각화(일부)

- 매 epoch마다 training loss, validation loss, validation accuracy를 출력한다. 학습이 진행됨에 따라 정확도가 이전 정확도보다 향상되지 않으면 학습 중단(early stopping) 지점을 알려준다. (본 가이드북에서는 50 사용)
- torch.save(model, PATH + 'model.pt') : 학습 결과를 PATH 위치에 저장

⑦-1. 심층신경망(DNN) 모델 학습

```
DNN_model = KAMP_DNN()
num_epochs =1000
criterion = nn.CrossEntropyLoss()
optimizer = torch.optim.Adam(DNN_model.parameters())
PATH ='save/DNN/'
DNN_loss_values, DNN_loss_values_v = train_model(DNN_model, criterion, optimizer,
num_epochs, train_dataloader, PATH)
```

```
[Epoch 1/1000] [Train_Loss: 1.217961 /Valid_Loss: 1.172896]
[Epoch 1/1000] [Accuracy  : 0.438563]
[Epoch 2/1000] [Train_Loss: 0.948678 /Valid_Loss: 0.913253]
[Epoch 2/1000] [Accuracy  : 0.482950]
[Epoch 3/1000] [Train_Loss: 0.897611 /Valid_Loss: 0.866707]
[Epoch 3/1000] [Accuracy  : 0.491600]
[Epoch 4/1000] [Train_Loss: 0.866548 /Valid_Loss: 0.850032]
[Epoch 4/1000] [Accuracy  : 0.492100]
[Epoch 5/1000] [Train_Loss: 0.855916 /Valid_Loss: 0.842592]
[Epoch 5/1000] [Accuracy  : 0.500162]
[Epoch 6/1000] [Train_Loss: 0.846409 /Valid_Loss: 0.835176]
[Epoch 6/1000] [Accuracy  : 0.515587]
[Epoch 7/1000] [Train_Loss: 0.839891 /Valid_Loss: 0.825707]
[Epoch 7/1000] [Accuracy  : 0.523962]
[Epoch 8/1000] [Train_Loss: 0.835031 /Valid_Loss: 0.817237]
[Epoch 8/1000] [Accuracy  : 0.536787]
```

[코드 28] 심층신경망 모델 학습

- DNN_model = KAMP_DNN() : 학습을 위해 심층신경망 모델 정의한다.
- num_epochs = 1000 : 학습의 반복 횟수 설정한다.
- criterion = nn.CrossEntropyLoss() : AI 분석 모델 학습을 위한 손실함수로 분류(Classification)에 주로 사용되는 nn.CrossEntropyLoss 사용한다.
- optimizer = torch.optim.Adam(DNN_model.parameters()) : SGD+momentum 과 RMSprop optimizer의 기능을 모두 갖춘 Adam optimizer 사용하였으며, 이 경우 따로 Learning rate의 설정은 불필요하다.
- PATH = 'save/DNN/' : 학습이 완료된 후 'model.pt' 데이터가 저장되는 위치
- DNN_loss_values, DNN_loss_values_v = train_model(DNN_model, criterion, optimizer, num_epochs, train_dataloader, PATH) : 심층신경망 모델의 학습을 진행하고, 손실그래프를 그리기 위한 training loss와 validation loss를 변수로 저장한다.

⑦-2. 합성곱신경망(CNN) 모델 학습

```
CNN_model = KAMP_CNN()
num_epochs =1000
criterion = nn.CrossEntropyLoss()
optimizer = torch.optim.Adam(CNN_model.parameters())
PATH ='save/CNN/'
CNN_loss_values, CNN_loss_values_v = train_model(CNN_model, criterion, optimizer,
num_epochs, train_dataloader, PATH)
```

```
[Epoch 1/1000] [Train_Loss: 1.279304 /Valid_Loss: 1.330062]
[Epoch 1/1000] [Accuracy  : 0.290475]
[Epoch 2/1000] [Train_Loss: 1.208981 /Valid_Loss: 1.203288]
[Epoch 2/1000] [Accuracy  : 0.464000]
[Epoch 3/1000] [Train_Loss: 1.147153 /Valid_Loss: 1.136544]
[Epoch 3/1000] [Accuracy  : 0.473275]
[Epoch 4/1000] [Train_Loss: 1.103614 /Valid_Loss: 1.088155]
[Epoch 4/1000] [Accuracy  : 0.491675]
[Epoch 5/1000] [Train_Loss: 1.053487 /Valid_Loss: 1.046402]
[Epoch 5/1000] [Accuracy  : 0.517300]
[Epoch 6/1000] [Train_Loss: 1.013904 /Valid_Loss: 1.006425]
[Epoch 6/1000] [Accuracy  : 0.557700]
[Epoch 7/1000] [Train_Loss: 0.978695 /Valid_Loss: 0.971965]
[Epoch 7/1000] [Accuracy  : 0.585475]
[Epoch 8/1000] [Train_Loss: 0.945550 /Valid_Loss: 0.926145]
[Epoch 8/1000] [Accuracy  : 0.583187]
```

[코드 29] 합성곱신경망 모델 구축

- 심층신경망과 변수명만 다른 동일 프로세스로 진행되므로 설명은 생략한다.

⑦-3. 순환신경망(RNN) 모델 학습

```
RNN_model = KAMP_RNN()
num_epochs =1000
criterion = nn.CrossEntropyLoss()
optimizer = torch.optim.Adam(RNN_model.parameters())
PATH ='save/RNN/'
RNN_loss_values, RNN_loss_values_v = train_model(RNN_model, criterion, optimizer,
num_epochs, train_dataloader, PATH)
```

```
[Epoch 1/1000] [Train_Loss: 1.386275 /Valid_Loss: 1.386278]
[Epoch 1/1000] [Accuracy  : 0.250000]
[Epoch 2/1000] [Train_Loss: 1.386158 /Valid_Loss: 1.386193]
[Epoch 2/1000] [Accuracy  : 0.224625]
[Epoch 3/1000] [Train_Loss: 1.384648 /Valid_Loss: 1.384242]
[Epoch 3/1000] [Accuracy  : 0.323912]
[Epoch 4/1000] [Train_Loss: 1.245015 /Valid_Loss: 1.231398]
[Epoch 4/1000] [Accuracy  : 0.418650]
[Epoch 5/1000] [Train_Loss: 1.094305 /Valid_Loss: 1.089046]
[Epoch 5/1000] [Accuracy  : 0.429775]
[Epoch 6/1000] [Train_Loss: 1.096556 /Valid_Loss: 1.086774]
[Epoch 6/1000] [Accuracy  : 0.442050]
[Epoch 7/1000] [Train_Loss: 1.084525 /Valid_Loss: 1.083589]
[Epoch 7/1000] [Accuracy  : 0.441012]
[Epoch 8/1000] [Train_Loss: 1.087142 /Valid_Loss: 1.081896]
[Epoch 8/1000] [Accuracy  : 0.451763]
```

[코드 30] 순환신경망 모델 구축

- 심층신경망과 변수명만 다른 동일 프로세스로 진행되므로 설명은 생략한다.

[단계 ⑧] AI 모델 평가

⑧-0. 모델 평가 코드(공통)

학습이 완료된 AI 모델의 평가를 위한 공통 code는 다음과 같다.

- 모델 평가

```
def test_model(model, PATH):
    model = torch.load(PATH +'model.pt')
    #---------------------- 모델 시험 ---------------------#
    model.eval()
    total =0.0
    accuracy =0.0
    for batch_idx, data in enumerate(test_dataloader):
        x_test, y_test = data

        t_hat = model(x_test)
        _, predicted = torch.max(t_hat.data, 1)
        total += y_test.size(0)
        accuracy += (predicted == y_test).sum().item()
    accuracy = (accuracy / total)
    #------------------------------------------------------#
    print(accuracy)
```

[코드 31] 모델 평가 코드

- 입력 : 평가를 진행할 AI 분석 모델, 모델 학습 결과가 저장된 위치(PATH)를 입력한다.
- 진행 : test data를 이용하여 앞선 검증 코드와 동일한 프로세스로 진행된다.
- 출력 : test data를 이용한 모델 정확도 결과를 출력한다.

- 혼동 행렬

```
from sklearn.metrics import classification_report, confusion_matrix
def draw_confusion_matrix(model, xt, yt, PATH):
    y_pred = []; y_true = []

    model.eval()
    y_hat = model(xt); output = (torch.max(torch.exp(y_hat), 1)[1]).data.cpu().numpy()
    y_pred.extend(output)
    labels = y_test.data.cpu().numpy()
    y_true.extend(labels)
    # 분류 항목
    classes = ('Normal', 'Type1', 'Type2', 'Type3')
    # Confussion Matrix 생성
    plt.figure(figsize = (7,5))
    dlen = float(len(x_test)) # test data 크기 : 여기서는 80000
    cm = confusion_matrix(y_true, y_pred)
    df_cm = pd.DataFrame(cm/dlen, index = [i for i in classes],columns = [i for i in classes])
    sns.heatmap(df_cm, annot=True, cmap=plt.cm.Blues)
    plt.title("Confusion Matrix", size=24, fontweight='bold')
    plt.xlabel("Predicted Label", size=16); plt.ylabel("Actual Label", size=16)
    plt.rc('xtick', labelsize=12); plt.rc('ytick', labelsize=12); plt.yticks(rotation=0)
    plt.savefig(PATH +'cm_output.png')
```

[코드 32] 혼동 행렬 plot 코드

- 입력 : 평가를 진행할 AI 분석 모델, 평가에 사용할 test data(x_test, y_test), 모델 학습 결과가 저장된 위치(PATH)를 입력한다.
- 진행 : test data를 이용하여 AI 분석 모델이 예측한 결과와 실제 라벨값이 얼마나 일치하는지 정확도를 계산한다.
- 출력 : test data를 이용한 모델 정확도 혼동 행렬 결과를 시각화하고 이를 저장 위치(PATH)에 cm_output.png로 저장한다.

- 손실 그래프

```
def plot_loss_graph(loss_values, loss_values_v):
    plt.figure()
    plt.plot(loss_values)
    plt.plot(loss_values_v)
    plt.title("Training & Validation Loss")
    plt.ylabel("loss", fontsize="large")
    plt.xlabel("epoch", fontsize="large")
    plt.legend(["train", "validation"])
    plt.tight_layout()
    # 결과 저장
    plt.savefig(PATH +'lossplot_output.png')
```

[코드 33] 손실 그래프 plot 코드

- 입력 : 학습 및 검증 loss의 history(loss_valus, loss_values_v)를 입력한다.
- 출력 : 학습 과정을 loss 값을 통해 시각화(plt.plot)하고, 이를 저장 위치(PATH)에 cm_output.png로 저장한다.

⑧-1. 심층신경망(DNN) 모델 평가

- 모델 평가

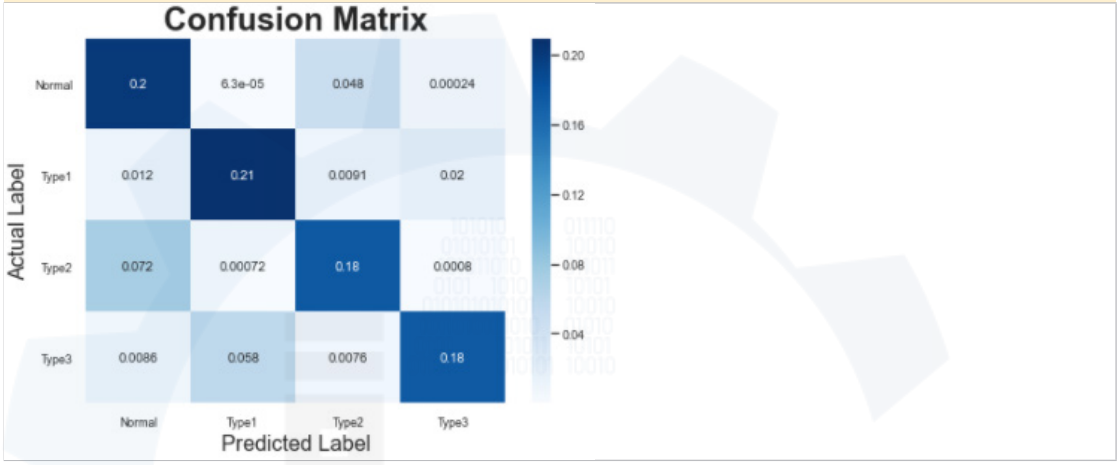
```
PATH ='save/DNN/'
test_model(DNN_model, PATH)
```

```
0.7627625
```

[코드 34] 심층신경망 모델 평가 결과

- 혼동 행렬

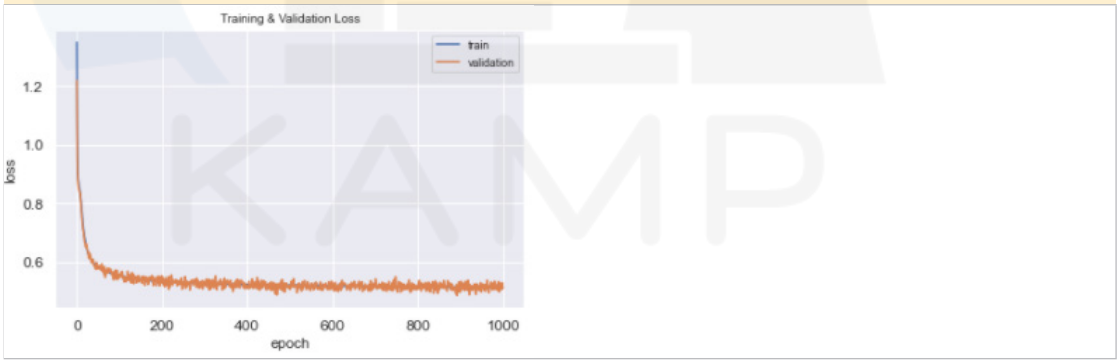
```
draw_confusion_matrix(DNN_model, x_test, y_test, PATH)
```



[코드 35] 심층신경망 모델 혼동 행렬

- 손실 그래프

```
plot_loss_graph(DNN_loss_values, DNN_loss_values_v)
```



[코드 36] 심층신경망 모델 손실 그래프

⑧-2. 합성곱신경망(CNN) 모델 평가

- 모델 평가

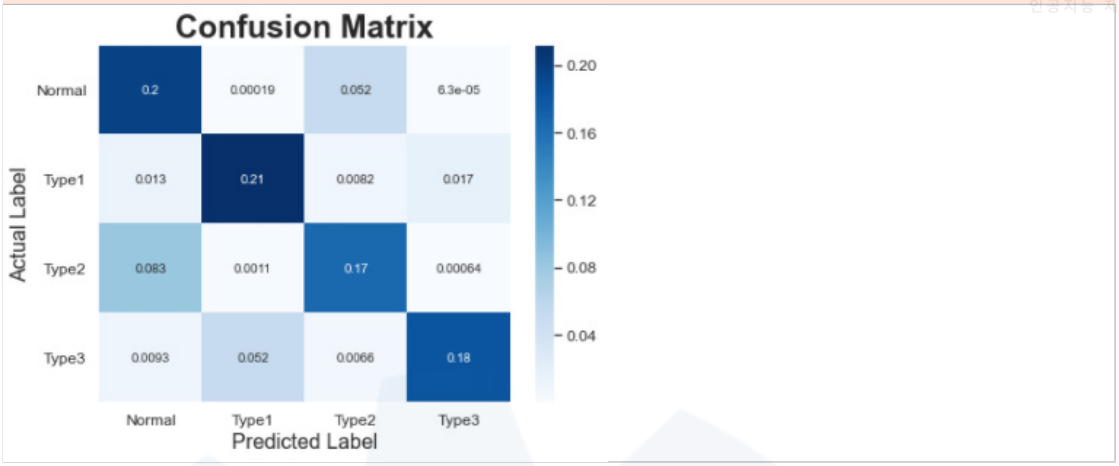
```
PATH ='save/CNN/'
test_model(CNN_model, PATH)
```

```
0.7567875
```

[코드 37] 합성곱신경망 모델 평가 결과

- 혼동 행렬

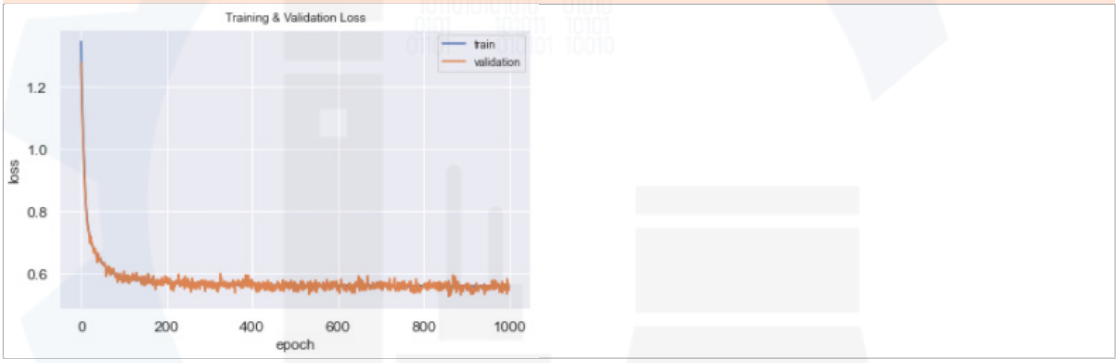
```
draw_confusion_matrix(CNN_model, x_test, y_test, PATH)
```



[코드 38] 합성곱신경망 모델 혼동 행렬

- 손실 그래프

```
plot_loss_graph(CNN_loss_values, CNN_loss_values_v)
```



[코드 39] 합성곱신경망 모델 손실 그래프

⑧-3. 순환신경망(RNN) 모델 평가

- 모델 평가

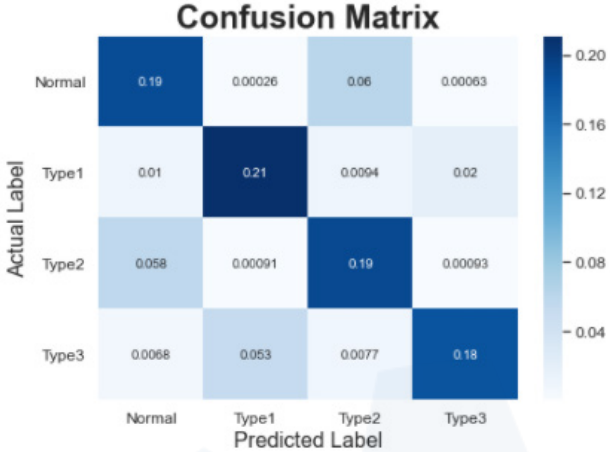
```
PATH ='save/RNN/'
test_model(RNN_model, PATH)
```

```
0.7718625
```

[코드 40] 순환신경망 모델 구축

- 혼동 행렬

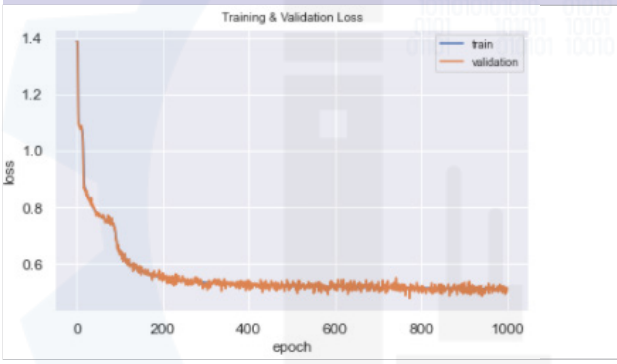
```
draw_confusion_matrix(RNN_model, x_test, y_test, PATH)
```



[코드 41] 순환신경망 모델 구축

- 손실 그래프

```
plot_loss_graph(RNN_loss_values, RNN_loss_values_v)
```



[코드 42] 순환신경망 모델 구축

⑧-4. 결과 분석 및 최종 모델 선정

- 3가지 AI 분석 모델의 성능([표 17] 참고)

DNN	CNN	RNN
0.7628	0.7568	0.7719

[표 17] AI 분석 모델의 성능

- 3가지 모델 모두 loss plot을 통해 학습 과정을 분석하면, epoch = 200까지는 loss 값의 변화가 매우 빠른 것을 확인할 수 있다. 즉 epoch = 400부터는 성능 개선이 거의 없으며, 계속 학습을 진행할 경우, 학습 효율이 떨어질 뿐만 아니라 과적합의 위험이 있으므로 적절한 위치에서 학습을 중단하는 것이 좋다. (본 가이드북에서는 epoch = 200~400)

- 결과적으로 본 회전기계 고장유형 AI 데이터셋에 대해서는 3개의 모델 중 순환신경망(RNN)이 가장 우수한 성능을 보임을 확인하였다. 그러나 3가지 모델 모두 비슷한 수준의 성능을 보이며, 추후 parameter study를 통해 모델을 좀 더 최적화할 수 있다.

3. 유사 타 현장의 「회전기계 고장유형 AI 데이터셋」 분석 적용

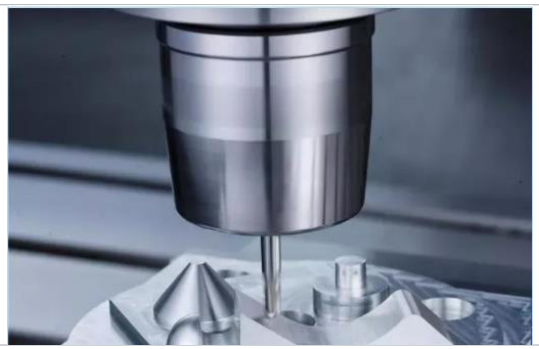
3.1. 본 분석이 적용 가능한 제조현장 소개

- 터빈, 펌프, 전동기, 압축기, 송풍기 등 회전기계 시스템이 작동하는 대부분의 산업 현장(발전소, 제조업 분야 등)에 적용할 수 있다. [그림 33]은 회전기계와 이를 사용하는 산업 현장의 사례이다.



[그림 33] GS EPS 복합화력 발전 설비(증기, 가스 터빈)

- CNC, Milling, 선반 등 재료나 공구(tool)가 회전 운동하는 시스템에서도 적용하여, 공구 파단 등 가공물의 상태에 영향을 미칠 수 있는 기계의 고장을 검출하는 데에 사용할 수 있다. [그림 34]는 회전하는 공구를 사용하는 기계 시스템의 사례이다.

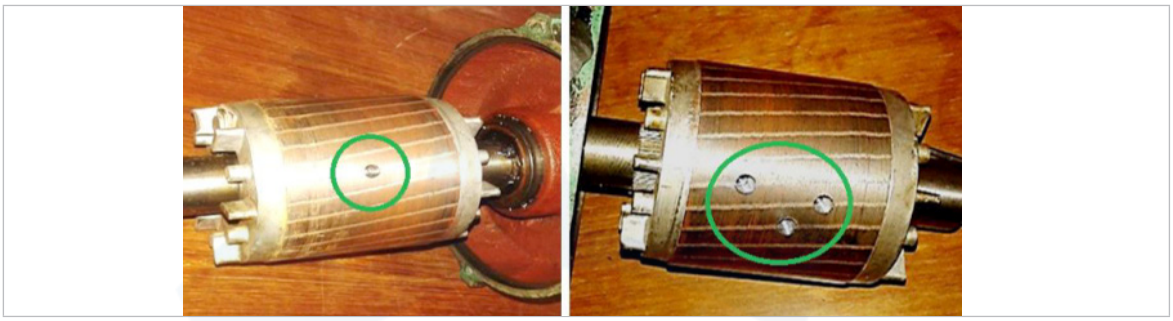


[그림 34] ALLES CNC 머신 센터

- 고장유형별 실제 사례 소개

◦ 질량 불균형 : 회전기계에서 불평형 진동이 발생하는 원인은 1) 설계상의 오류로 인한 설계 비대칭, 2) 잘못된 가공으로 인한 비대칭 3) rotor에 부착된 오염물 혹은 마모에 의한 불평형이 있다. (본 가이드북은 볼트를 이용하여 편심을 구현)

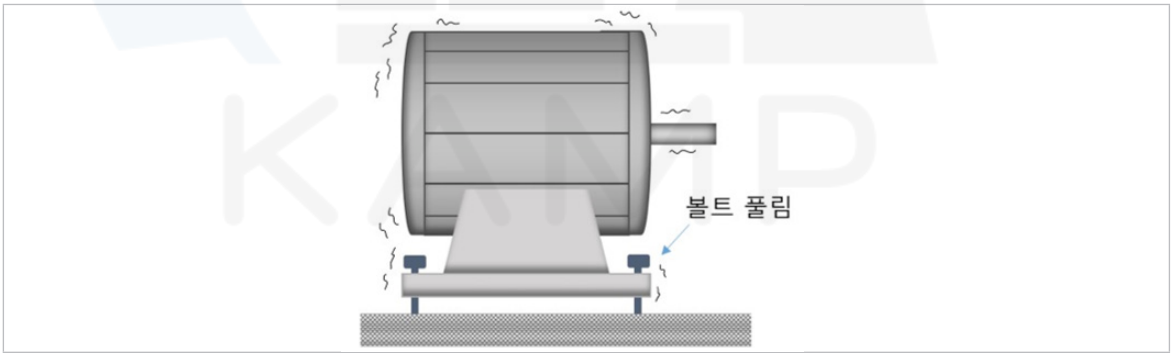
◦ 이러한 회전 불균형의 실제 사례는 다음과 같다. ([그림 35] 참고)



[그림 35] 파손으로 인해 rotor에 발생한 질량 불균형

* Sharma, A. “Analysis of broken rotor bar fault diagnosis for induction motor,” IEEE, pp. 1–5(2017)."

◦ 지지 불량(기계적 이완 Mechanical Looseness) : 회전기계를 지면 혹은 평판 등에 견고하게 지지하지 않은 경우(기계의 다리의 구조적인 이완), 기계의 수평이 틀어지거나 장비의 기울어짐 등이 확인될 수 있다. 외부의 충격이나 지진 등의 이벤트가 발생하여 기계장비의 수평이 틀어진 경우, 해당 기계시스템 운영에 영향을 줄 수 있으며, 제조업의 경우 가동 중인 기계장비에서 생산되는 제품들이 불량으로 이어질 수 있다. ([그림 36] 참고)



[그림 36] 볼트 불림으로 인해 발생한 지지 불량

3.2. 본 「회전기계 고장유형 AI 데이터셋」 분석을 원용하여 타 제조 현장 적용 시 주요 고려사항

- 정상/비정상 상태에 대한 Labeling이 필요하며, Labeling 작업은 정상/비정상 상태를 판단할 수 있는 현업 전문가의 도움을 받는 것을 추천한다.
- 진동데이터의 전처리 시 본 가이드북에서는 선형보간(resampling), 이동평균필터(filtering), Min-max 정규화 방식(normalization)을 사용하였는데, 산업 현장과 진동데이터마다 최적의 전처리 방식이 존재하므로, sampling rate나 이동평균 값(M) 등 parameter의 조정과 더불어 전처리 방법에서도 적용 현장 및 취득 데이터에 대한 고찰이 요구된다.

3 부록 _ 분석환경 구축을 위한 설치 가이드

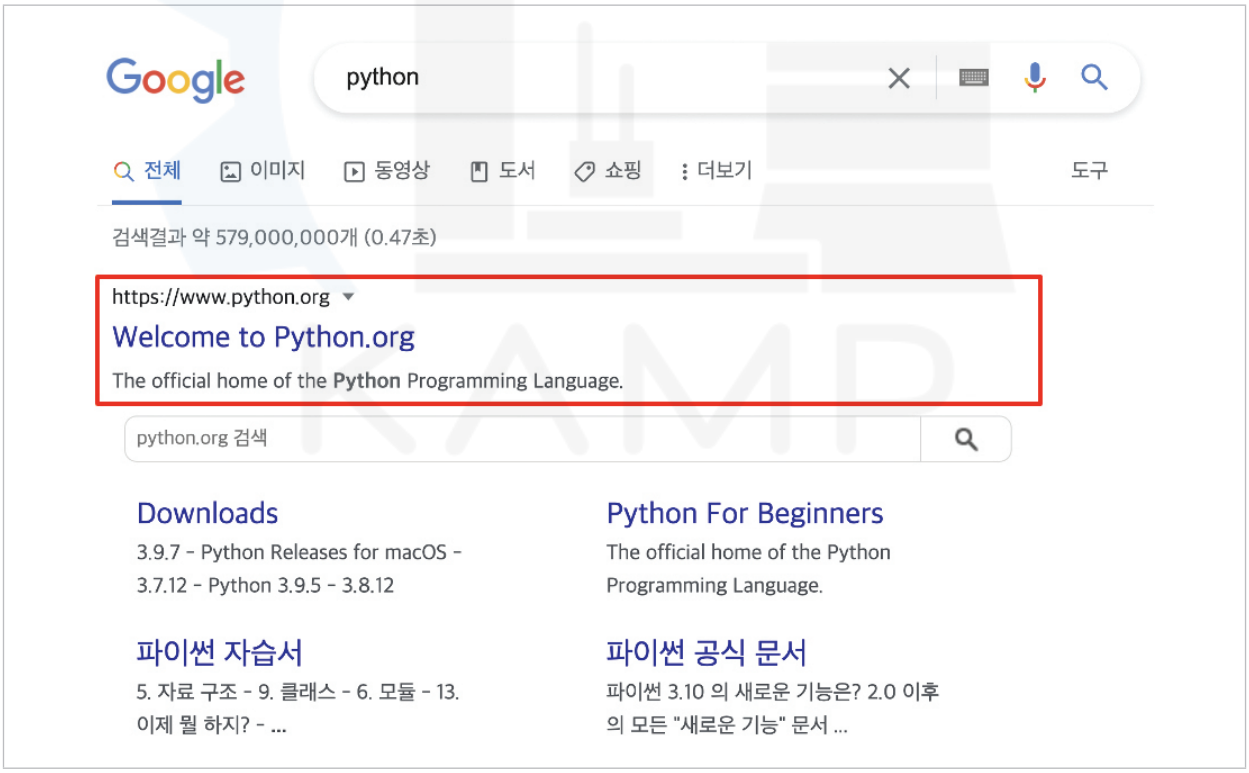
1. 파이썬(Python) 설치

Python은 컴퓨터 프로그래밍 언어로서, 데이터 분석 및 AI 분석모델 개발 등에 널리 사용되는 도구이다. 다음은 다운로드 및 설치가 간편하고 활용도가 높은 python을 로컬 컴퓨터에 설치하는 방법을 안내한다.

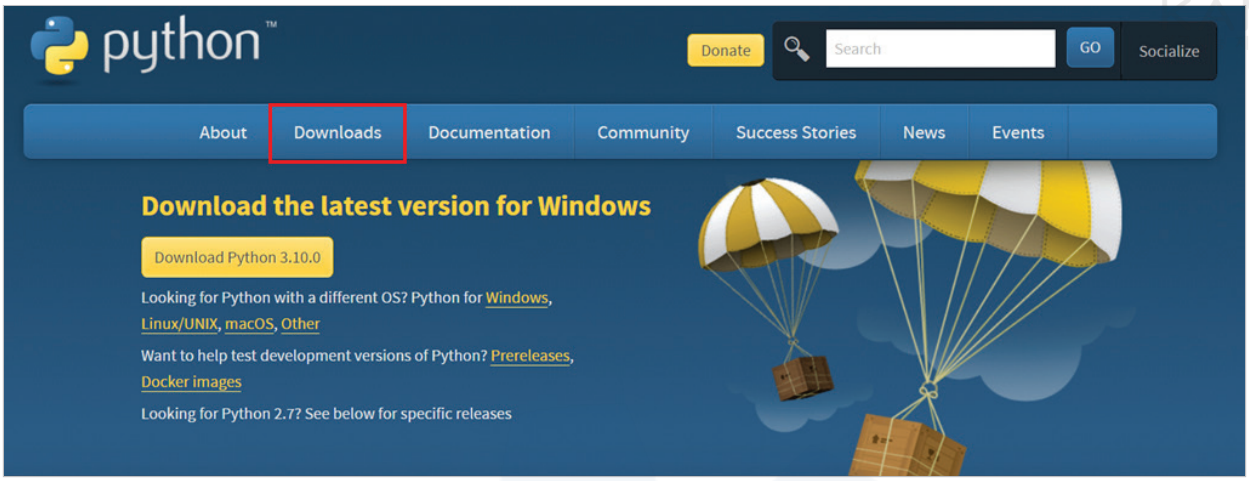
① google.com 등의 검색 엔진에 'python'을 검색한다.



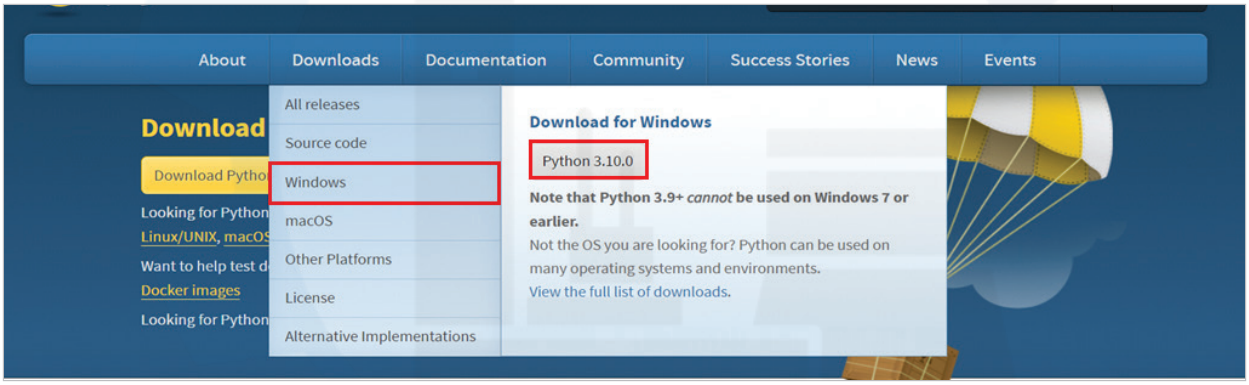
② 제일 처음에 보이는 'Welcome to Python.org'를 클릭한다.



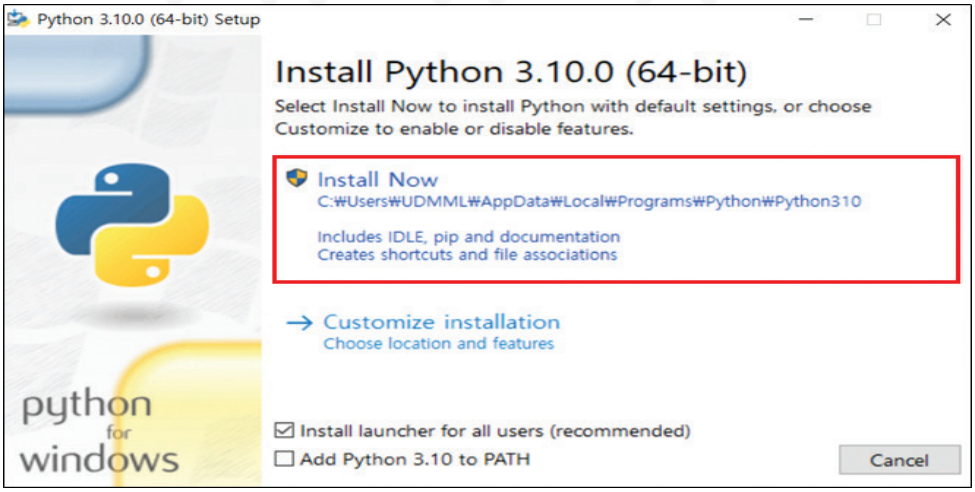
③ 클릭후 보이는 페이지의 정면에서 상단 좌측 2번째 'Downloads'를 클릭한다.



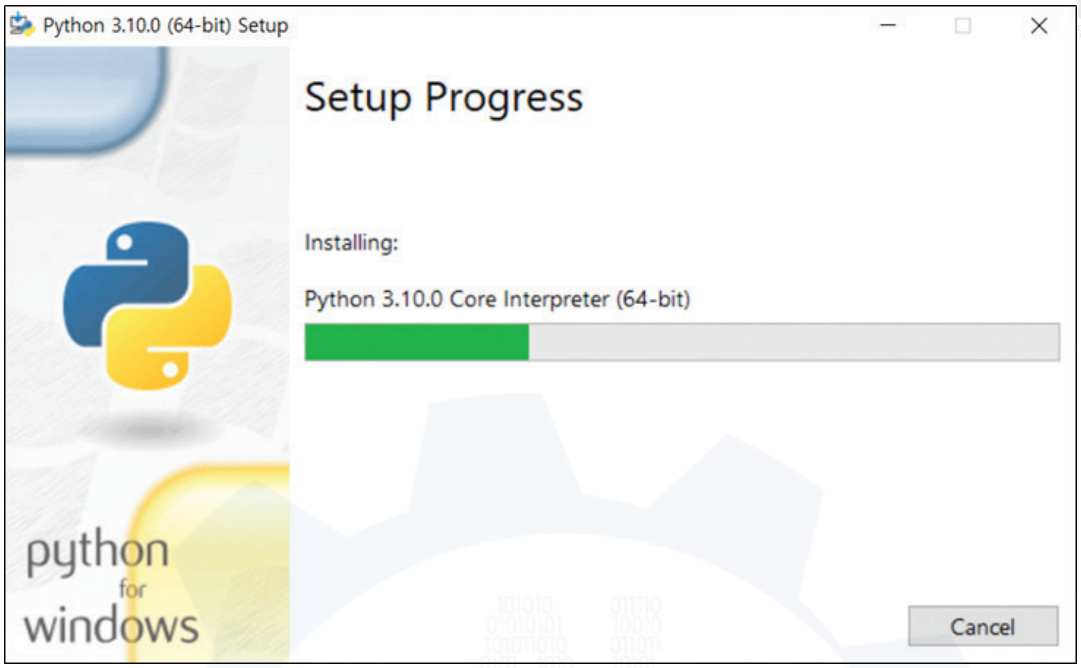
④ 컴퓨터의 운영체제에 따라 상단에서 3번째(MacOS의 경우 4번째) 탭을 선택한 후, Python 3.10.0을 다운로드한다. (Python 버전은 업데이트 될 수 있으며, 본 가이드북에서는 python 3.7.3을 이용하였지만, 이후 가상 환경에서 설정하도록 하고 최신 버전으로 다운로드해도 무방하다.)



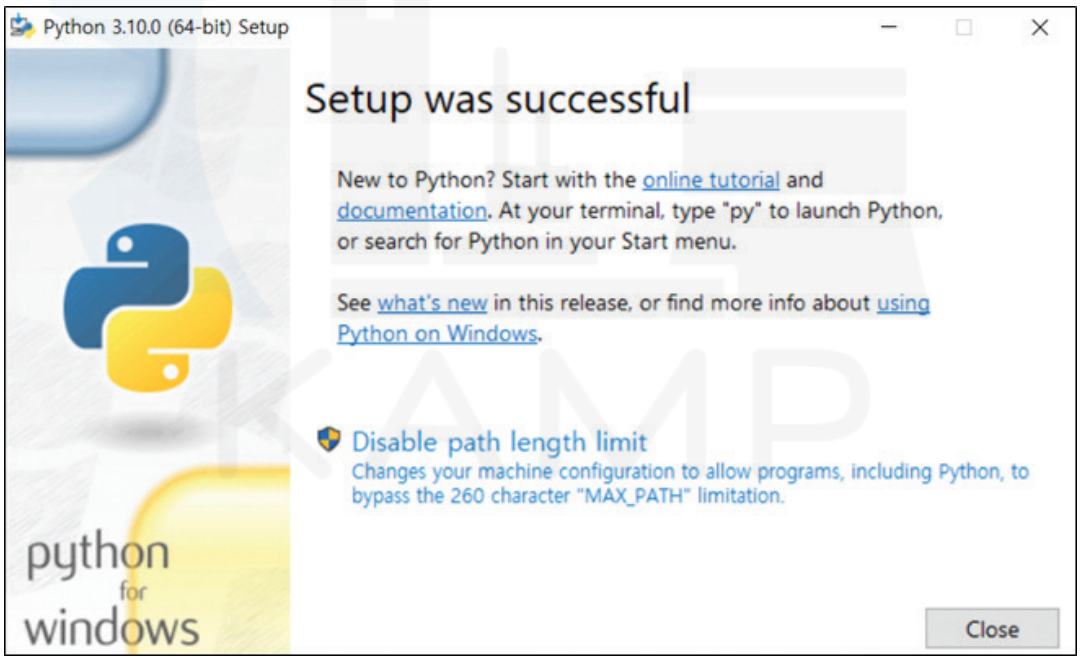
⑤ 아래와 같은 설치창에서, 'Install Now'를 클릭한다.



⑥ 아래와 같은 설치 진행창이 완료가 될 때까지 유지한다.



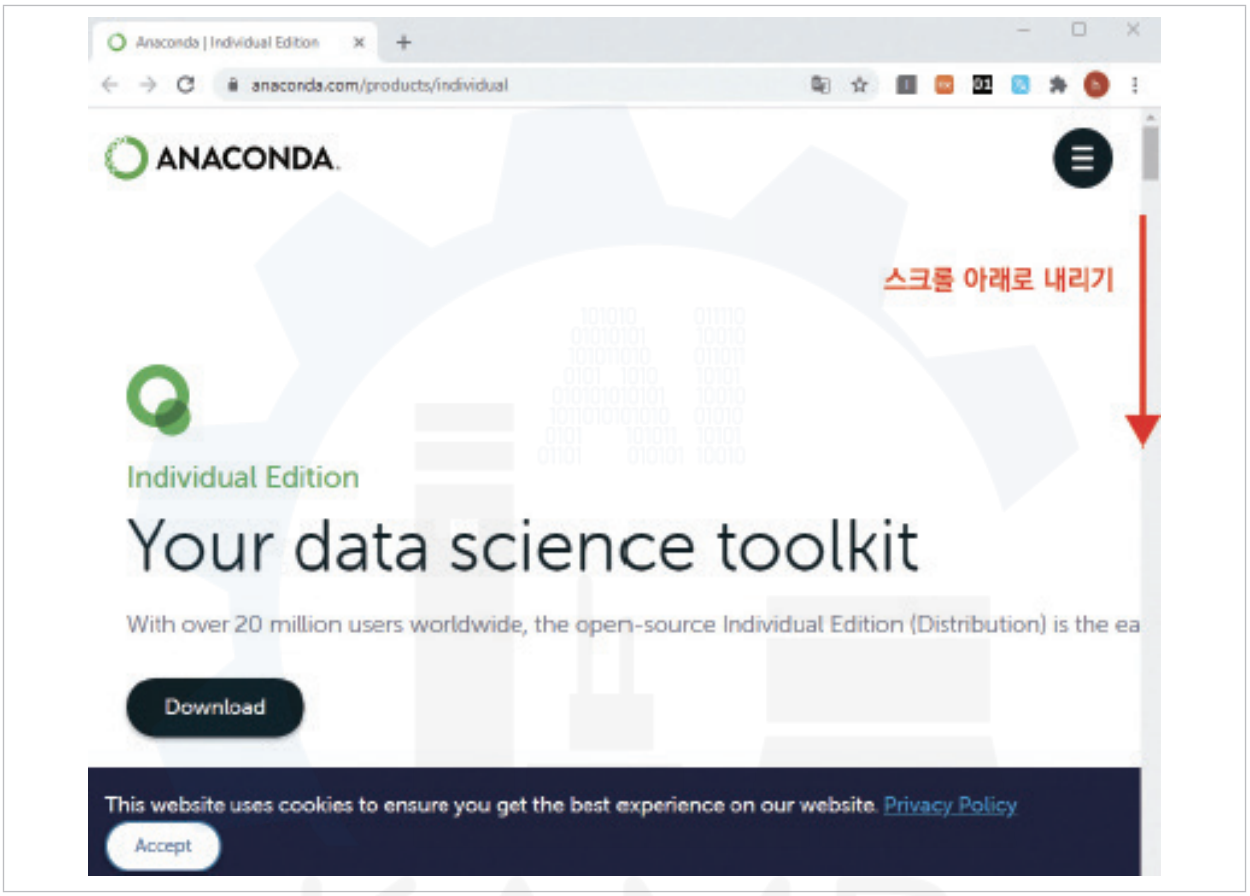
⑦ 아래와 같은 창이 나타나면 설치 완료되었으므로, 확인 후 종료한다.



2. Anaconda 설치

Anaconda란 python 등의 데이터 분석 도구를 사용할 때 필요한 고급 기능 및 분석을 보조하는 도구이다. anaconda를 설치함으로써 다양한 기능들을 효율적으로 사용할 수 있으며, 결과물을 쉽게 확인할 수 있다. 다음은 anaconda를 설치하고, 데이터를 분석할 수 있는 환경을 구축하는 방법에 대해 안내한다.

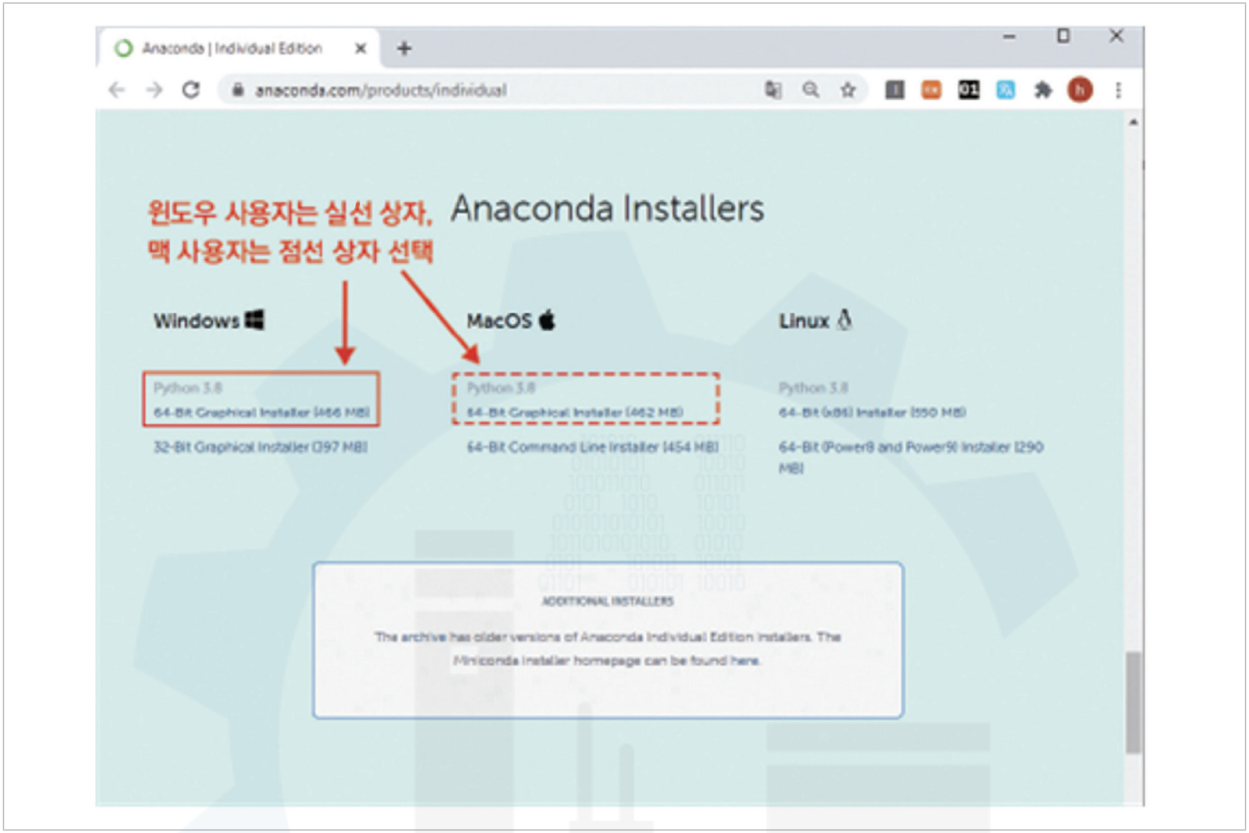
① https://www.anaconda.com/distribution/ 로 접속한다.



② 스크롤을 내려 다음의 화면을 확인한 후, 로컬 컴퓨터 사용 환경에 맞는 anaconda 설치 파일을 다운로드한다.

▶ Windows: 64-Bit Graphical Installer

▶ MacOS: 64-Bit Graphical Installer



③ 다운로드한 anaconda 설치 파일에서 마우스 오른쪽 버튼을 눌러, 방패 모양의 '관리자 권한으로 실행'을 클릭한다.

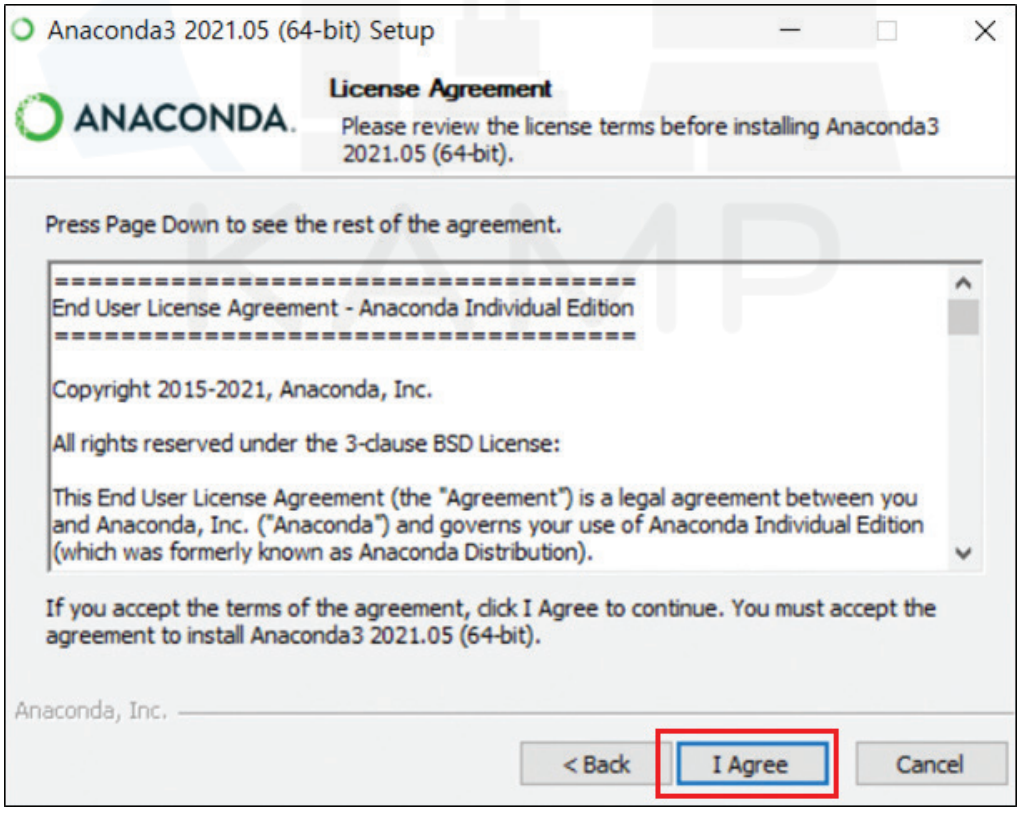
▶ (예) '다운로드' 폴더로 anaconda를 다운로드한 경우



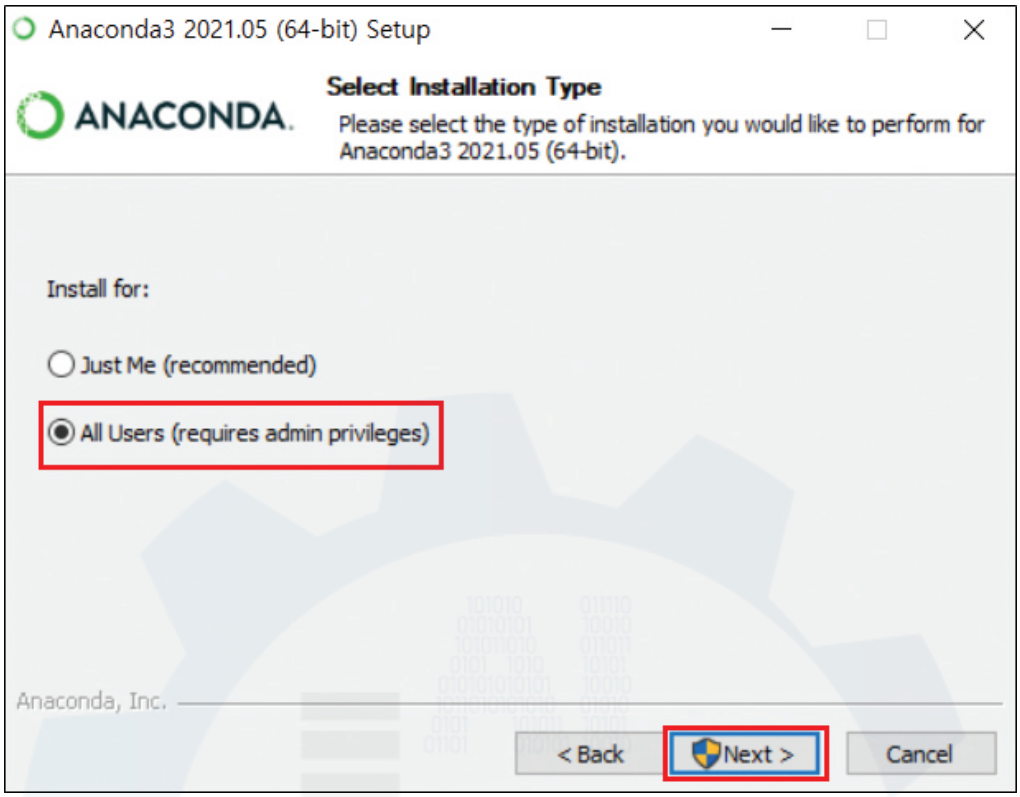
④ 설치 파일을 실행한 다음 'Next' 버튼을 클릭하여 다음 단계로 진행한다.



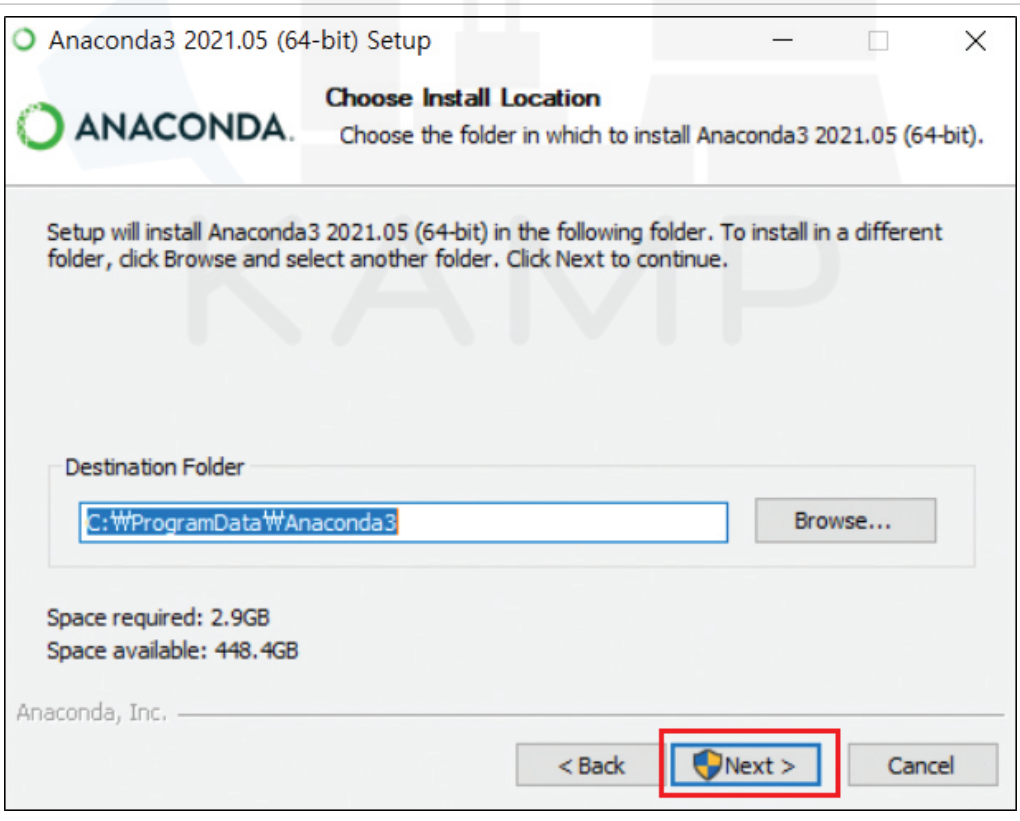
⑤ 다음 단계에서 아래와 같은 창이 나타나면, 'I Agree'를 선택한다.



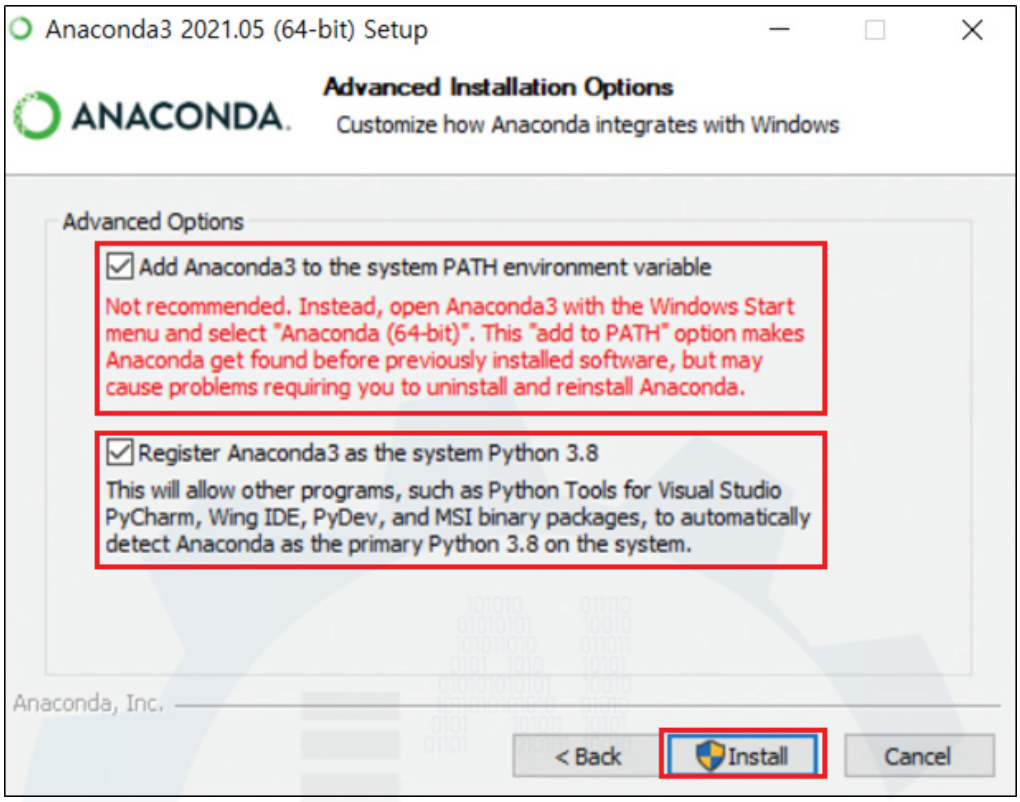
⑥ 아래와 같은 창이 나타나면, 'All Users'를 선택하여 다음 단계로 진행한다.



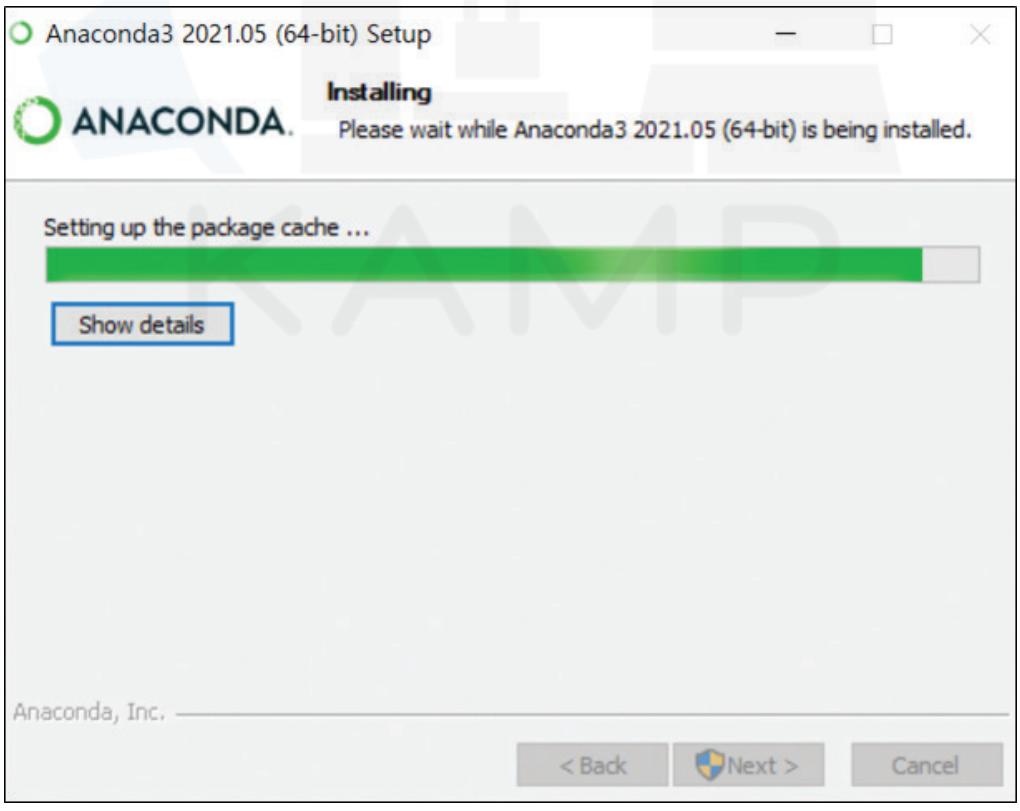
⑦ 다운로드할 위치를 설정한 뒤, 'Next'를 선택하여 다음 단계로 진행한다.



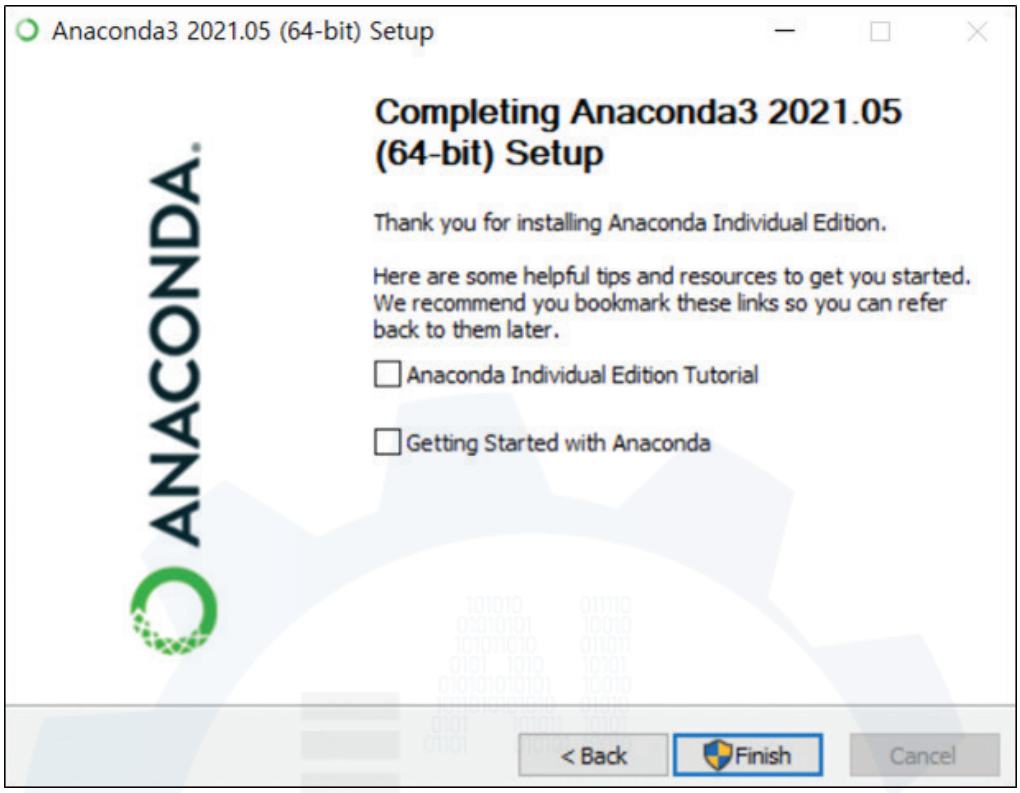
⑧ Advanced options에 대해 아래와 같이 모두 선택 후, 'Install'를 클릭한다.



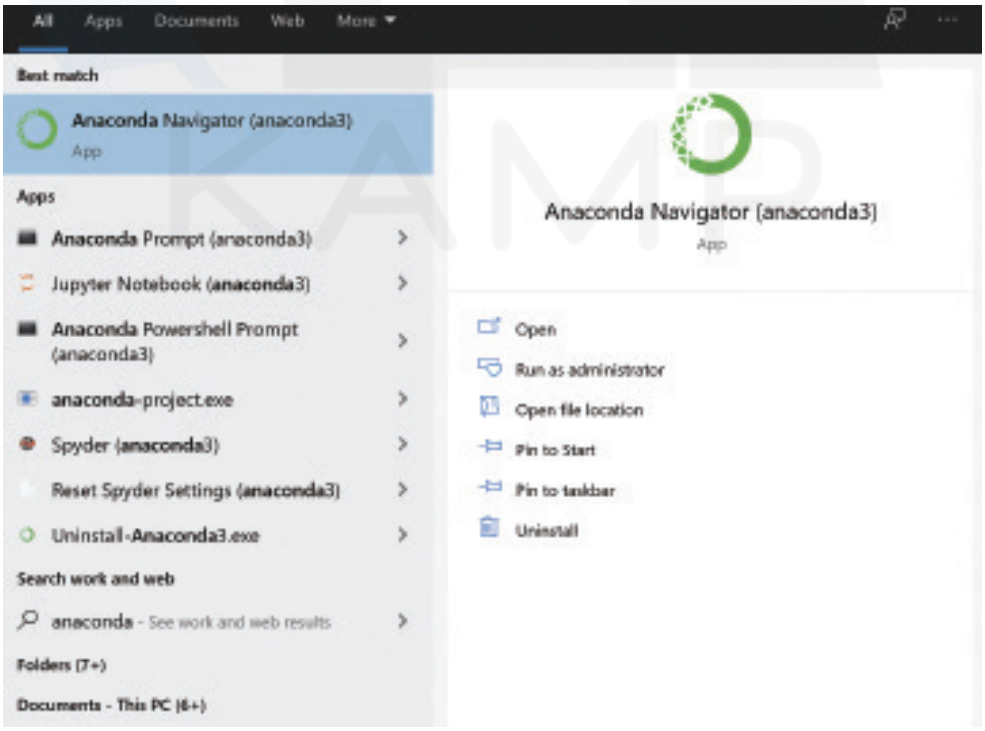
⑨ 다음과 같은 설치창으로 넘어가면, 완료될 때까지 대기한다. (5분이상 소요)



⑩ 마지막 화면에서, 모두 체크 해제한 후, 'Finish'를 눌러 설치를 완료한다.



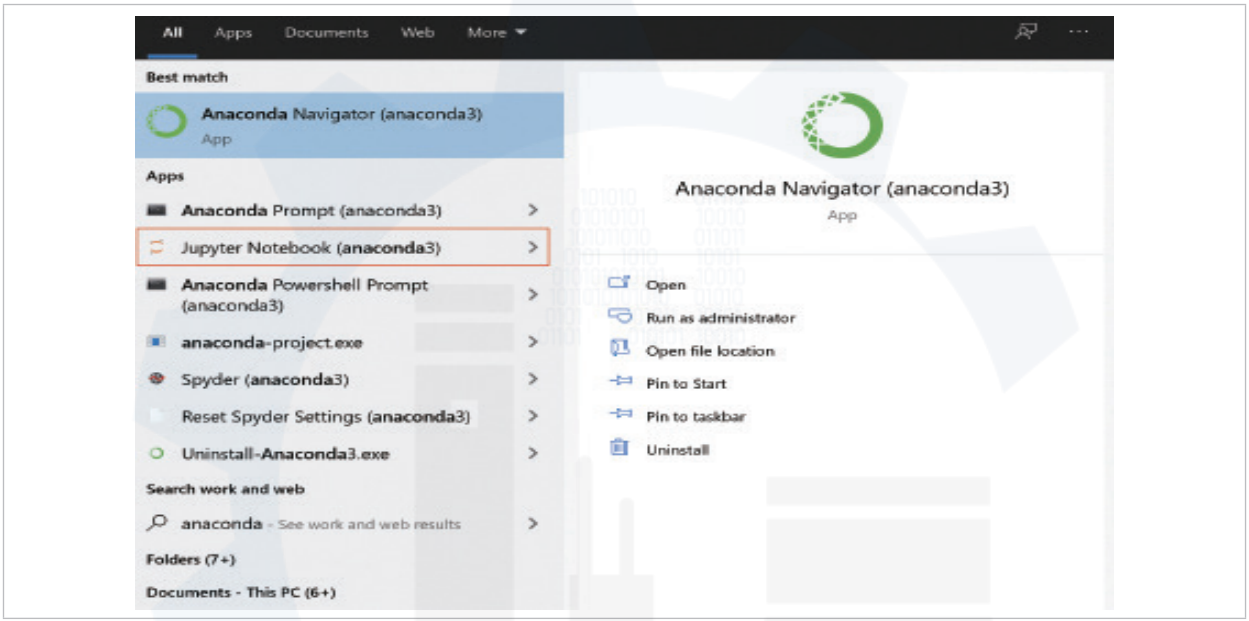
⑪ PC의 '시작 버튼( )'을 눌러서 화면과 같이 anaconda prompt가 설치되었는지 확인한다. Anaconda Navigator, Anaconda Prompt, Jupyter Notebook 등의 다른 응용 프로그램들도 함께 확인된다면 설치 완료되었다.



3. Jupyter Notebook 실행

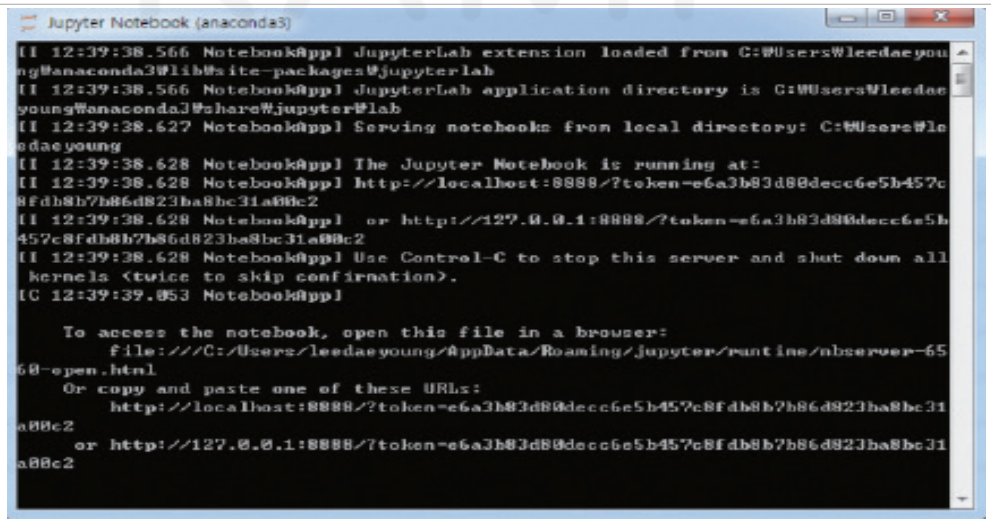
Jupyter notebook은 사용자가 코딩(분석 코드 작성)을 할 수 있는 도구이다. 쉽게 비교하자면, 문서 도구로 마이크로소프트 사의 'Word' 프로그램이나, 한컴소프트 사의 '한글' 프로그램 등과 같은 도구라고 생각할 수 있다. 데이터 분석에 다양한 입력, 실행 도구가 있지만, 본 가이드북에서는 jupyter notebook을 활용하는 방법을 안내한다.

① PC의 '시작 버튼( )'을 눌러서 화면과 같이 'anaconda'를 검색한 후 폴더 리스트 중 'Jupyter Notebook (anaconda3)'을 실행한다.

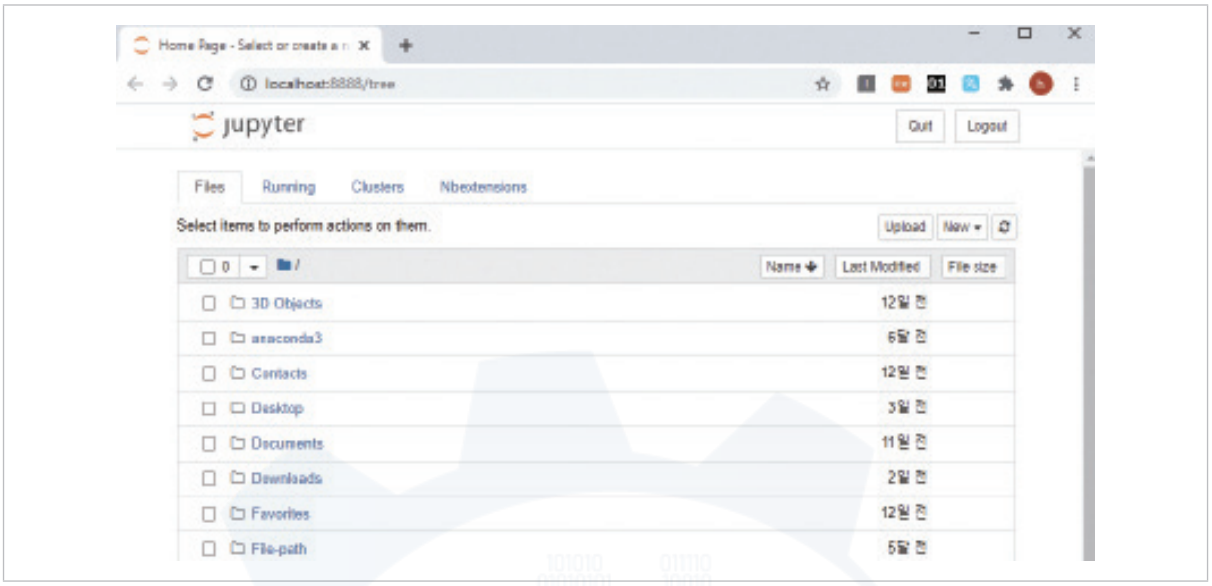


② Jupyter Notebook을 실행하면, 아래와 같이 2개의 윈도우가 실행된다.

(1) 아래의 윈도우는 jupyter notebook이 실행되는 환경 상태를 나타내주는 상태 표시 창이다. jupyter notebook을 사용하는 동안에는 종료하지 않도록 한다.



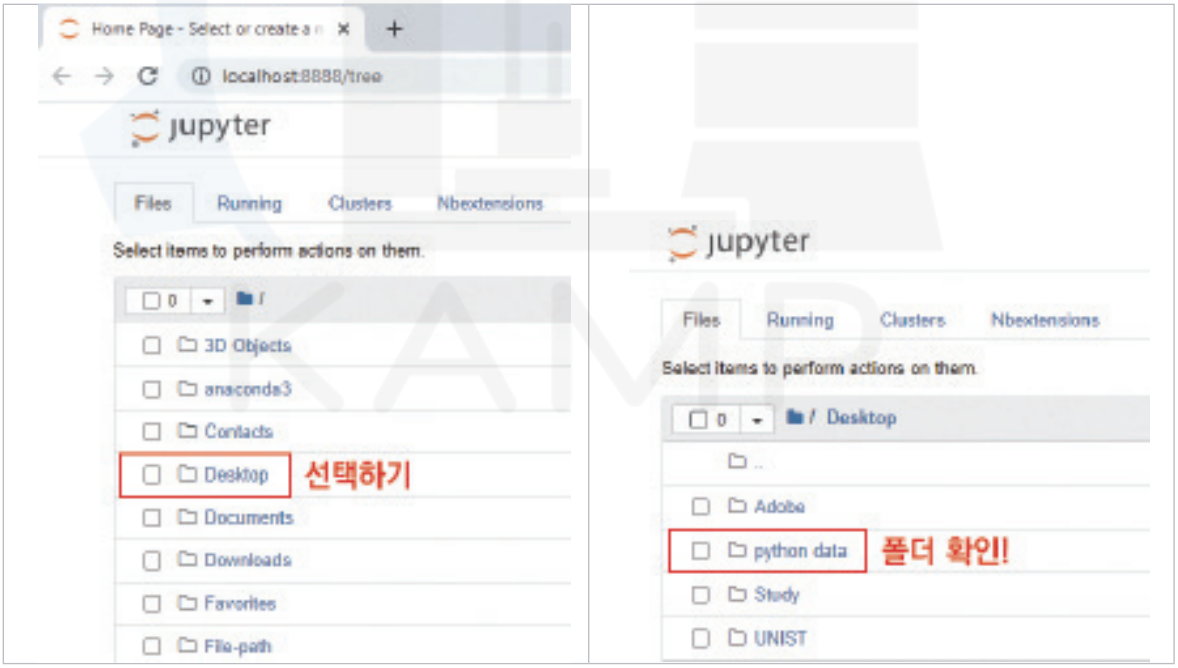
(2) 기본 브라우저(크롬, 인터넷 익스플로러)에서 jupyter notebook이 열린다.
(다른 확장 프로그램 사용하고 싶다면 기본 브라우저를 변경해주어야 한다.)



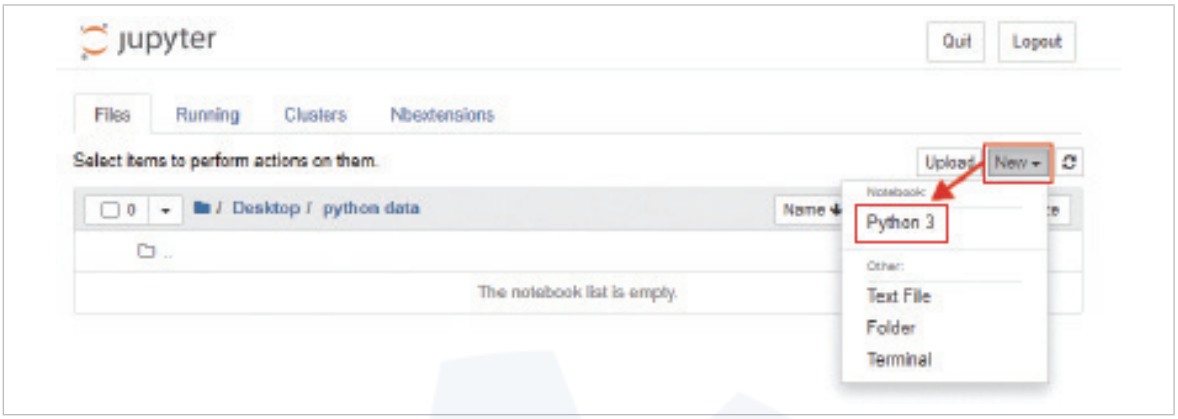
③ 데이터 분석을 진행할 폴더를 생성한다.

▶ (예) '바탕화면'에 'python data' 폴더를 생성한 다음, python code를 실행하고 저장한다.

(1) jupyter notebook에서 'python data' 폴더를 확인한다.

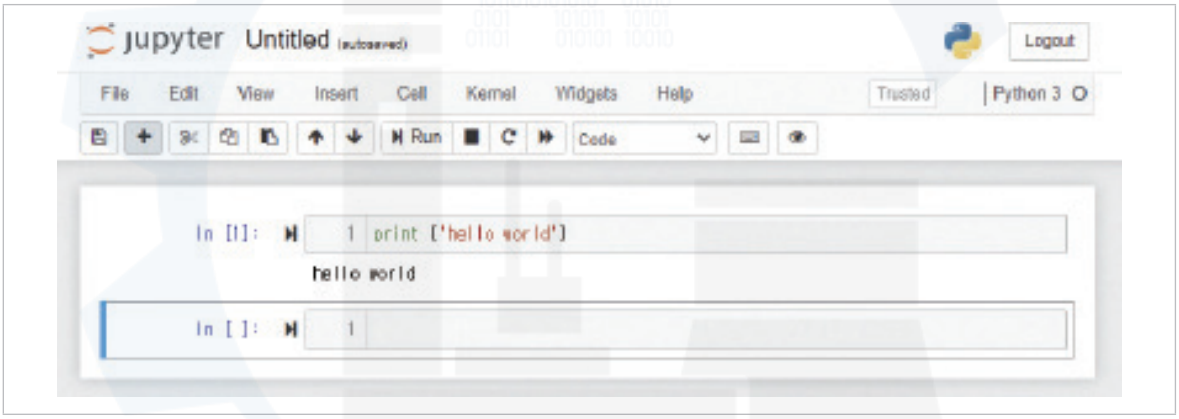


(2) python data에서 python 파일을 생성한다. 오른쪽 상단의 'New'를 누른 후, 'Python 3'을 선택한다.



(3) 'hello world'의 출력을 확인해본다.

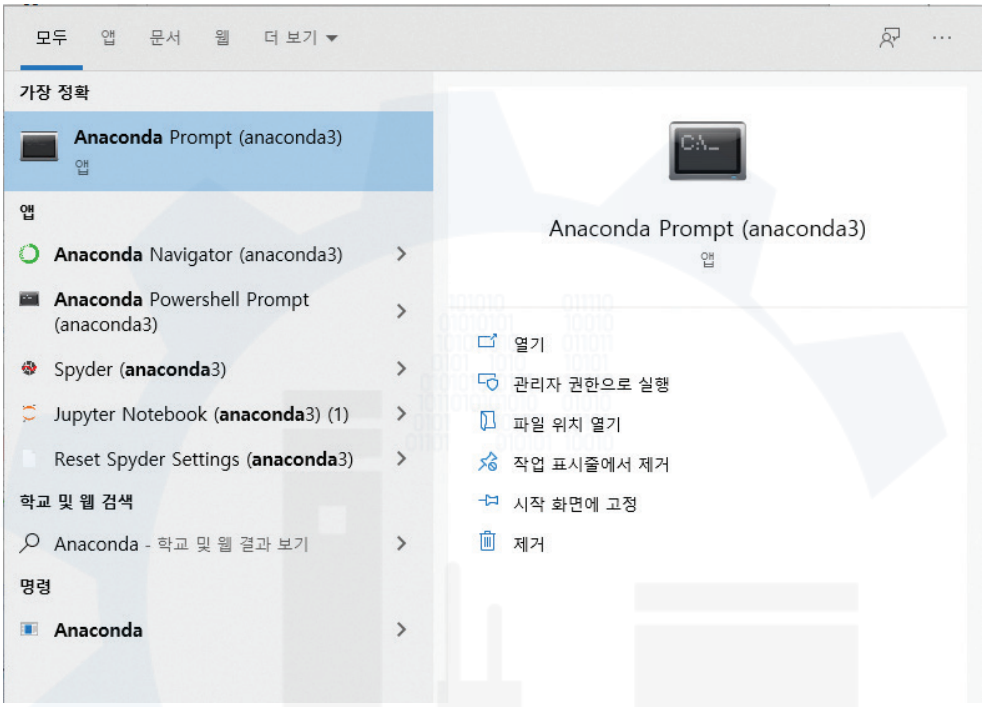
보이는 In [ ] 우측 회색 창에 print('hello world') 입력한 다음, 상단의 Run 버튼 혹은 shift + Enter 키를 눌러서 실행한다. 코드 파일의 확장자는 '*.ipynb'으로 저장된다.



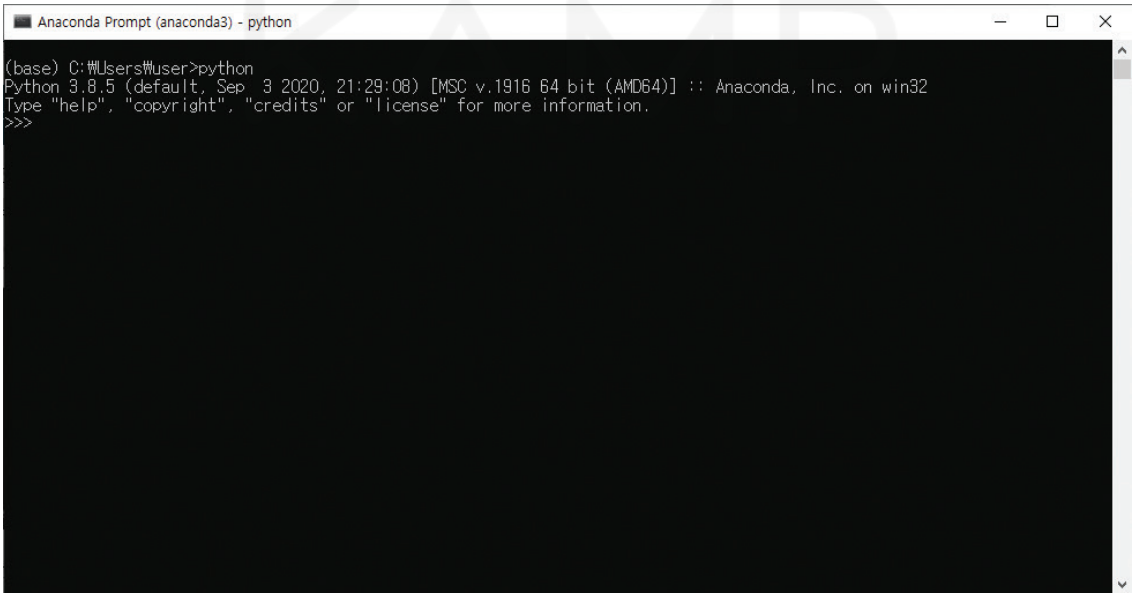
4. anaconda 가상환경 설정

가상환경 : 독립적인 작업환경에서 패키지 및 버전 관리를 하기 위한 가상의 환경을 의미한다. 각 패키지(모듈, 라이브러리)는 버전에 따라 의존성을 갖고 있으므로 소스코드에서 권장한 버전이 아니라면 오류가 발생할 수 있으며, 이를 관리하기 위해 가상환경을 사용하여 가상환경마다 다른 패키지 버전을 사용할 수 있다.

① 'Anaconda Prompt' 실행

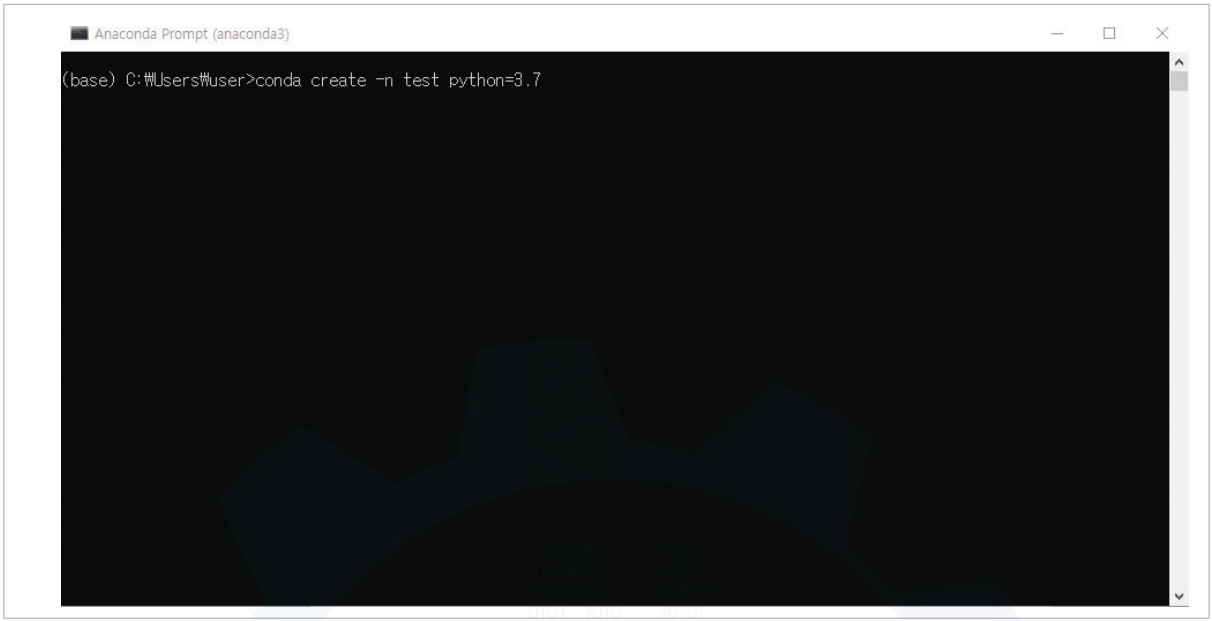


* 'Anaconda Prompt'는 실습을 진행하는 동안 종료시키면 안 되며, 켜둔 상태로 진행

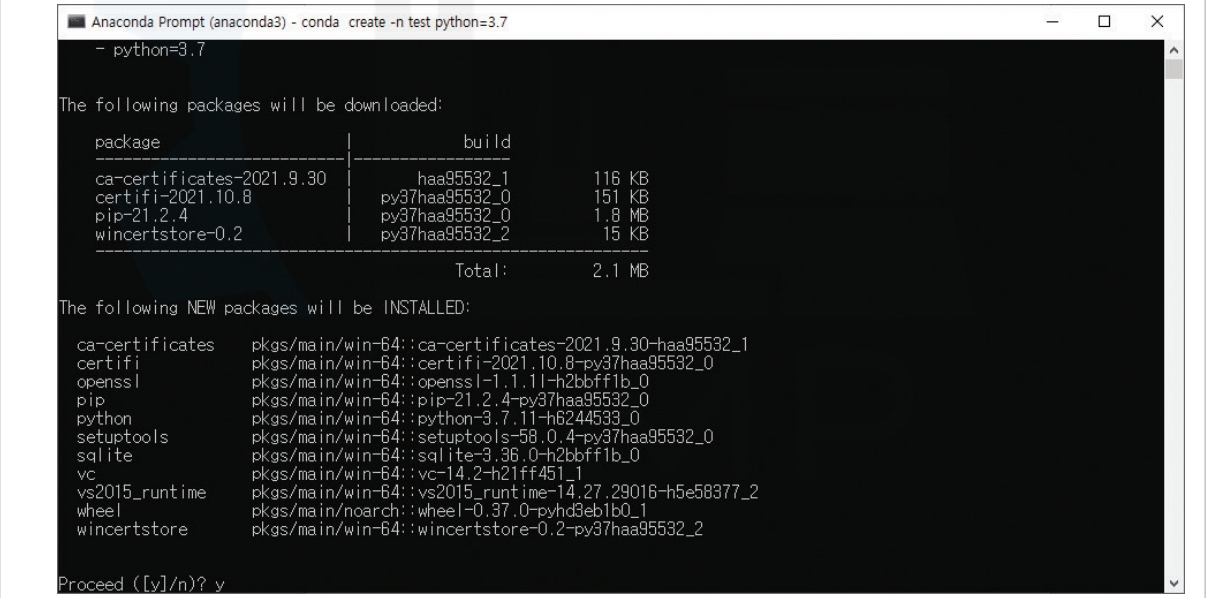


② 가상환경 생성

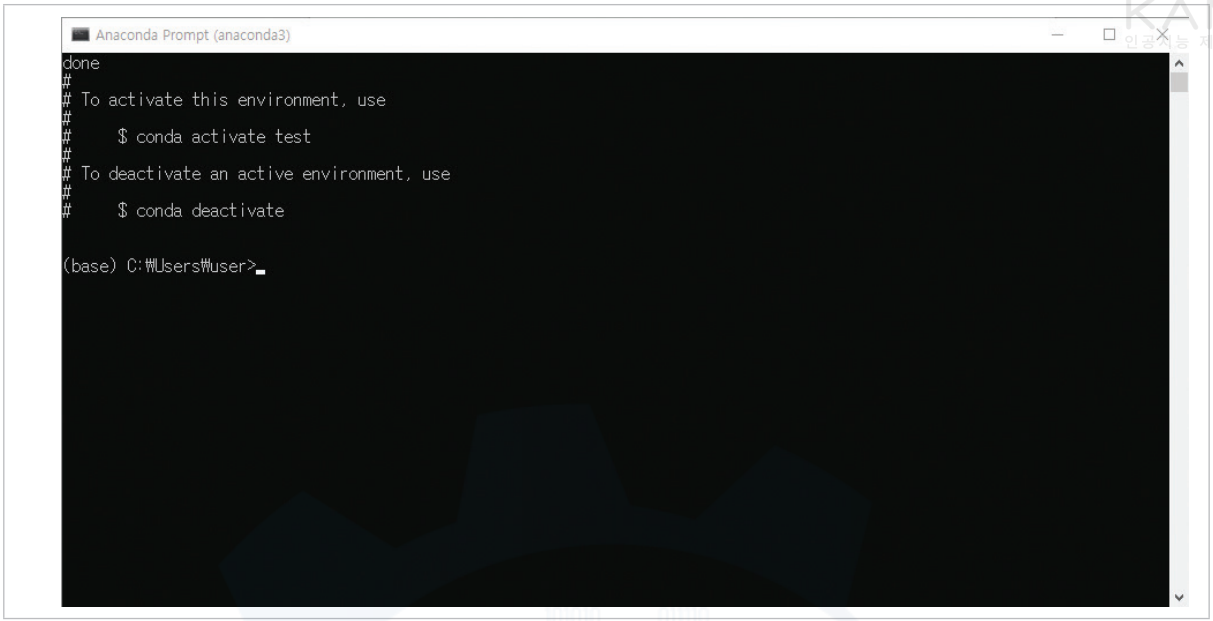
1) ‘conda create -n test python=3.7’ 입력 후 엔터



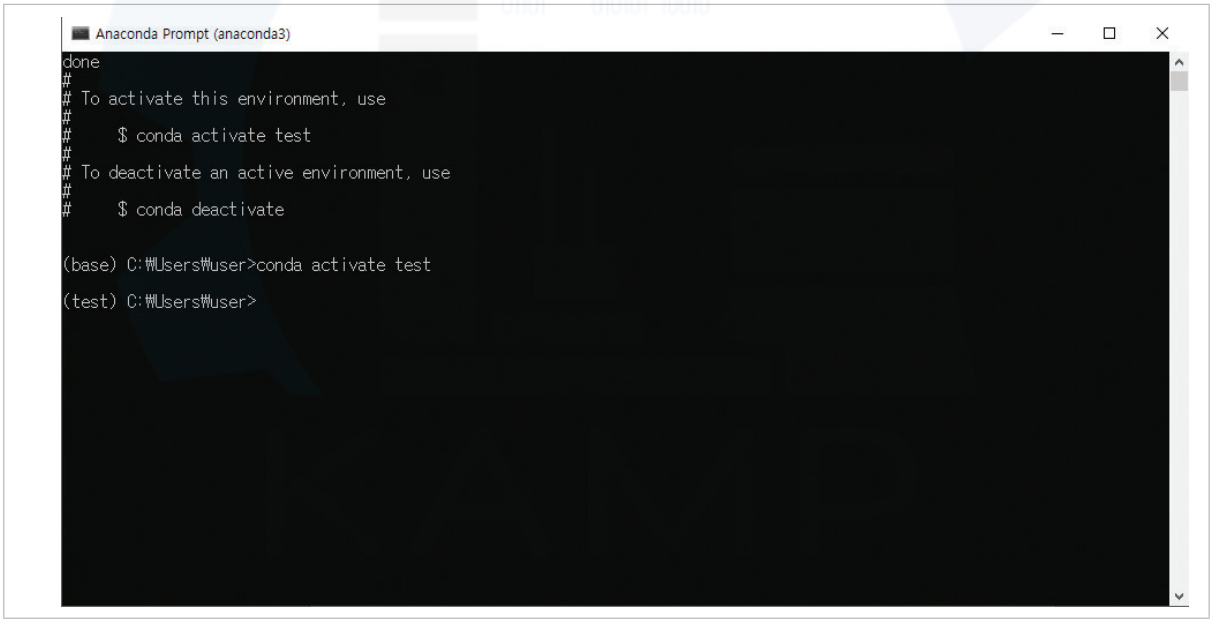
2) ‘y’ 입력 후 엔터



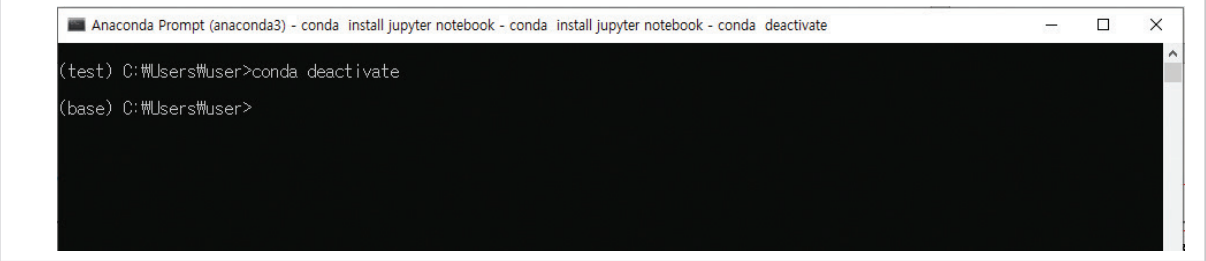
3) 가상환경 생성 완료



4) 가상환경 진입 - ‘conda activate test’ 입력 후 엔터



5) 가상환경 종료 - ‘conda deactivate’ 입력 후 엔터



5. Jupyter Notebook 설정

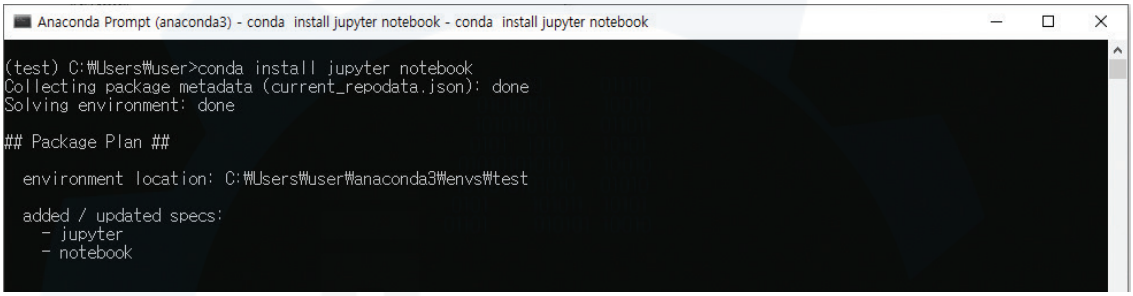
Jupyter Notebook : python 코딩을 할 수 있는 대화형 인터프리터(Interpreter)로 웹 브라우저 환경에서 python 코드를 작성 및 실행할 수 있는 도구이다.

① 가상환경 진입

'conda activate test' 입력 후 엔터

② jupyter notebook 설치

'conda install jupyter notebook' 입력 후 엔터, 중간에 'y' 입력 후 엔터



③ jupyter notebook 테스트

- 'jupyter notebook' 입력 후 엔터

④ jupyter notebook 시작 폴더 설정

1) 'jupyter notebook --generate-config' 입력 후 엔터

2) 출력되는 경로에 진입하여 'jupyter_notebook_config.py' 파일을 메모장으로 열기

3) 메모장에서 '#c.NotebookApp.notebook_dir=''' 문장 찾기

4) 'c.NotebookApp.notebook_dir = folder_directory'로 변경

```
## The directory to use for notebooks and kernels.
#  Default: ''
# c.NotebookApp.notebook_dir = ''
```

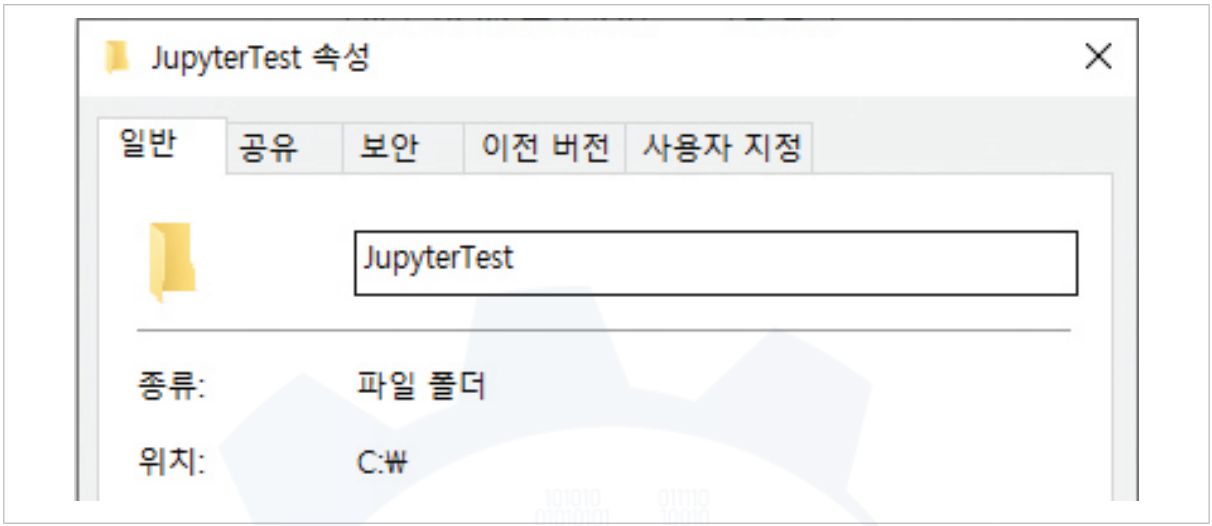
[그림 113] 기존 가상환경 directory code 코드

```
## The directory to use for notebooks and kernels.
#  Default: ''
c.NotebookApp.notebook_dir = 'C:\JupyterTest'
```

[그림 114] 변경된 가상환경 Directory 코드

* folder_directory는 jupyter notebook을 시작하고 저장하는 기본 폴더이며, 이 폴더에 모든 작업물이 저장된다. (시작 전 folder_directory를 해당 위치에 생성시켜줘야 함)

* folder_directory 예시 : 'C:\JupyterTest' (folder_directory는 작은따옴표까지 포함)



5) 변경 완료 후 저장 및 메모장 종료

⑤ 주피터 노트북 커널 추가

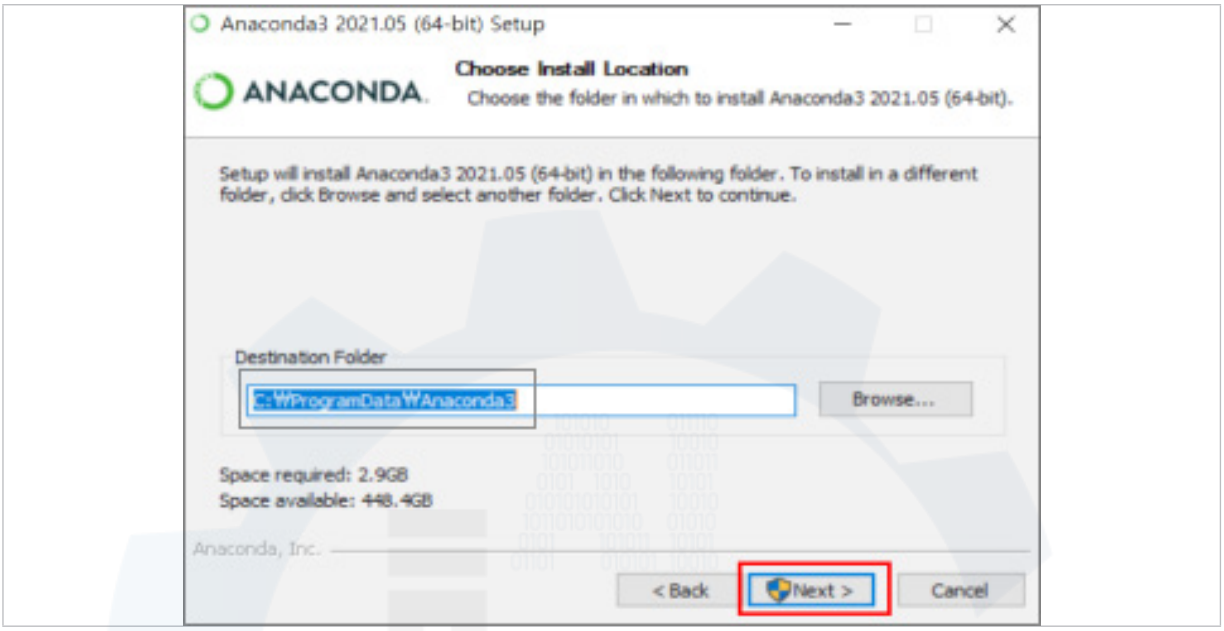
1) 'pip install ipykernel' 입력 후 엔터

2) 'python -m ipykernel install --user --name test --display-name "test"' 입력 후 엔터

6. 오류 발생 케이스

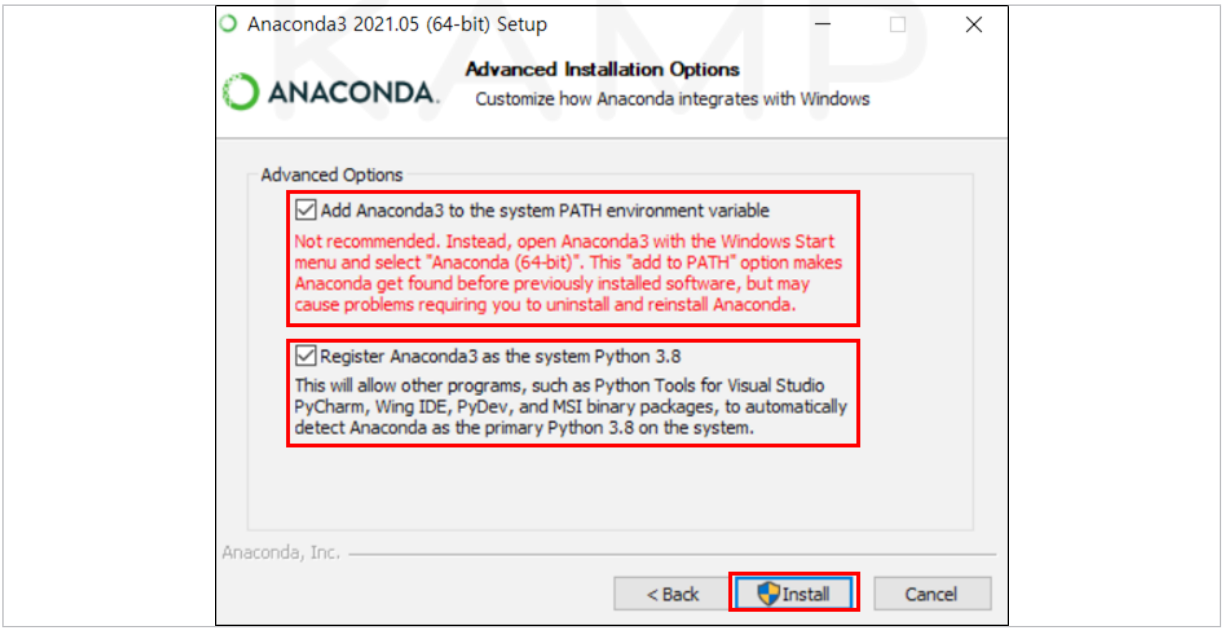
• anaconda 패스 설정 오류

- 아래의 경로를 변경하지 않는 편이 좋다. 모든 모듈은 아래의 베이스를 기본으로 하므로 특이한 사항이 아니라면 아래의 경로를 사용한다.



- 만일 경로를 다르게 지정했다면 보통은 잘 진행된다. 하지만 경우에 따라 모듈을 불러올 때 에러가 발생할 수 있다. 이러한 이슈가 발생한다면 anaconda를 삭제하고 재설치 작업을 진행하는 편이 좋다.

• 'Add Anaconda3 to the system PATH environment variable'의 옵션 선택을 안 한 경우



• 패키지 사이의 버전 충돌, 호환되지 않는 패키지

- 케라스(Keras), 텐서플로(TensorFlow)처럼 타 패키지에 영향을 많이 받는 모듈은 최대한 가상환경을 생성하고 base보다는 가상환경을 생성해서 설치하는 것이 좋다.
- anaconda를 사용한다면 가능한 conda 명령어로 설치(install)하고 관리하는 것을 추천하며, 이유는 아래와 같다.
  - conda 명령어의 경우, anaconda에서 지원하는 패키지만 관리한다.
  - pip 명령어의 경우, python에서 사용 가능한 패키지만 관리한다.
  - 주의할 점은 간혹 같은 패키지이지만 설치 방법이 다른 경우가 있으며, 이들은 서로 호환이 안 될 수도 있다는 점이다.
- conda 명령어로 설치(install)가 되지 않는 경우
  - pip로 설치(install) 시도를 해보거나 그래도 동작하지 않는다면 아래와 같이 진행하면 된다.

    ① 콘솔 창을 연다.

    ② anaconda 경로 아래에 bin이라는 폴더 경로까지 이동한다.

    ③ 'pip install 설치 패키지'로 설치하면 동작하는 것을 확인할 수 있다.

    ④ 즉, 아래와 같이 설치하면 된다.

```
C:₩Users₩user>cd ₩
C:₩>cd C:₩Users₩user₩anaconda3₩bin₩ pip install pandas
```

[콘솔 창에서 pip로 패키지 설치]

4 부록 _ 데이터 품질 전처리 [실습 코드]

앞선 '2.분석 실습'에서 데이터 품질 전처리에 필요한 6가지 특성에 대하여 논의하였다. (완전성, 유일성, 유효성, 일관성, 정확성, 무결성)

▶ **완전성 품질지수**

$$\text{완전성 품질지수} = \left(1 - \frac{\text{결측 데이터의 개수}}{\text{전체 데이터의 개수}}\right) \times 100$$

① 결측 데이터의 값이 30% 이상인 데이터들은 데이터의 완전성을 떨어뜨리기 때문에 각 열(column)에 대한 결측 데이터값의 비율을 확인하여 삭제한다.

```
perc=30
sensor1.isna().sum()/len(sensor1)*100
```

```
time      0.0
normal    0.0
type1     0.0
type2     0.0
type3     0.0
dtype: float64
```

→

```
perc=30
sensor1.isna().sum()/len(sensor1)*100>perc
```

```
time      False
normal    False
type1     False
type2     False
type3     False
dtype: bool
```

[분석 도구를 활용한 완전성 품질지수 확인]

② 총 결측 데이터의 개수를 확인하기 위해 'isnull()' 함수를 사용하여 각 열(column)에서의 결측 데이터 개수를 구한 뒤 'sum()' 함수를 이용하여 합산한다.

```
cmpt_len = sensor1.isnull().sum().sum()
print("결측치 = %d개 \n완전성 지수 : %.2f%% "%(cmpt_len,(1-cmpt_len/len(sensor1))*100))
```

```
결측치 = 0개
완전성 지수 : 100.00%
```

[분석 도구를 활용한 완전성 품질지수 구하기 1]

③ 결측된 데이터들은 실제로 0의 값이므로 0으로 채운 후 그 결과를 확인한다.

```
sensor1 = sensor1.fillna(0)
cmpt_len = sensor1.isnull().sum().sum()
print("결측치 = %d개 \n완전성 지수 : %.2f%% "%(cmpt_len,(1-cmpt_len/len(sensor1))*100))
```

```
결측치 = 0개
완전성 지수 : 100.00%
```

[분석 도구를 활용한 완전성 품질지수 구하기 2]

▶ 유일성 품질지수

$$\text{유일성 품질지수} = \left(\frac{\text{유일한 데이터의 개수}}{\text{전체 데이터의 개수}}\right) \times 100$$

① 데이터에서 유일한 값을 갖는 열(column)을 찾는다. (여기서는 time column)

- 만일 유일한 데이터 값을 갖는 열(column)이 존재하지 않는다면, 몇 개의 열(변수)를 결합한 데이터를 생성하여 유일한 데이터셋을 만들 수 있다.

② 유일한 값을 갖는 열(column)에 대해 'groupby()' 함수를 이용하여, 각 열(column)에서의 유일한 데이터 값을 확인하고, 'size()' 함수를 이용하여 그 개수를 구한다.

```
sensor1.groupby(['time']).size()
```

```
time
0.000000      1
0.000736      1
0.001472      1
0.002208      1
0.002944      1
             ..
139.996768    1
139.997504    1
139.998240    1
139.998976    1
139.999712    1
Length: 190218, dtype: int64
```

[분석 도구를 활용한 유일한 값 확인하기]

③ 구한 유일한 데이터 개수를 이용하여 유일성 품질지수를 구한다.

```
uniq_len=len(sensor1.groupby(['time']).size())
print("유일성 지수 : %.2f%% "%((uniq_len/len(sensor1))*100))
```

```
유일성 지수 : 100.00%
```

[분석 도구를 활용한 유일성 품질지수 구하기]

▶ 유효성 품질지수

$$\text{유효성 품질지수} = \left(\frac{\text{유효성을 만족한 데이터의 개수}}{\text{전체 데이터의 개수}}\right) \times 100$$

① 데이터가 유효 범위 내에 있는가?

- 데이터셋에 정의된 수집 범위를 이용하여 조건(유효 범위)을 모두 충족하는지 확인한다. 다음의 코드로부터 min과 max 값을 확인할 수 있는데, 이 값을 충족하는지 확인한 후 이 범위를 위반하는 데이터가 존재할 시에 경고문을 출력하고 값을 확인한다.

```
nonval_count =0
for col in sensor1.columns:
    print(col+"의 min 값 :",sensor1[col].min(),col+"의 max 값 :",sensor1[col].max())
    col_min = sensor1[col].min()
    col_max = sensor1[col].max()
    tmp_chk =False
    for dc in sensor1[col]:
        tmp_chk = (col_min <=dc)
        if not tmp_chk:
            print("non validation data detected",dc)
            nonval_count+=1
            break
val_rate_1 =  (1-nonval_count/len(sensor1))
```

```
time의 min 값 : 0.0 time의 max 값 : 139.999712
normal의 min 값 : -2.251143 normal의 max 값 : 2.037394
type1의 min 값 : -6.69943 type1의 max 값 : 6.61612
type2의 min 값 : -2.366852 type2의 max 값 : 2.311678
type3의 min 값 : -6.941574 type3의 max 값 : 7.540028
```

* 경고문이 출력되지 않으므로 모든 데이터가 유효 범위 내에 있는 것을 확인할 수 있다.

[분석 도구를 활용한 유효성 품질지수 구하기 (데이터 범위)]

② 데이터가 형식에 맞는가?

- 데이터의 형식을 확인한다.

```
sensor1.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 190218 entries, 0 to 190217
Data columns (total 5 columns):
 #   Column  Non-Null Count   Dtype
---  ------  --------------   -----
 0   time    190218 non-null  float64
 1   normal  190218 non-null  float64
 2   type1   190218 non-null  float64
 3   type2   190218 non-null  float64
 4   type3   190218 non-null  float64
dtypes: float64(5)
memory usage: 7.3 MB
```

[분석 도구를 활용한 유효성 품질지수 구하기 (데이터 형식)]

③ 다음의 코드를 실행하여, ②의 data type을 이용하여 유효성 품질지수를 확인한다.

```
nonval_count =0
for col in sensor1.columns:
    tmp_chk =False
    col_c =''
    for i,dc in enumerate(sensor1[col]):
        if i==0:
            if 'int'in str(type(dc)):
                col_c ='int'
            elif 'float'in str(type(dc)):
                col_c ='float'
        tmp_chk = (col_c in str(type(dc)))
        if not tmp_chk:
            nonval_count+=1
val_rate_2 = (1-nonval_count/len(sensor1))
print((val_rate_1+val_rate_2)/2*100,"%")
```

```
100.0%
```

* 본 데이터는 유효성이 있다고 할 수 있다.

[분석 도구를 활용한 유효성 품질지수 구하기 (데이터 형식 2)]

▶ 일관성 품질지수

$$\text{일관성 품질지수} = \left(\frac{\text{일관성을 만족한 데이터의 개수}}{\text{전체 데이터의 개수}}\right) \times 100$$

- 본 데이터의 구조, 값, 형태가 일관이 있는지 확인한다. 'dtypes()' 함수를 이용하여 전체 데이터의 형태를 확인한다.

```
print(sensor1.dtypes)
nonval_count =0
for col in sensor1.columns:
    tmp_chk =False
    col_c =''
    for i,dc in enumerate(sensor1[col]):
        if i==0:
            if 'int'in str(type(dc)):
                col_c ='int'
            elif 'float'in str(type(dc)):
                col_c ='float'
        tmp_chk = (col_c in str(type(dc)))
        if not tmp_chk:
            nonval_count+=1
val_rate_2 = (1-nonval_count/len(sensor1))
print((val_rate_1+val_rate_2)/2*100,"%")
```

```
time      float64
normal    float64
type1     float64
type2     float64
type3     float64
dtype: object
100.0 %
```

* 본 데이터는 일관성이 있다고 할 수 있다.

[분석 도구를 활용한 데이터의 일관성 확인하기]

▶ **정확성 품질지수**

$$\text{정확성 품질지수} = 1 - \left(\frac{\text{정확성에 위배되는 데이터의 개수}}{\text{전체 데이터의 개수}} \right) \times 100$$

① 데이터의 '평균값'이 수집된 데이터에 한하여, 다음과 같이 정확성 품질 지수를 확인할 수 있다.

② 누락된 데이터가 있는지 확인하기 위해, '*.mean()' 함수를 이용하여 데이터들의 평균을 구한다.

③ '*.mean()' 함수로 구한 데이터의 평균에서 데이터의 '평균값'과의 차이를 구한다.

④ 이 차이가 0이 아니면 데이터를 위배하는 데이터가 존재하게 된다.

* 회전기계 고장유형 AI 데이터셋은 누락된 데이터가 존재하지 않으므로 정확성을 판단하지 않는다.

▶ **무결성 품질지수**

$$\text{무결성 품질지수} = \left\{ 1 - \left(\frac{\text{유일성, 유효성, 일관성 중 100\%가 아닌 지수}}{3} \right) \right\} \times 100$$

- 본 데이터의 유일성, 유효성 그리고 일관성은 모두 100%이다. 따라서 3 가지 지수 모두가 100%를 만족하므로 무결성 품질지수는 100%이다.

▶ **가중치 지수** = 품질 지수 * 가중치

- 데이터의 특성에 따라 품질지수 항목의 중요도는 달라진다. 활용되는 데이터에서 중요한 품질지수가 무엇인지를 평가하고, 데이터의 특성에 따라 적절한 가중치를 부여한다.

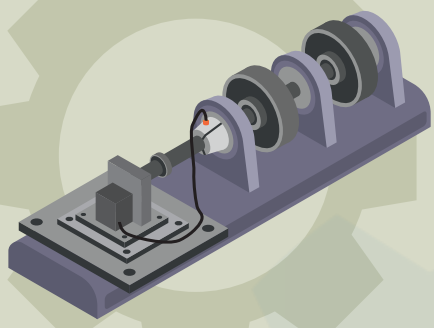
▶ **데이터 품질 지수** = Σ 가중치 지수

- 최종적으로 구한 6가지 데이터 품질지수에 적절한 가중치를 부여하여 품질지수를 구한다.

구 분	품질지수	가중치	가중치지수	오류율
완전성	100%	20%	20%	0%
유일성	100%	0%	0%	0%
유효성	100%	20%	20%	0%
일관성	100%	20%	20%	0%
정확성	100%	20%	20%	0%
무결성	100%	20%	20%	0%
품질지수	100%	100%	100%	0%

[표 3] 데이터 품질지수

「회전기계 고장유형 AI 데이터셋」 분석실습 가이드북



중소벤처기업부 KOSMO 스마트제조혁신추진단 KAIST

34141 대전광역시 유성구 대학로 291 한국과학기술원(KAIST) **T.** (042)350-2114 **F.** (042)350-2210(2220)